推断期学习

从 Test-Time Training 到 Nested Learning

待定

2026 年 4 月 3 日

前言

> "A model is not just a function —it is a learning system operating at multiple timescales."
>
> — 教材编者归纳

本书想回答什么问题

如果一个深度学习模型在训练完成之后被部署上线，它的参数就固定了。面对新的测试数据，它只能做一件事：前向传播。但是，这真的是全部吗?

过去几年的研究正在逐步打破这一直觉。一系列工作表明，在某种意义上，模型可以在推断（inference）阶段继续" 学习"：通过更新隐藏状态、通过注意力机制隐式执行梯度下降、或者通过专门设计的 TTT 层在测试时持续调整内部参数。

本书旨在系统讲述这条演化路径：

Test-Time Training (2020) → 双层优化与元学习 →
快权重与外部记忆 → 线性注意力与 Fast Weights 的统一 →
In-Context Learning 作为隐式优化 → TTT Layers →
Titans 多时间尺度记忆 → Nested Learning 统一视角

本书不是什么

本书**不是**又一本 Transformer 调参指南，也不是系统的统计学习理论教材。读者不需要掌握 PAC 学习、VC 维或 Rademacher 复杂度。所需的数学背景在附录 A 中说明，大致是：线性代数、多元微积分、概率统计、梯度下降。

本书也**不把 Nested Learning 当作已被整个社区确认的终极框架**。Nested Learning 作为最后一章的统一视角被仔细介绍，但全书始终明确区分哪些是机制性共识、哪些是解释视角、哪些是开放问题。

目标读者

本书面向计算机科学或相关专业的高年级本科生，要求有使用 PyTorch 等框架进行基本模型训练的经验，了解梯度下降和基本的前馈神经网络结构。

致谢

（待补充）

如何阅读本书

章节结构

每一章都围绕**一个核心问题**组织，而不是从论文名词出发。每章的固定结构如下：

1. **本章想回答什么问题** —用一句话点明本章的核心驱动问题
2. **最小例子** —用最简单的具体情境引入
3. **直觉解释** —以自然语言说清楚机制
4. **数学形式化** —给出最低必要的数学
5. **代表论文** —如何读懂关键文献
6. **和前后章节的关系** —明确位置
7. **动手实验** —可以在 ≤ 200 行代码内完成的小实验
8. **常见误解** —学生最容易犯的错误
9. **练习题** —至少 3 道
10. **本章小结** —核心要点

两个贯穿全书的玩具问题

为了保持叙事连贯性，全书反复使用以下两个极简模型：

玩具问题

玩具问题 A：在线线性回归 / 联想记忆
给定流式输入 (x_t, y_t)，目标是学习线性映射 $\hat{y} = Wx$。这个问题可以用来讲 ICL、梯度下降等价、TTT 更新、在线优化。

玩具问题

玩具问题 B：键值检索 / MQAR
给定键值对 (k_i, v_i) 序列，测试时给出查询 q，要求返回匹配的 v_i。这个问题可以用来讲 Attention、Linear Attention、Fast Weights、Titans 的记忆能力。

学术诚实约定

本书在涉及具有争议性的机制解释时，会明确使用以下标记：

- **定理/定义**：经过严格数学证明的结论
- **视角**：一种富有启发性的解释，有一定实验支撑，但不是严格等价
- **开放问题**：仍在研究中，不应作为定论引用

如何阅读本书的神经科学类比?

> “类比是理解的脚手架，不是理解本身。建完大楼，脚手架应当可以拆除。”
>
> — 本书写作方针

在阅读本书时，你会频繁遇到来自脑科学与神经科学的类比。本指南明确说明：**类比用来帮助理解，不是用来代替定义和推导**。请在阅读第一章之前通读此页。

一、为什么大量使用神经科学类比?

本书的核心对象——测试时训练、快权重、多时间尺度记忆系统、嵌套优化——在纯数学语言中是精确但抽象的。神经科学类比的作用是：**提供每个机制的”计算直觉”**，让读者在还没有完全消化符号之前，就能形成一个关于” 这件事在做什么” 的粗糙但正确的图景。

具体来说，类比帮助读者建立以下直觉：

1. **计算角色直觉**：这个模型组件” 在做什么”?（例如：某模块在做类似工作记忆的事——短暂持有当前上下文，用完即丢）

2. **时间尺度直觉**：为什么有些东西快变、有些慢变?（例如：快权重对应短时突触可塑性，慢权重对应需要大量重复的皮层巩固）

3. **设计动机直觉**：为什么架构要这样分层?（例如：Titans 的长短记忆分离，可以用互补学习系统来理解设计意图）

二、类比会在哪里失效?

类比不是机制等同。大脑和人工神经网络在以下层面有根本差异：

差异维度	大脑／生物系统	人工模型
学习信号来源	局部突触活动、神经递质（多巴胺等）、时序相关性（STDP）	显式损失函数 + 全局反向传播梯度
更新规则	Hebbian、BCM、STDP——局部、噪声大、无全局目标	精确可微的矩阵运算，有明确损失目标
能量与结构约束	受代谢代价、解剖结构、神经元兴奋性约束	仅受算法设计和硬件约束
时间步的含义	”一步”对应的是皮层震荡周期、突触延迟等生理过程	”一步”是一个 token 或一次 forward pass，与任何生理时间无直接对应
记忆系统的定位	海马体、杏仁核、前额皮层等有明确解剖位置	模型组件是参数矩阵，没有空间定位

三、类比的四个层次——相似性分级阅读法

本书严格区分以下四种程度的相似性。遇到类比时，请辨认它属于哪一层：

1. **计算角色相似**（最宽泛）：两者都在执行相同的抽象功能。例：模型的 Attention 和生物的选择性注意，都在做”权重分配式检索”。
2. **算法组织相似**：两者都有”快变量 vs 慢变量”、”在线更新 vs 长期整合”的区分。例：Titans 长短记忆分离和 CLS 理论中海马体/新皮质的分工。
3. **实现机制不同**（几乎总是不同）：即使功能相似，具体实现完全不同。例：Titans 的 Surprise 门控是一个精确的标量数学运算；生物系统的突触选择性强化涉及多种神经递质的级联反应，不可简化为一条数学公式。
4. **学习信号不同**（几乎总是不同）：模型依赖显式损失函数和梯度；生物系统依赖局部协同活动、化学调制信号。

本书的类比通常只到第 1–2 层成立。第 3–4 层几乎总是差异。

四、如何阅读”神经科学直觉”盒子和”类比边界”盒子

全书每个关键章节都有两个显式的配对盒子：

盒子	阅读目标
神经科学直觉	理解当前机制的计算功能类比；建立直觉
类比边界与失效条件	了解类比在哪里停止成立；防止把比喻当结论

这两个盒子必须配对阅读。只读”神经科学直觉”盒子而跳过”类比边界”盒子，会导致严重的概念混淆。

五、你绝对不能从本书类比推出的结论

1. **”这个模型模块等价于这块脑区”**
   模型组件的功能相似不等于实现等同。TTT-Layer 的内循环更新不等于突触可塑性，只是在”在线适应”这个计算角色上有相似。

2. **”我们的模型已经解释了大脑如何工作”**
   恰恰相反。本书的任何架构，相比于大脑在多种神经递质、异步耦合网络、生化约束下同时工作的复杂度，都是极度简化的近似。

3. **”从大脑原理可以直接推导出这个模型的正确性”**
   模型的正确性由数学推导和实验结果保证，不由生物类比保证。

4. **”既然类比成立，生物系统也在解决同一优化问题”**
   生物系统没有明确的全局损失函数，没有反向传播。”解决类似的功能问题”不等于”执行相同的算法”。

六、本书写作方针的一句话总结

用严格的形式化保证真，用清楚的解释保证懂，
用神经科学类比保证直觉，但永远不把类比偷换成结论。

阅读全书时，请将类比视为**脚手架**：它帮助你在理解数学公式的过程中保持方向感。一旦你理解了形式化对象本身，脚手架就可以拆除。数学是大楼，类比是脚手架——不要把脚手架住进去。

目录

前言 **ii**

如何阅读本书 **iv**

如何阅读本书的神经科学类比? **vi**

第一章 引言：重新思考“训练”与“测试”的边界 **1**

1.1 本章想回答什么问题 1
1.2 最小例子：面对不断变轨的小球 2
1.3 直觉解释：静态字典 vs. 活着的大脑 2
1.4 数学形式化与核心术语 3
1.4.1 时间轴的区分 3
1.4.2 参数空间的切割 3
1.4.3 普通推断 vs. 自适应推断的基础方程异同 4
1.5 两条 TTT 线索的本质差异 4
1.6 术语对照表 5
1.7 时间尺度与符号总览 5
1.8 后续章节中反复出现的主题 6
1.9 代表文献指路 6
1.10 和后序章节的关系 7
1.11 练习题 8
1.12 本章小结 8

第二章 为什么模型能在测试时学习? **10**

2.1 本章想回答什么问题 10
2.2 最小例子：旋转预测作为自监督信号 11
2.3 直觉解释：双头共享表征架构 11
2.4 数学形式化：TTT 2020 算法 12
2.4.1 预训练阶段 12
2.4.2 测试时适应阶段 12
2.5 为什么辅助任务能帮助主任务? 13

2.6 计算代价分析与工程局限 13
2.7 神经科学直觉与类比边界 14
2.8 代表论文精读 15
2.9 和前后章节的关系 15
2.10 实验 16
2.11 常见误解 17
2.12 练习题 17
2.13 本章小结 18

第三章 双层优化与元学习：让参数初始化点”内行” 19
3.1 本章想回答什么问题 19
3.2 什么是元学习：用学习自行车类比 19
3.3 数学设定：任务分布与情节式学习 20
3.4 双层优化的正式框架 21
3.5 MAML 算法：优化初始化点 21
3.6 二阶导数：Hessian 项的来源与一阶近似 22
3.7 Worked Example：双层优化的几何直觉 23
3.8 内外循环语言：全书的统一语法 24
3.9 Learned Optimizers：学习优化器本身 24
3.10 和前后章节的关系 25
3.11 神经科学直觉与类比边界 27
3.12 练习题 29
3.13 本章小结 29

第四章 快权重、外部记忆与时间尺度 31
4.1 本章想回答什么问题 31
4.2 最小例子：从查找表到连续向量的模糊检索 32
4.3 外积更新规则的直觉与推导 32
4.4 慢权重 vs. 快权重：两个时间尺度的正式定义 35
4.5 快权重写入：一个完整的 Worked Example 35
4.6 干扰分析：当键不正交时会发生什么 36
4.7 慢权重学习什么：读写参数 (W_K, W_V, W_Q) 37
4.8 Ba et al. (2016) 的快权重注意器 37
4.9 HyperNetworks：快权重的参数生成推广 38
4.10 连接 Memory-Augmented Neural Networks 39
4.11 和前后章节的关系 39
4.12 练习题 41
4.13 本章小结 41

第五章 Attention、Linear Attention 与递推视角 43
5.1 本章想回答什么问题 43
5.2 前置概念：内积、核函数与相似度度量 43
5.3 标准 Softmax 注意力的推导与理解 44
5.4 从 Softmax 到线性注意力：去掉归一化的代价与收益 45
5.5 线性注意力 = 快权重：严格的数学等价 47
5.6 带遗忘因子的线性注意力与衰减位置偏置 48
5.7 Worked Example：用线性注意力做 MQAR 48
5.8 标准注意力为什么不能被递推 49
5.9 代表论文精读指南 49
5.10 和前后章节的关系 50
5.11 神经科学直觉与类比边界 51
5.12 练习题 52
5.13 本章小结 53

第六章 上下文学习（ICL）到底是不是学习? 54
6.1 本章想回答什么问题 54
6.2 最小例子：在上下文中做线性回归 55
6.3 直觉解释：从归纳头到梯度步 55
6.3.1 感性解释 1：归纳头（Induction Heads） 55
6.3.2 感性解释 2：一步梯度下降的构想 55
6.4 数学形式化：一层线性注意力等价于一步梯度下降 56
6.5 代表论文精读 58
6.6 三层证据体系：ICL 的科学地位 59
6.7 等价性在哪里失效 60
6.8 “学习” 在这里到底指什么 61
6.9 和前后章节的关系 62
6.10 神经科学直觉与类比边界 63
6.11 练习题 64
6.12 本章小结 65

第七章 Learning to (Learn at Test Time)：通过架构学习微调 66
7.1 本章想回答什么问题 66
7.2 最小例子：在序列中背诵密码 67
7.3 直觉解释：模型内部的俄罗斯套娃 67
7.4 数学形式化：把隐状态作为学习器参数 68
7.4.1 标准序列结构的隐藏层 68
7.4.2 隐状态升级为一个微型模型 68
7.5 代表论文精读 72
7.6 和前后章节的关系 72

7.7 神经科学直觉与类比边界 74
7.8 练习题 75
7.9 本章小结 75

第八章 TTT Layers：隐藏状态就是学习器 77
8.1 本章想回答什么问题 77
8.2 最小例子：需要不断刷新规则的过滤筛选 78
8.3 直觉解释：为什么是“就是学习器” 78
8.4 数学形式化：从 TTT-Linear 到 TTT-MLP 79
8.4.1 TTT-Linear 的规范方程 79
8.4.2 向高容量跃迁：TTT-MLP 79
8.5 代表论文精读与发展 81
8.6 与经典序列模型的系统对比 81
8.7 工程实现：从理论到硬件 82
8.8 和前后章节的关系 83
8.9 练习题 84
8.10 本章小结 85

第九章 Titans 与多时间尺度记忆：如何在长河中钉住一根针 86
9.1 本章想回答什么问题 86
9.2 最小例子：大海捞针 87
9.3 直觉解释：三种记忆的分工 87
9.4 数学形式化：惊讶度门控写入 88
9.5 Worked Example：三步追踪 Surprise 的行为 90
9.6 与前后章的关系 90
9.7 代表论文 91
9.8 神经科学直觉与类比边界 91
9.9 Lab 9：手写 Surprise 门控并观察遗忘-保留权衡 93
9.10 常见误解 93
9.11 练习题 94
9.12 本章小结 94

第十章 统一视角：Test-Time Regression 与联想记忆 96
10.1 本章想回答什么问题 96
10.2 最小例子：在线联想回忆的统一性 97
10.3 直觉解释：优化的物理实体化 97
10.4 数学形式化：Test-Time Regression 98
10.4.1 基石定义 98
10.4.2 闭式解：Recursive Least Squares (RLS) 98
10.4.3 一阶近似：在线 SGD 99

10.4.4 极端截断：忽略预测反馈 99
10.5 统一表格：所有模型作为 TTR 特例 100
10.6 Worked Example：从 TTR 推导 TTT-Linear 101
10.7 神经科学直觉与类比边界 102
10.8 这个统一视角的边界 104
10.9 代表论文精读 104
10.10 和前后章节的关系 105
10.11 实验 105
10.12 常见误解 106
10.13 练习题 106
10.14 本章小结 106

第十一章 Nested Learning：当架构本身就是优化 108
11.1 本章想回答什么问题 108
11.2 最小例子：三层嵌套的回归 109
11.3 直觉解释：为什么“架构”可能只是一种截断 110
11.4 数学形式化：L 层嵌套优化 110
11.5 统一：把前面所有模型归约为 Nested Learning 的特例 112
11.6 共识与解释：必须澄清的学术边界 112
11.7 Nested Learning 尚未证明什么 114
11.8 为什么本章是一个综合，而不是一张许可证 114
11.9 代表论文 115
11.10 神经科学直觉与类比边界 115
11.11 和前后章节的关系 117
11.12 Lab 11：手写三层嵌套优化并测量截断效应 117
11.13 常见误解 118
11.14 练习题 119
11.15 本章小结 119

最低必要数学背景 121
.1 线性代数核心 121
.1.1 外积（Outer Product） 121
.1.2 矩阵的 Frobenius 范数 121
.2 最小二乘与岭回归 122
.2.1 普通最小二乘（OLS） 122
.2.2 岭回归（Ridge Regression） 122
.3 递归最小二乘（RLS） 122
.3.1 推导思路 122
.3.2 RLS 更新公式 123
.4 Sherman-Morrison 公式 123

.5 矩阵微积分基础 123
.5.1 标量对矩阵的梯度 123
.5.2 链式法则（矩阵版本） 124
.5.3 Jacobian 与 Hessian 124
.5.4 预处理（Preconditioning） 124
.6 在线学习与遗憾（Regret） 124
.6.1 设定 124
.6.2 累积遗憾 125
.6.3 Worked Example：Follow-the-Leader 125
.7 常用符号对照表 125

Lab 汇总与动手实验指南 **126**
.8 环境准备 126
.9 Lab 索引 126
.10 Lab 设计原则 126
.11 各 Lab 详细指南 127
.11.1 Lab 1：静态参数模型的 OOD 脆弱性 127
.11.2 Lab 2：TTT 2020 自监督旋转预测 127
.11.3 Lab 3：MAML 正弦波回归 127
.11.4 Lab 4：外积联想记忆 128
.11.5 Lab 5：线性注意力与 Softmax 注意力对比 128
.11.6 Lab 6：ICL 的隐式梯度下降验证 128
.11.7 Lab 7：TTT-RNN 单元 128
.11.8 Lab 8：TTT-Linear vs GRU 遗忘对比 129
.11.9 Lab 9：Needle-in-Haystack 混合记忆 129
.11.10 Lab 10：在线 OLS 与 TTT-Linear 对比 129
.11.11 Lab 11：Nested Learning 对应表 129

术语、符号与神经科学对照表 **130**
.12 核心数学符号 130
.13 机器学习术语 vs. 神经科学类比术语对照 131
.14 容易混同但绝不能混同的概念 135
.15 学术诚实申明 136

参考文献 **137**

第一章　引言：重新思考“训练”与“测试”的边界

> “The dichotomy between training and testing is an artificial artifact of how we build algorithms, not a reflection of how intelligent systems actually operate in the world.”
>
> — 引言

1.1　本章想回答什么问题

在传统的深度学习教育中，我们被灌输了一个极其极其根深蒂固的概念：**训练与测试是相互独立的两个阶段**。

- **训练阶段**：需要大量的数据、GPU 集群、耗时数周的反向传播计算。这时模型的参数 θ 在不断变化。

- **测试/推理阶段**：模型被冻结了，参数 θ 锁定不变。模型被发往生产环境执行前向预测。不管在这个阶段模型犯了多少荒谬的错误，或者见到了多少明显的新规律，它都不会有丝毫长进。

然而，真实世界的数据分布永远在流动（Distribution Shift）。这种被冻结的“死”模型常常会在遇到一点点没有见过的输入时全盘崩溃。本章，乃至本书想要回答的一个核心问题是：

> 为什么我们在测试（推断）时不能也让模型继续学习和适应？在极端的算力与架构重塑下，我们能否打破这一道被视为经典的铁丝网，让“测试时”本身就是“训练时”？

我们将从基础的术语定义开始，引入本书反复提及的基石问题与数学环境，并帮助读者在脑海中建立对动态自适应系统的生物学直觉。

1.2 最小例子：面对不断变轨的小球

玩具问题 A：预测弹跳路径分布的偏移

假设我们训练了一个简单的模型来预测桌球的弹跳轨迹（回归问题）。在训练集中，桌台的摩擦系数是确定的 $f = 0.1$。模型在预测试阶段完美学会了给出下三个时间点的位置 (x, y)。

部署（测试）时：小桌台的布面被替换了，摩擦系数隐性变成了 $f = 0.5$（发生了分布偏移 Distribution Shift）。

- **传统僵化参数模型**：看到球弹出后，仍旧按照 $f = 0.1$ 的物理公式去推断，每次都会预测小球滑得更远。尽管模型“看到”了球实际上停在更近的地方，它不具备参数更改能力，下一次依然预测错误。
- **测试时适应系统（Test-Time Adaptive System）**：在第一根球的轨迹发生后，它将**本次不完美的输出预测和观测的偏差**作为短效损失，在微秒内部就对模型做了一次瞬时的浅层（或隐式）再调整。到第三根球时，它已经隐约知道需要“把预测缩短”，从而适应了新桌布。

这就是在测试时实施学习的最原始直觉展现。

1.3 直觉解释：静态字典 vs. 活着的大脑

要理解后续将要介绍的所有前沿架构，首先我们需要剥离对“参数就是绝对真理”的执念。

在常规范式下，我们训练大语言模型就好比是在印刷一本包罗万象的厚重大字典（预训练）。字典印完之后（进入测试），你来查询任何东西，它都是以固定页面结构直接反射回去的。

而测试时学习的思想是：预训练不应当仅仅学习“字典固有的页面内容（参数）”，更多的是要学习去搭建一个“能在被提问时立刻用身边的草稿纸结合你的上文去打个草稿解题的小助理机制”。你需要的不是去读一本死书，而是在提供一种可供在线校准的思考架构引擎。

神经科学直觉：选择性注意与快速适应

在神经科学中，这种现象非常常见。即使是脑部已经完成了“基本常识结构发育”（长期预训练），人类在进入一个全然陌生的暗室时：

- **计算角色上的相似**：由于光线的改变，人的视觉皮层并不需要回到“娘胎里”（重新进入漫长的预训练中心）重组架构，而是在最初的几秒适应内利用局部的误差预期，迅速改变当前处理通道的敏感电位（在线

快速适应）。

- **工作记忆分配**：遇到长上下文的理解任务（例如听长篇演讲），大脑的前额叶皮层拥有选择性注意（Selective Attention）能够临时把某一句重要的话“贴在脑前”（推断阶段的局部存储与计算），而非把它写死到长达十年寿命的长期记忆神经元里去。

测试时间的学习，本质就是在计算机工程中去唤醒这种本该存在的“现场试错适应”（Sensorimotor calibration）以及短时间尺度的工作缓存运算能力。

这并不是说“模型就是大脑”

虽然从直觉上我们可以把“在测试环节自我纠偏”类比为大脑的临场适应反应，但你绝不能因此建立以下推论：

- **实现机制上的云泥之别**：人工模型依赖的是计算出精确的标量损失，进而利用明确的数学雅可比求导链甚至二阶反置矩阵实施“向回退”的精确修正同步。真实大脑在类似短时调整中不依靠标准的反向传播，而是大量利用局部电位扰动和多模式神经递质（像是多巴胺漫灌带来的时标突触可变）发生。
- 我们在人工智能里探讨的设计是对人类这种“具有多个反馈速度环”功能的**数学粗糙模拟仿化**，而不是对大脑生理器官的数字重现！它只是在“功能组织目的性”上有高度一致，但在数学解剖下是两种截然不同的演化路线。

1.4 数学形式化与核心术语

为了整本书的讨论同出一辙，这里严格定义我们的基础场景与数学标记。

1.4.1 时间轴的区分

- t_{train}：指代主模型被喂养海量人类通用数据集以定型宏观静态参数的过程。（俗称：外循环，预训练）。
- t_{test} 或 t：模型被部署，面对特定任务时的单个序列推移阶段（俗称：内循环，推断时间步骤）。

1.4.2 参数空间的切割

我们将参数概念做了彻底解绑：

- **慢权重（Slow Weights / Global Parameters）** θ：在 t_{train} 通过 SGD/Adam 从巨大训练集中极其缓慢地固定下来。在推演 t_{test} 时理论上被冻结了不会发生显存重写式变化，它们承载**全局常识与提取规则**。

- **快权重 / 隐状态参数（Fast Weights / Inner State）** $\boldsymbol{S}_t$：仅在这个特定序列推演的 t_{test} 开始时由零初构，在推算过程的毫秒间隙受输入数据影响实时疯狂改写。在整个推理序列结束时被直接从内存抛弃删除！它们承载**本地适应与特定任务偏好**。

1.4.3 普通推断 vs. 自适应推断的基础方程异同

普通序列推断的函数定义为在给定慢权重 θ 后：

$$y_t = f(x_t; \theta, \boldsymbol{S}_{t-1}) \quad \text{其中} \quad \boldsymbol{S}_t = g(x_t, \boldsymbol{S}_{t-1}; \theta) \tag{1.1}$$

在这里隐藏状态 $\boldsymbol{S}$ 的更新 g 是一个在设定死的一组逻辑下的普通变换组合（如加减拼接）。

而测试时学习体系下：

$$\boldsymbol{S}_t \leftarrow \text{Optimizer}_\eta(\boldsymbol{S}_{t-1}, \mathcal{L}_{\text{inner}}(x_t; \boldsymbol{S}_{t-1}, \theta)) \tag{1.2}$$

这里的 $\boldsymbol{S}_{t-1}$ 被视为一组被接受优化迭代的小型网络内部参数组。$\mathcal{L}_{\text{inner}}$ 被定义为某个无监督或是由于上下文自然生成出的误差拟合源项！注意：它是求极值反向退算的优化过程，而不是固定前向！

1.5 两条 TTT 线索的本质差异

第一条线索：TTT 2020 风格（外科手术式微调）

Sun et al. (2020) 提出的”Test-Time Training” 指的是：当测试样本 x_{test} 到来时，利用一个**自监督辅助任务**对网络参数 θ 执行真实梯度下降更新，然后再做主任务预测。

输入测试样本 → 自监督微调（修改 θ） → 主任务预测

关键点：这需要调用标准的反向传播（`.backward()`）。每处理一个测试样本，都要运行一次或多次梯度步骤。这种方法**真实地改变了模型的持久化参数**。这就是第 2 层更新。

第二条线索：TTT-Layer 2024 风格（内嵌计算式更新）

Sun et al. (2024a) 以及相关工作（见第 八 章）把"TTT" 重新定义为：将序列模型的**隐藏状态** s_t 本身看作一个微型学习器的参数，在前向传播过程中，每个新 token 到来时就执行一次基于状态的梯度步骤。

$$\text{前一隐藏状态 } s_{t-1} \to \text{输入 } x_t \to \text{内部梯度步骤（更新 } s_t\text{）} \to \text{输出 } y_t$$

关键点：*整个过程都在前向传播内完成*。没有外部的 `.backward()` 调用。模型的**持久化参数** θ **不变**；变化的是每次推断的隐状态。这就是第 1 层更新。

1.6 术语对照表

本书涉及多个研究社区的术语，它们有时指代相同的概念。以下对照表帮助读者在不同章节之间保持术语一致。

术语	同义词 / 近义词	本书中的含义
预训练（Pretraining）	外循环训练	在大规模数据集上学习 θ
微调（Fine-tuning）	适应（Adaptation）	在目标任务上进一步优化 θ
测试时训练 / TTT	测试时适应	在推断时对参数或状态执行优化步
上下文学习（ICL）	In-context learning	仅通过前向传播利用上下文示例
内循环（Inner loop）	快适应、测试时更新	对 $\boldsymbol{S}_t$ 或 θ 的序列级更新
外循环（Outer loop）	元训练、预训练	对 θ 的数据集级更新
慢权重（Slow weights）	全局参数 θ	推断时冻结的持久化参数
快权重（Fast weights）	隐状态 $\boldsymbol{S}_t$	推断时实时更新的临时状态

1.7 时间尺度与符号总览

本书的核心论点是："学习" 发生在多个时间尺度上。以下表格总结了不同时间尺度对应的变量和章节。

时间尺度	符号	更新频率	首次出现
最慢：预训练	θ	整个数据集（数周）	本章
中等：元适应	η, W_K, W_V	任务级别	第 三 章
快：逐 token	$\boldsymbol{S}_t$	每个时间步	第 四 章
最快：窗口注意力	KV cache	窗口内的精细检索	第 九 章

1.8 后续章节中反复出现的主题

以下概念将在全书 11 章中反复出现。在这里先建立预期，后续遇到时可以回溯本节。

1. **外积更新**（$vk^\top$）：第 四 章引入，在第 五、六、十 章反复出现。是联想记忆、梯度步、快权重更新的统一数学形式。

2. **双层优化**（内循环 + 外循环）：第 三 章引入，在第 七、十一 章推广为多层嵌套。

3. **“查询-键-值” 三元组**（q, k, v）：第 五 章引入，贯穿所有后续章节。

4. **“隐状态是参数” 的观点**：第 七 章首次出现，在第 八、九、十一 章中成为核心架构设计原则。

5. **“机制等价 vs. 解释性视角” 的区分**：第 六 章引入，在第 十、十一 章成为学术诚信的核心议题。

1.9 代表文献指路

由于本章为总计概念梳理引言。我们将本教材引用的整个领域发展线，切分为三部曲，后附关键奠基文章：

1. **概念开局与人工修剪期**（涉及第 2,3,4 章）：在发现普通模型极其脆弱后，研究者直接在推断流程前用代码加入一个小的微调。Sun et al. (2020)《Test-time training with self-supervision for generalization under distribution shifts》是这一领域的正式定命之作。

2. **架构内化隐式化期**（涉及第 5,6 章）：探索为什么无需人工手改代码而仅凭多层大结构的自然前向传递本身，竟然神奇地实现了类似在线训练的效果（即 ICL 本质证明），如 Oswald et al. (2023a)；以及使用快权重代替显式反算的起步。

3. **全面内部结构重铸期**（涉及第 7 至 11 章）：将大模型底层进行大修，把 RNN、注意力全部整合为“自包含隐状态回归层”，如 Sun et al. (2024a) 的 TTT-layers，直到向终极嵌套多阶系统演化迈进。

1.10 和后序章节的关系

本章负责确立马步桩台。理解了“θ 产生寻找适应梯度的能力”和“$\boldsymbol{S}_t$ 被适应梯度去改写自身”的双元区别之后，紧接的第 二 章就会直接从经典的计算机视觉例子去直观解释当分布（比如下雨天的模糊图像导致与训练严重不符）开始脱轨时，我们在测试点现场强行加几步优化梯度的具体实施方式以及其力挽狂澜的作用。

Lab 1: 验证静态参数大模型在简单位移下有多“脆弱”

目标：手动编写脚本并观察纯固定参数在遭遇 OOD (Out-of-Distribution) 未见系统时的预测灾难。并实施一次最简单的“测试补偿”。

1. 构建一个两层的全连接神经网络用于给二维坐标打点做分类。分布是 $y = 2x$ 这条斜线上下的分离。
2. **训练期**：数据全为 $[-1, 1]$ 内取样，使用 PyTorch 训练直到损失 < 0.01；**切断**优化器更新机制（`eval()` 态）。
3. **测试期**：开始送入 $+50$ 整体位移后的 $[49, 51]$ 特征域空间的新样本点。
4. **观察**：虽然线性关联未变，但是由于未见过高分位数范围值，静态的分类网络前向非线性激活发生过激失真，原本完全正确的判断预测输出全部降到了近乎 50% 的瞎猜状态。
5. **补偿（实验核心）**：如果在部署阶段，我们将这批新数据自己和自己构成的均值或进行一个简单的自适应自编码重构误差（人工给一个无监督 Loss），然后仅对预测层前进行 **1 次 SGD 现场反向推正**。再去分类打标。你会发现在毫秒之间，精度又拉回了 98%。

这就是本书致力于追求去系统化的、发生于推测期的核心魅力。

常见误解

常见误解 1：测试时的训练是在给大字典加加页数扩容。

纠正：不！这是典型的死记硬背理解。模型在推断时的学习操作更多地是为了**提取关联与抑制主预训练库中的无关特征**，是将特定数据的模式（比如当前文章的写作风格）暂时置顶提升的在线优化工作，它处理过后立刻就会随着当前提问消失。这绝对不是说把这次用户的提问永久固化进去了大模型的

基座文件池。

常见误解

常见误解 2：我们现在常用的 ChatGPT 就是拥有这种不断根据我的对话进行内部极速梯度改写本领才看起来这么聪明的。

纠正：在撰写这本教材的绝大多数前线工业模型（GPT-4 时代等）并**没有**采用内化的测试梯度层（TTT 系列还在学术至工业的飞速跨越期中）。大多数线上产品能针对你的语境显得“能自我调节”，全靠极其暴力的 Transformer 并发超长缓存检索历史序列硬生生建立多维组合（即第 6 章讲的隐式 ICL），它们并不是去更改什么底层态参数本身进行的，这也正是极其昂贵并在将来必定需要通过“参数层即是学习者”演变所必然取代的源头。

1.11 练习题

练习 1.1. 概念对位理解：请用三句话分别解释以下概念并且指出它们产生及作用的阶段：(1) 全局慢权重 θ；(2) 分布偏移 (Distribution Shift)；(3) 工作记忆与隐藏态学习器。并明确标出哪一个概念更贴近于在传统预训练（Pre-train）中所侧重的。

练习 1.2. 数学定义的扩展：如果在上面数学形式化章节中，针对公式 g 更新隐态所使用的损失函数 $\mathcal{L}_{\text{inner}}$ 被彻底置零了，上述方程会不可避免地退化为什么？而此时如果还需要根据输入数据维持不同的回答可能性输出，只能借由让网络外部变态级加大哪一项去进行对过去的死记硬背？这体现了什么代价的权衡（Trade-off）？

练习 1.3. 关于真实应用成本：如果我们如上述玩具问题中的小球或者位移打标情况，每一次都必须要人工去计算一种叫做自恢复误差来进行推断时候的反向传播以获得能力。它在实际手机端部署上的最大两项无法接受的阻碍会是什么（结合框架环境和计算图建立考量）？

1.12 本章小结

- **划开坚冰壁垒：**打破“训练完后就只管输出永远不再接受改写”的静态推算理念！提出了需要且必需针对输入流发生不可预测移位时执行实时、小规模优化的全新宏大议程。
- **确立概念范式：**给出了主从区分：在主端（具有几近不变、承载泛化规则提炼特征的网络骨架的全局慢权重）与局侧前线（面对短序列新数据流立刻展开梯度回归学习改写的极速抛弃式隐态）。
- **提供了研究锚点：**这本教材不仅仅是写给研究人员的工具合集，更是为了改变你认知深度网络“智能构成成分结构”底层思想观的思维体操。

读完本章，你应该能够：

- 准确抛弃“训练和测试截然不可并协互通”的陈规旧俗。
- 采用“系统预先学会一种打草稿寻找短时修正目标的能力”的视角看待为何大模型不会完全在测试态卡死。
- 掌握全局结构慢权重 θ 与伴数据流局部发生作用变造的隐藏快状态 $\boldsymbol{S}_t$ 此一通贯全书的双轨分类学术语标定。

第二章　为什么模型能在测试时学习?

> "There is always a signal, even without the label. If a model can predict its own rotation, it has already aligned its feature space enough to guess the class."
>
> — Sun et al., 2020

2.1　本章想回答什么问题

上一章确立了"训练结束后仍可继续学习"的必要性。但这立刻引发了一个核心技术问题:

> 在测试阶段，我们没有真实标签。没有标签就无法计算监督损失，无法计算梯度。那么模型到底依靠什么信号来调整自己的参数?

Sun et al. (2020) 的回答是：用**自监督学习（Self-Supervised Learning）**从输入数据本身构造伪标签，由此产生梯度信号。这就是 Test-Time Training（TTT）2020 的核心思想。

本章将:

1. 给出 TTT 2020 的完整算法描述（§2.4）
2. 严格推导为什么自监督辅助头能间接改善主任务性能（§2.5）
3. 讨论这种方法的工程代价和局限性（§2.6），为后续章节引入"架构内化更新"的动机做铺垫

2.2 最小例子：旋转预测作为自监督信号

玩具问题 B：旋转预测实现域适应

训练阶段：一个图像分类器在清晰、正放的动物照片上训练。训练时同时引入辅助任务：把每张图分别旋转 $0°, 90°, 180°, 270°$，让网络预测旋转角度（4 分类问题）。

测试阶段：输入图像变成了模糊、倾斜的监控画面——这是一种**分布偏移（Distribution Shift）**。分类头的性能急剧下降。

TTT 修正：对每张测试图像，先执行 K 步旋转预测的辅助优化：

1. 对测试图施加 4 种旋转，构造伪标签 $\{(R_j(x_{\text{test}}), j)\}_{j=0}^{3}$
2. 以旋转分类损失为目标，对特征提取器 θ_e 做 K 步梯度更新
3. 用更新后的提取器运行主分类头

关键洞见：正确预测旋转角度需要网络理解图像的“上下左右”——这迫使特征提取器适应当前图像的分布特性，从而间接改善主分类性能。

2.3 直觉解释：双头共享表征架构

TTT 2020 的架构如下：

组件	角色
f_{θ_e}	特征提取器（共享骨干，如 ResNet）
f_{θ_m}	主任务头（分类/回归）
f_{θ_s}	辅助自监督头（如旋转预测）

训练时，三个组件联合训练，使得 θ_e 同时学习对主任务和辅助任务有用的表征。

测试时，**只更新** θ_e（通过辅助损失的梯度），主任务头 θ_m 和辅助头 θ_s 的参数保持冻结。

这个机制之所以有效，是因为一个关键假设：

> 使表征适应辅助任务（旋转预测）所需的特征调整，与使表征适应主任务（分类）所需的特征调整，共享足够大的子空间。

如果这个假设成立，那么“为了更好地预测旋转”而修正 θ_e，会“顺带”改善分类性能。

2.4 数学形式化：TTT 2020 算法

2.4.1 预训练阶段

在训练集 $\{(x_i, y_i)\}_{i=1}^N$ 上联合优化：

$$\min_{\theta_e,\theta_m,\theta_s} \frac{1}{N}\sum_{i=1}^{N}\Big[\mathcal{L}_{\text{main}}(f_{\theta_m}(f_{\theta_e}(x_i)),\, y_i) + \alpha\cdot\mathcal{L}_{\text{ss}}(f_{\theta_s}(f_{\theta_e}(\hat{x}_i)),\, \hat{y}_i)\Big] \tag{2.1}$$

其中：

- $\mathcal{L}_{\text{main}}$ 是主任务损失（如交叉熵）
- $\mathcal{L}_{\text{ss}}$ 是自监督辅助损失
- $\hat{x}_i = P(x_i)$ 是对输入施加的自监督变换（旋转、遮盖、打乱等）
- $\hat{y}_i$ 是变换的伪标签（在旋转预测中即旋转角度的类别索引）
- $\alpha > 0$ 是平衡主损失和辅助损失的超参数

2.4.2 测试时适应阶段

对每一个测试样本 x_{test}，执行以下步骤：

Step 1：保存参数快照

$$\theta_e^{(0)} \leftarrow \theta_e^* \qquad \text{(从预训练得到的参数开始)} \tag{2.2}$$

Step 2：构造自监督信号

施加变换族 $\{P_j\}_{j=1}^J$，得到 J 组伪标记数据：

$$\{(P_j(x_{\text{test}}),\, j)\}_{j=1}^J \tag{2.3}$$

Step 3：K 步梯度更新（仅更新 θ_e）

$$\theta_e^{(k)} = \theta_e^{(k-1)} - \eta\nabla_{\theta_e}\mathcal{L}_{\text{ss}}\Big(f_{\theta_s^*}(f_{\theta_e^{(k-1)}}(P_j(x_{\text{test}}))),\, j\Big), \quad k = 1,\ldots,K \tag{2.4}$$

Step 4：用适应后的参数进行推断

$$\hat{y} = f_{\theta_m^*}(f_{\theta_e^{(K)}}(x_{\text{test}})) \tag{2.5}$$

Step 5：参数重置

$$\theta_e \leftarrow \theta_e^* \qquad \text{(恢复为预训练参数)} \tag{2.6}$$

注 (参数重置的必要性). 步骤 5 至关重要。如果不重置 θ_e，对第 n 个测试样本的适应会 “污染” 对第 $n+1$ 个样本的推断。由于相邻测试样本可能来自不同的分布偏移模式，累积的参数漂移会迅速导致性能崩溃。

2.5 为什么辅助任务能帮助主任务?

直觉上，辅助任务与主任务之间存在 “表征共享” 关系。我们可以用以下定理给出一个形式化的条件。

定理 2.1 (辅助损失的间接改善条件)**.** *设 θ_e^* 是预训练后的特征提取器参数，$\Delta = -\eta\nabla_{\theta_e}\mathcal{L}_{ss}$ 是自监督损失的一步更新。*

如果主任务损失的梯度与辅助损失的梯度之间满足 **正相关条件**:

$$\langle \nabla_{\theta_e}\mathcal{L}_{main}(\theta_e^*),\ \nabla_{\theta_e}\mathcal{L}_{ss}(\theta_e^*)\rangle > 0 \tag{2.7}$$

则辅助更新 $\theta_e^* + \Delta$ 在一阶近似下也会减少主任务损失:

$$\mathcal{L}_{main}(\theta_e^* + \Delta) \approx \mathcal{L}_{main}(\theta_e^*) - \eta\langle \nabla\mathcal{L}_{main},\ \nabla\mathcal{L}_{ss}\rangle < \mathcal{L}_{main}(\theta_e^*) \tag{2.8}$$

证明. 一阶 Taylor 展开: $\mathcal{L}_{\text{main}}(\theta_e^*+\Delta) \approx \mathcal{L}_{\text{main}}(\theta_e^*) + \langle \nabla\mathcal{L}_{\text{main}}, \Delta\rangle$。代入 $\Delta = -\eta\nabla\mathcal{L}_{\text{ss}}$ 即得 $\approx \mathcal{L}_{\text{main}}(\theta_e^*) - \eta\langle \nabla\mathcal{L}_{\text{main}}, \nabla\mathcal{L}_{\text{ss}}\rangle$。由条件 (2.7)，括号内的内积 > 0，因此主损失下降。 □

注. 条件 (2.7) 并不总是成立。当辅助任务与主任务之间的特征共享不足时（例如用 “颜色直方图匹配” 辅助任务来帮助 “语义分割”），梯度可能正交甚至反向，导致测试时适应反而损害主任务性能。这是 TTT 方法的核心风险。

2.6 计算代价分析与工程局限

TTT 2020 的主要工程问题在于测试时的**计算开销**。

	标准推断	**TTT 推断**
每样本计算量	1 次前向传播	$K \times J$ 次前向 + K 次反向
GPU 内存	仅存激活值	需存梯度计算图
延迟	∼1ms	∼50–500ms (取决于 K)
参数管理	无	需保存/恢复 θ_e^*

对于**逐 token 的序列模型**（如语言模型），TTT 2020 的 “全局反向传播 + 参数重置” 范式完全不可行：每生成一个 token 就要对整个网络反向传播一次。

从外部微调到架构内化

TTT 2020 的核心贡献是证明了“测试时学习”的**可行性**——模型确实可以利用自监督信号适应。但它的工程瓶颈指向了后续研究的方向：**能否把“测试时的参数更新”内化到网络架构本身中**——不是在外部跑反向传播，而是设计一种层结构，其前向传播天然等价于执行参数更新？这正是第 三–八 章的主题。

2.7 神经科学直觉与类比边界

神经科学直觉：感觉-运动的在线校准

类比对象：人类的**棱镜适应（Prism Adaptation）**现象可以帮助理解 TTT 的核心机制。

当受试者佩戴会使视觉偏移 20° 的棱镜时，初始的手眼协调完全失准（类似分布偏移后模型性能下降）。但仅需数次伸手抓握尝试，受试者就能重新校准——**即使没有人告诉他“正确的手臂位置在哪里”**。校准依赖的是**自身的本体感觉反馈**（手臂的运动感知信号），而非外部标签。

共同之处（算法层面）：

棱镜适应	TTT 2020
视觉偏移 = 分布偏移	测试图像来自新分布
本体感觉反馈 = 自监督信号	旋转预测提供伪标签
小脑校准视觉-运动映射	梯度更新特征提取器
摘除棱镜后需重新适应	参数重置避免跨样本污染

这个类比在哪里失效

一、更新规则不同（实现机制层）

TTT 2020：通过显式的全局反向传播计算梯度，沿着整个计算图将误差传播到 θ_e 的每个参数。需要存储完整的计算图。

棱镜适应：小脑的误差校正依赖于**局部的**攀缘纤维信号和 Purkinje 细胞的突触可塑性，是一种逐层局部的适应，不存在全局误差反传。

二、适应范围不同（表征层）

TTT 更新的是神经网络中**所有共享层**的参数——这是一种全局性的表征调整。生物的棱镜适应主要局限于 **小脑-顶叶**回路的特定连接，不会改写整个

视觉皮层的表征。

读者不应从这个类比推出：

- **[X]** “TTT 2020 实现了类脑的在线适应”
- **[X]** “反向传播等价于小脑的误差校正”
- **[X]** “参数重置等价于摘除棱镜后的重新适应”

类比成立的层次：*算法层面*——两者都使用非任务本身的辅助信号来间接校准任务相关的表征。
类比失效的层次：实现机制（全局反向传播 vs. 局部突触可塑性）、更新范围（所有共享层 vs. 特定回路）均不同。

2.8 代表论文精读

Sun et al. (2020) —Test-Time Training with Self-Supervision for Generalization under Distribution Shifts 这是 TTT 范式的奠基论文。核心贡献：证明了旋转预测作为自监督辅助任务，能够有效改善分布偏移下的分类性能。在 CIFAR-10-C、ImageNet-C 等基准上，TTT 显著优于不做任何适应的基线。更重要的是，该论文建立了 “测试时利用自监督信号” 这一范式的可行性证明（existence proof）——为后续所有测试时学习研究奠定了基础。

本教材用处：证明无标签条件下的测试时适应是可行的。本章的整个框架（双头架构、辅助损失、参数重置）直接来源于这篇论文。

Krause et al. (2018) —Dynamic Evaluation of Neural Sequence Models 在语言模型领域，Dynamic Evaluation 是一种更早期的测试时适应策略：在部署时持续用语言模型的自回归损失（下一个 token 预测）对模型参数进行在线微调。这可以被理解为一种不需要额外辅助头的 TTT 方案，因为语言模型的主任务本身就是自监督的。

本教材用处：说明 TTT 思想在 NLP 领域的自然对应物，并为后续的 TTT-Layer（第 八 章）提供额外动机。

2.9 和前后章节的关系

- **来自第 一 章**：上一章确立了 “测试阶段学习” 的必要性。本章给出了最简单的实现方案——外部挂载自监督辅助头 + 反向传播。
- **通向第 三 章**：TTT 2020 的 “预训练阶段学习辅助头” 和 “测试阶段用辅助头

微调” 天然形成了一个双层优化结构：外循环（预训练）学习 “如何设定辅助任务的参数”，内循环（测试时）用辅助任务微调特征提取器。这正是 MAML 等元学习方法的核心框架。

- **通向第 四–八 章**：本章揭示的工程瓶颈——“每次推断都需要全局反向传播”——驱动了后续研究的核心目标：把测试时更新**内化到网络层中**，使前向传播本身就执行参数适应，无需外部反向传播。

2.10 实验

Lab 2：旋转自监督 TTT 的完整实现

目标：在 CIFAR-10 上实现 TTT 2020，并测量适应效果和计算开销。

1. **构建双头网络**：使用 ResNet-18 作为特征提取器 θ_e，接一个 10 类分类头 θ_m 和一个 4 类旋转预测头 θ_s。
2. **联合预训练**：在 CIFAR-10 训练集上用式 (2.1) 联合训练，$\alpha = 0.5$。记录收敛后的测试准确率（无分布偏移）。
3. **制造分布偏移**：对 CIFAR-10 测试集施加高斯模糊 + 随机旋转（非 90° 倍数），记录不做 TTT 时的准确率下降。
4. **实施 TTT**：对每张测试图执行式 (2.4)，$K = 1, 3, 10$。记录准确率恢复情况。
5. **测量计算开销**：比较 “标准推断” 和 “TTT 推断” 的每张图延迟和 GPU 内存占用。
6. **不重置实验（对照组）**：去掉式 (2.6) 的参数重置，观察在处理 100 张连续测试图后的性能衰退曲线。

预期结果：

- TTT ($K = 3$) 应将准确率恢复 5–15%
- 不重置参数会导致准确率在约 50 张图后开始下降
- TTT 每张图的延迟约为标准推断的 10–50 倍

2.11 常见误解

常见误解

误解 1：只要设计任意一个自监督辅助任务，TTT 就能改善主任务性能。
纠正：只有当辅助任务的梯度与主任务的梯度之间满足正相关条件 (2.7) 时，辅助更新才能帮助主任务。如果辅助任务所需的特征调整与主任务正交或冲突，TTT 不仅无效，还可能有害。选择合适的辅助任务是 TTT 方法的核心设计难点。

常见误解

误解 2：既然我们已经有了 TTT 2020，为什么还需要设计特殊的网络层（TTT-Layer 等）？直接给任何模型挂一个辅助头不就行了？
纠正：TTT 2020 对每个测试样本都需要执行**完整的反向传播**，计算开销远超正常推断。对于序列模型（每个 token 都需要适应），这种开销是灾难性的。后续章节的核心动机正是：把 “测试时更新” 从外部的反向传播 **内化**为前向传播层的固有行为，使推断效率与普通层相当。这是架构层面的革命，不是算法层面的修补。

2.12 练习题

练习 2.1. 参数污染分析：如果在式 (2.4) 的 K 步更新后不执行参数重置（式 (2.6)），而是直接用 $\theta_e^{(K)}$ 处理下一张测试图。假设第二张测试图来自与第一张完全不同的分布偏移，用一阶 Taylor 展开分析为什么 $\theta_e^{(K)}$ 对第二张图的主任务损失可能比 θ_e^* 更差。

练习 2.2. 梯度对齐条件：设主任务是 CIFAR-10 物体分类，考虑以下三种辅助任务：（a）旋转预测、（b）颜色通道排列预测、（c）图像拼图重组。分析每种辅助任务在 “含高斯模糊” 和 “含几何变形” 两种分布偏移下，梯度对齐条件 (2.7) 是否可能被满足，并说明理由。

练习 2.3. 从 CV 到 NLP：为基于 Transformer 的语言模型设计一种自监督辅助任务，使其能在测试时适应领域偏移（例如从学术论文切换到社交媒体文本）。你的设计需要说明：（a）辅助任务的具体定义、（b）伪标签如何构造、（c）最可能满足的梯度对齐条件。

练习 2.4. 步数 K 的权衡：分析 TTT 更新步数 K 的增加对以下指标的影响：（a）测试时计算开销、（b）辅助任务损失 $\mathcal{L}_{\text{ss}}$、（c）主任务性能。说明为什么存在一个最优的 K^*，超过该步数后主任务性能可能反而下降（过拟合于单样本）。

2.13 本章小结

- **TTT 2020 的核心思想**：通过自监督辅助任务（如旋转预测），在测试时为模型提供无需真实标签的梯度信号。辅助更新调整共享特征提取器，间接改善主任务性能。
- **有效性条件**：辅助损失与主损失的梯度之间必须满足正相关条件（定理 2.1），否则 TTT 可能无效甚至有害。
- **工程代价**：每个测试样本需要 K 次完整的反向传播，计算开销远超标准推断。参数重置是必要的，否则会导致跨样本污染。
- **通向后续章节的桥梁**：TTT 2020 的工程瓶颈（全局反向传播、高延迟、参数管理复杂）直接驱动了“把测试时更新内化到网络架构中”的研究方向——这正是从第 三 章开始的主线。
- **历史地位**：TTT 2020 是“测试时学习”的**可行性证明**——它证明了这条路走得通。后续的快权重、TTT-Layer、Titans 等是“如何高效走这条路”的进化。

读完本章，你应该能够：

- 写出 TTT 2020 的完整算法（预训练 + 测试时更新 + 推断 + 重置）
- 用一阶 Taylor 展开论证辅助梯度何时帮助/何时损害主任务
- 解释为什么参数重置是必要的
- 量化 TTT 的计算开销，理解其对序列模型不可行的原因
- 阐述为什么 TTT 2020 的工程瓶颈引出了“内化更新到网络层”的后续研究方向

第三章　双层优化与元学习：让参数初始化点” 内行”

> “The goal of meta-learning is not to learn well, but to learn how to learn well.”
>
> — 研究社区共识

3.1　本章想回答什么问题

第 二 章告诉我们：当测试时遇到分布偏移，我们可以在测试阶段对自监督目标做几步梯度下降来适应新数据。这个方案总体可行，但有一个潜在问题：

> 如果模型的参数初始化点 θ 是由普通的监督学习产生的，那么它对”被微调”这件事根本没有做任何准备。几步梯度下降可能走错方向，可能步长不合适，可能根本不往有用的地方走。

本章的问题：**能否在训练阶段就找到一个初始化点 θ，使得对任何新任务，从 θ 出发做少量梯度步就能快速适应？**

这是**元学习（Meta-Learning）**的核心问题，其数学形式是**双层优化（Bilevel Optimization）**。

元学习的重要性超出 TTT 本身：

- 它提供了这本书全程使用的” 内循环/外循环” 语言（第 七–八 章将反复用到）
- 它解释了 TTT-Layer 中外部大参数 θ 和序列内隐状态之间的关系
- 它是理解”training 和 inference 作为嵌套优化” 这个统一视角的基础

3.2　什么是元学习：用学习自行车类比

考虑两种学自行车的方法：

方法 A（朴素）：每次遇到新学生，从零开始手把手教所有平衡技巧。每个学生平均需要 20 节课。

方法 B（元学习）：先”学会教自行车的方法”——掌握哪些解释方式有效、哪些练习游戏有用。新学生只需 3 节课。

方法 B 的”学会教方法”对应元学习的外循环：它学到了”如何快速学习新任务”的通用知识。内循环则是每个学生实际学习的过程（快速适应）。

定义 3.1 (元学习，非正式定义)**.** 元学习（learning to learn）的目标是：通过在一个**任务分布** $p(\mathcal{T})$ 上训练，使模型获得在新任务上**快速适应**的能力。

关键点：不是在单一任务上训练好，而是学会”适应新任务的方式”。

3.3 数学设定：任务分布与情节式学习

情节式学习（Episodic Learning）

元学习的训练过程被组织为一系列”情节”（Episode），每个情节对应一个任务 $\mathcal{T}_i$：

定义 3.2 (情节式学习设定)**.**

- **任务分布**：$p(\mathcal{T})$，例如”输入 $d = 10$ 正弦波参数”产生的各种正弦拟合任务。
- **支撑集（Support Set）**：$\mathcal{S}_i = \{(x_j, y_j)\}_{j=1}^{K}$，K 个训练样本（”K-shot”），用于在该任务内做适应。
- **查询集（Query Set）**：$\mathcal{Q}_i = \{(x_j, y_j)\}_{j=1}^{M}$，M 个测试样本，用于评估适应效果。
- 任务训练损失：$\mathcal{L}_{\mathcal{S}_i}(\theta) = \frac{1}{K}\sum_{(x,y)\in\mathcal{S}_i} \ell(f_\theta(x), y)$
- 任务验证损失：$\mathcal{L}_{\mathcal{Q}_i}(\phi) = \frac{1}{M}\sum_{(x,y)\in\mathcal{Q}_i} \ell(f_\phi(x), y)$

玩具示例：正弦波族 Meta-Learning

任务分布：从 $p(\mathcal{T})$ 采样振幅 $A \in [0.1, 5]$、相位 $\phi \in [0, \pi]$，生成任务 $y = A\sin(x + \phi)$。

每个情节：

- 采样 $K = 5$ 个 support 点（x 均匀采样于 $[-5, 5]$，计算对应 y）
- 要求模型在这 5 个点上”快速学会”该任务的 A 和 ϕ
- 用 query 集上的 MSE 衡量适应效果

问题：如果模型是随机初始化的 2 层 MLP，5 个点 + 几步梯度下降效果很差。元学习要找一个初始化 θ^*，使得 5 步梯度下降后就能很好地拟合任意正

弦波。

3.4 双层优化的正式框架

什么是双层优化

定义 3.3 (双层优化，Bilevel Optimization)**.** 双层优化是一类" 目标函数依赖于另一个优化问题之解" 的优化问题：

$$\min_{\theta} \quad F(\theta, \phi^*(\theta)) \tag{3.1}$$

$$\text{s.t.} \quad \phi^*(\theta) = \arg\min_{\phi} G(\theta, \phi) \tag{3.2}$$

外层优化（outer-level）优化 θ，目标 F 依赖于内层最优解 $\phi^*(\theta)$；内层优化（inner-level）给定 θ 后求 ϕ^*。

元学习（MAML）是双层优化的一个特例：

- **外层变量** θ：模型权重（初始化点）
- **内层操作**：给定 θ，在 $\mathcal{S}_i$ 上做 k 步梯度下降得到 ϕ_i
- **外层目标**：在 $\mathcal{Q}_i$ 上的平均损失（用内循环适应后的 ϕ_i 评估）

3.5 MAML 算法：优化初始化点

内循环：快速适应

对于每个任务 $\mathcal{T}_i$，从当前 θ 出发做 k 步梯度下降：

$$\phi_i^{(0)} = \theta \tag{3.3}$$

$$\phi_i^{(t+1)} = \phi_i^{(t)} - \alpha \nabla_{\phi_i^{(t)}} \mathcal{L}_{\mathcal{S}_i}(\phi_i^{(t)}), \quad t = 0, \ldots, k-1 \tag{3.4}$$

记 $\phi_i \triangleq \phi_i^{(k)}$（$k$ 步后的参数）。
在 $k = 1$（单步近似）时：

$$\phi_i = \theta - \alpha \nabla_\theta \mathcal{L}_{\mathcal{S}_i}(\theta) \tag{3.5}$$

外循环：元更新

外循环在 query 集上评估内循环后的表现，并更新 θ：

$$\min_{\theta} \quad \mathcal{L}_{\text{meta}}(\theta) \triangleq \mathbb{E}_{\mathcal{T}_i \sim p(\mathcal{T})} \left[\mathcal{L}_{\mathcal{Q}_i}(\phi_i(\theta))\right] \tag{3.6}$$

元梯度更新：

$$\theta \leftarrow \theta - \beta \nabla_\theta \mathcal{L}_{\text{meta}}(\theta) \tag{3.7}$$

完整 MAML 算法

Algorithm 1 MAML（简化版，$k = 1$）

Require: 任务分布 $p(\mathcal{T})$，内循环学习率 α，外循环学习率 β

```
随机初始化 θ
for 每个元训练步骤 do
    采样一批任务 {T_1, ..., T_B} ~ p(T)
    for 每个任务 T_i do
        计算内循环参数：φ_i = θ − α∇_θ L_{S_i}(θ)
    end for
    外循环更新：θ ← θ − β∇_θ Σ_i L_{Q_i}(φ_i(θ))
end forreturn θ（最优初始化点）
```

3.6 二阶导数：Hessian 项的来源与一阶近似

计算元梯度 $\nabla_\theta \mathcal{L}_{\mathcal{Q}_i}(\phi_i(\theta))$ 涉及链式法则：

$$\frac{d}{d\theta} \mathcal{L}_{\mathcal{Q}_i}(\phi_i(\theta)) = \nabla_{\phi_i} \mathcal{L}_{\mathcal{Q}_i}(\phi_i) \cdot \frac{d\phi_i}{d\theta} \tag{3.8}$$

对于 $k = 1$ 的内循环（式 (3.5)）：

$$\frac{d\phi_i}{d\theta} = \frac{d}{d\theta} \left[\theta - \alpha \nabla_\theta \mathcal{L}_{\mathcal{S}_i}(\theta)\right] \tag{3.9}$$

$$= I - \alpha \underbrace{\nabla_\theta^2 \mathcal{L}_{\mathcal{S}_i}(\theta)}_{\text{Hessian}} \tag{3.10}$$

因此元梯度为：

$$\nabla_\theta \mathcal{L}_{\mathcal{Q}_i}(\phi_i(\theta)) = \nabla_{\phi_i} \mathcal{L}_{\mathcal{Q}_i} \cdot (I - \alpha H_{\mathcal{S}_i}) \tag{3.11}$$

其中 $H_{\mathcal{S}_i} = \nabla_\theta^2 \mathcal{L}_{\mathcal{S}_i}(\theta)$ 是 Hessian 矩阵（二阶导数）。

为什么 Hessian 计算昂贵 对于 d 维参数向量，Hessian 是 $d \times d$ 矩阵。一个有 10^7 个参数的网络，Hessian 有 10^{14} 个元素——远超显存容量。

FOMAML：一阶近似（去掉 Hessian 项）

$$\nabla_\theta^{\text{FOMAML}} \mathcal{L}_{\mathcal{Q}_i}(\phi_i(\theta)) \approx \nabla_{\phi_i} \mathcal{L}_{\mathcal{Q}_i}(\phi_i) \tag{3.12}$$

即：直接用 ϕ_i（内循环更新后的参数）在查询集上的梯度来估计元梯度，忽略了 ϕ_i 对 θ 的依赖。

注. 为什么 FOMAML 在实践中有效？因为内循环步长 α 很小时，H 项系数为 αH，当 $\alpha \ll 1$ 时一阶近似误差 $O(\alpha)$ 很小。此外，一阶近似本身提供了一种合理的”元梯度”估计：直接优化内循环更新后参数在查询集上的性能，有直观的合理性。

Reptile：更简单的一阶近似 Nichol et al. (2018) 提出了更简洁的 Reptile 算法：在每个任务上做 k 步梯度下降得到 ϕ_i，直接用 $(\phi_i - \theta)$ 作为元梯度方向（相当于”把 θ 朝每个任务的解方向拉”）：

$$\theta \leftarrow \theta + \beta \cdot \frac{1}{B} \sum_i (\phi_i - \theta) \tag{3.13}$$

Reptile 不需要任何二阶导数，实现极简，效果接近 FOMAML，是工程实践中的常用选择。

3.7 Worked Example：双层优化的几何直觉

例子 3.4 (双层优化的等高线图解)**.** 设两个任务：

- 任务 1：最小化 $\mathcal{L}_1(\theta) = (\theta - 3)^2$（最优点在 $\theta = 3$）
- 任务 2：最小化 $\mathcal{L}_2(\theta) = (\theta + 3)^2$（最优点在 $\theta = -3$）

朴素 MTL（多任务学习）：直接最小化 $\mathcal{L}_1(\theta) + \mathcal{L}_2(\theta) = (\theta - 3)^2 + (\theta + 3)^2$，最优点在 $\theta = 0$。这个点离两个任务最优点各 3 个单位。

元学习(MAML):最小化 $\mathcal{L}_{\text{meta}}(\theta) = (\underbrace{\theta - \alpha \cdot 2(\theta - 3)}_{\phi_1(\theta)} - 3)^2 + (\underbrace{\theta - \alpha \cdot 2(\theta + 3)}_{\phi_2(\theta)} + 3)^2$

设 $\alpha = 0.5$：$\phi_1(\theta) = \theta - (\theta - 3) = 3$（一步就到最优!）$\phi_2(\theta) = \theta - (\theta + 3) = -3$

$\mathcal{L}_{\text{meta}}(\theta) = (3 - 3)^2 + (-3 + 3)^2 = 0$，对所有 θ 成立。

解读：在这个线性二次情形下，MAML 初始化点的选择不重要（因为一步就到最优）；更实际的情形是一步不够，此时 θ 的选择至关重要。

为了展示 θ 的重要性，把任务改为：$\mathcal{L}_1(\theta) = (\theta - 3)^2, \mathcal{L}_2(\theta) = (\theta + 3)^2$，内循环只做一步 $\alpha = 0.1$。

$\phi_1(\theta) = \theta - 0.2(\theta - 3) = 0.8\theta + 0.6$ $\phi_2(\theta) = \theta - 0.2(\theta + 3) = 0.8\theta - 0.6$

$\mathcal{L}_{\text{meta}}(\theta) = (0.8\theta + 0.6 - 3)^2 + (0.8\theta - 0.6 + 3)^2 = (0.8\theta - 2.4)^2 + (0.8\theta + 2.4)^2 = 2(0.8\theta)^2 + 2(2.4)^2$（展开后 $2 \cdot (0.8\theta)^2 + 2 \cdot 5.76$）

最优 $\theta^* = 0$（元学习确认：两个任务平衡）——从 0 出发一步梯度更新后：$\phi_1(0) = 0.6$（距最优 3 差 2.4，起点 $\theta = 0$ 比 $\theta = 1$ 更均衡地靠近两个最优）。

核心直觉：元学习的初始化点是”在少步梯度下降后，同时对多个任务最优”的那个点。

3.8 内外循环语言：全书的统一语法

元学习提供了全书最重要的一对词汇：

定义 3.5 (内循环与外循环).

- **内循环（Inner Loop）**：在特定数据/任务上执行的、**快速适应**的优化过程。操作的对象：任务适应参数 ϕ_i（MAML 中），或隐状态 s_t（TTT-Layer 中）。时间尺度：短（几步梯度，或每个 token 一步）。
- **外循环（Outer Loop）**：跨任务/跨序列的、**元知识学习**过程。操作的对象：初始化点 θ 或网络的持久参数。时间尺度：长（跨越大规模训练数据）。

后续章节如何使用这对语言：

系统	内循环	外循环
MAML（第 三 章）	在 $\mathcal{S}_i$ 上 k 步 GD	在 $\mathcal{Q}_i$ 上更新 θ
TTT 2020（第 二 章）	在测试样本上做辅助 GD	大规模预训练 θ
TTT-RNN（第 七 章）	隐状态 $\boldsymbol{S}_t$ 的梯度步	外部参数 θ 的预训练
TTT-Layer（第 八 章）	每 token 触发的内部 GD	书写规则的预训练学习
Titans（第 九 章）	Neural Memory 的逐步更新	主网络和门控的训练

理解了”内外循环”这对语言，后面所有章节的架构设计都会变得清晰。

3.9 Learned Optimizers：学习优化器本身

Andrychowicz et al. (2016) 把元学习推到了更极端的方向：不只学习一个好的初始化点，而是学习**整个内循环优化器**本身。

思路 传统梯度下降：$\phi_{t+1} = \phi_t - \alpha \nabla \mathcal{L}$（固定规则）

学习的优化器：$\phi_{t+1} = \phi_t + m_\psi(\nabla \mathcal{L}, \phi_t, t)$ 其中 m_ψ 是一个 LSTM，其参数 ψ 通过元学习在任务分布上训练。

元学习目标（双层）

$$\min_{\psi} \quad \mathbb{E}_{\mathcal{T}} \left[\sum_{t=1}^{T} \mathcal{L}(\phi_t(\psi)) \right] \tag{3.14}$$

$$\text{其中} \quad \phi_{t+1}(\psi) = \phi_t(\psi) + m_\psi(\nabla_{\phi_t} \mathcal{L}) \tag{3.15}$$

与本书主线的关系 学习的优化器（Learned Optimizer）是理解以下联系的桥梁：

- 外循环学到的”优化规则”m_ψ 是慢权重
- 内循环每步 $\nabla \mathcal{L}$ 驱动的更新 $m_\psi(\nabla \mathcal{L})$ 是快权重式的动态适应
- 这和 TTT-Layer 的”外部参数 θ 学习更新规则，内部状态 $\boldsymbol{S}$ 根据规则更新”完全对应

视角（非定论）

”学习的优化器”和”TTT-Layer”之间的等价是一种深刻的教学视角，而不是在论文中明确建立的等价关系。本书这里做的是教材层面的概念统一，不应被引用为”某论文的原话证明”。

3.10 和前后章节的关系

- **来自第 二 章**：TTT 2020 让我们接受了”测试时更新参数”的合法性。元学习（本章）解决了”如何在训练时准备好以便被快速微调”的问题，让测试时适应更高效。
- **通向第 四–五 章**：快权重可以被视为元学习原理在序列模型架构中的一次实例化——慢权重学习”如何建立联想”（元知识），快权重在推断时做”快速适应”（内循环）。
- **通向第 七–八 章**：TTT-RNN 和 TTT-Layer 的设计就是把 MAML 的内外循环结构**嵌入到序列模型的前向传播中**，无需调用外部反向传播库。
- **通向第 十一 章**：Nested Learning 的”嵌套多级优化”框架直接对应于元学习的双层结构，只是被推广到了更多层级。

Lab 3：用 PyTorch 实现 FOMAML，在正弦波族上验证元学习

目标：验证元学习初始化点确实比随机初始化需要更少梯度步就能适应新任务。

1. **任务生成函数**：

```
import numpy as np
def sample_sinusoid_task():
    A = np.random.uniform(0.1, 5.0)
    phi = np.random.uniform(0, np.pi)
    def task_fn(x): return A * np.sin(x + phi)
    return task_fn
```

2. **模型**：2 层 MLP，输入 $\mathbb{R}^1 \to$ 隐 $\mathbb{R}^{64}$ ReLU $\to$ 输出 $\mathbb{R}^1$。

3. **内循环**：对每个任务的 $K = 5$ 个 support 点做 1 步手动梯度更新（不用 optimizer）：

```
# 手动一步梯度（保留计算图以便外循环求梯度）
loss_s = mse_loss(model(x_support), y_support)
grads = torch.autograd.grad(loss_s, model.parameters(),
    create_graph=True)
phi = [p - alpha * g for p, g in zip(model.parameters()
   , grads)]
```

4. **外循环**：对 $B = 16$ 个任务，用内循环参数 ϕ_i 计算 query 损失，平均后更新模型：

```
meta_loss = mean([evaluate(phi_i, x_query_i, y_query_i)
    for each task])
meta_optimizer.zero_grad()
meta_loss.backward()
meta_optimizer.step()
```

5. **对比实验**（两条曲线）：

   - 元学习后的 θ^*：从 θ^* 出发，在新任务的 5 个 support 点上做 $1, 2, 5, 10$ 步梯度，记录 query 集 MSE。
   - 随机初始化的 θ_{rand}：做同样步数，记录 MSE。

6. **画图**：x 轴 = 梯度步数（0–10），y 轴 =query MSE。验证元学习曲线下降更快、在 5 步内达到的最终 MSE 更低。

常见误解

常见误解 1：元学习 = few-shot learning，只有 $K = 1$ 或 $K = 5$ 的场景才需要。

纠正：元学习的本质是”优化适应过程”，不局限于 few-shot 场景。联邦学习中的个性化（用本地少量数据快速适应）、在线学习中的热启动、机器人学习中的快速策略适应，都是元学习思路的应用，K 可以是几百甚至更大。

常见误解

常见误解 2：MAML 一定需要二阶导数，计算很贵，没有实用价值。

纠正：FOMAML（一阶近似）和 Reptile 完全避开了二阶导数，计算开销只比普通梯度下降多一个常数因子（多跑一次内循环），在实践中广泛使用。真正的成本更多来自”需要用大量任务来形成有效元训练集”这个数据要求，而不是计算。

3.11 神经科学直觉与类比边界

神经科学直觉：免疫系统的适应性学习

类比对象：人体**免疫系统**的两层防御结构可以帮助理解双层优化的核心逻辑。

- **先天免疫（Innate Immunity）**：进化在基因组中编码的固定防御机制（类似预训练的初始化 θ）。
- **适应性免疫（Adaptive Immunity）**：对新病原体的快速特异性适应——B 细胞通过亲和力成熟在数天内产生高亲和力抗体（类似内循环的 k 步梯度适应）。

共同之处（算法层面）：

免疫系统	MAML 双层优化
进化 = 外循环	元训练 = 外循环
亲和力成熟 = 内循环	任务适应 = 内循环
基因组 = 初始化	θ = 共享初始化
抗体特异性 = 任务特化参数	ϕ_i = 任务特化参数

这个类比帮助理解什么：为什么需要两个时间尺度——慢的过程（进化/元训练）学习“如何快速适应”的策略，快的过程（免疫/内循环）利用这个策略对新挑战做特异性响应。

这个类比在哪里失效

一、“外循环”的机制不同（实现层）

MAML	外循环通过**反向传播穿过内循环**来计算梯度，这是一种精确的、可微的优化过程。
免疫系统	“外循环”是**自然选择**——一种无梯度的、基于变异和筛选的优化。没有显式的目标函数。

二、“内循环”的搜索方式不同

MAML 的内循环是**梯度下降**——沿损失函数的最陡方向移动。免疫系统的亲和力成熟是**随机突变 + 选择**——探索是随机的，没有梯度引导。

读者不应从这个类比推出：

- **[X]** “MAML 模拟了免疫系统的适应过程”
- **[X]** “元学习的外循环等价于进化”
- **[X]** “内循环的梯度步等价于亲和力成熟”

类比成立的层次：两层时间尺度分离的学习结构。

类比失效的层次：外循环机制（梯度 vs. 自然选择）、内循环搜索方式（梯度 vs. 随机突变）均不同。

3.12 练习题

练习 3.1. 双层优化梯度：设内循环为 $k = 1$ 步梯度，$\phi(\theta) = \theta - \alpha\nabla_\theta\mathcal{L}_\mathcal{S}(\theta)$。令 $F(\theta) = \mathcal{L}_\mathcal{Q}(\phi(\theta))$，展开 $\nabla_\theta F$，写出包含 Hessian 的完整表达式（式 (3.11)）。

然后写出 FOMAML 近似（忽略 Hessian），说明这个近似等价于" 用内循环参数 ϕ 在 query 集上的普通梯度"。

练习 3.2. Reptile 的几何理解：证明当任务数 $B = 2$，k 足够大时，Reptile 等价于在两个任务的最优解 θ_1^*, θ_2^* 之间做加权平均。（提示：$k \to \infty$ 时 $\phi_i \to \theta_i^*$，Reptile 更新 $\theta \leftarrow \theta + \beta/2 \cdot (\theta_1^* + \theta_2^* - 2\theta)$，找到不动点。）

练习 3.3. 内外循环的对应关系：对 MAML 和 TTT 2020（第 二 章）分别填写下表：

角色	MAML	TTT 2020
外循环参数（慢权重）	θ	?
内循环参数（快适应）	ϕ_i	?
外循环目标	query 集损失	?
内循环目标	support 集损失	?
内循环数据	support set $\mathcal{S}_i$	?
内循环施加场景	元测试时	?

练习 3.4. 连接 Nested Learning：MAML 是二层嵌套优化（内层 ϕ，外层 θ）。如果我们在 MAML 上再加一层——用另一个元元学习器来优化 MAML 的外循环超参数（如 α）——这变成了三层嵌套优化。这与第 十一 章的 Nested Learning 框架有什么关系？三层嵌套在实践中面临什么额外困难？

3.13 本章小结

- **元学习目标**：在任务分布 $p(\mathcal{T})$ 上训练，让模型获得" 用少量样本/步骤快速适应新任务" 的能力。
- **MAML 双层结构**：内循环 = 在 $\mathcal{S}_i$ 上做 k 步梯度下降得到 ϕ_i；外循环 = 用 ϕ_i 在 $\mathcal{Q}_i$ 上的损失来更新初始化点 θ。
- **二阶导数问题**：完整的 MAML 需要计算 Hessian；FOMAML 和 Reptile 用一阶近似绕开，实践中常用。

- **内外循环语言**是全书的统一语法——第 四–十一 章所有架构设计都可以用这对词汇来组织。
- **Learned Optimizer**（Andrychowicz et al.）把内循环本身（优化器）也变成元学习的对象，与 TTT-Layer 有深刻概念联系。

读完本章，你应该能够：

- 用双层优化的语言写出 MAML 的目标函数
- 推导一步内循环后的元梯度（含 Hessian 项）
- 解释 FOMAML 的一阶近似合理性
- 将 MAML 的内/外循环对应到 TTT、快权重、TTT-Layer 的类似结构
- 实现一个最简 FOMAML 并在正弦波上验证

第四章　快权重、外部记忆与时间尺度

> "Memory is not a single thing. It is a family of mechanisms operating at different speeds, capacities, and timescales."

4.1　本章想回答什么问题

到目前为止我们已经建立了一个基本认知：模型可以在测试时通过梯度下降来适应新数据（第二章），而元学习给了我们一个训练框架来让这种适应变得高效（第三章）。

但这里有一个令人不舒服的问题还没有解决：

> 每次处理一个测试样本，都要调用一次 `.backward()`（反向传播），这在延迟敏感的场景（实时系统、在线服务）里开销很大。有没有更轻量的方式让模型在处理序列时"积累上下文知识"？

本章的答案是：**快权重（Fast Weights）**。

快权重是一种介于"激活值（向量）"和"持久参数"之间的第三类存储结构——一个**动态矩阵**，能在纯前向计算中把序列历史压缩成可查询的联想记忆，无需任何反向传播。

理解快权重有三个原因很重要：

1. 它是 Transformer 注意力机制的理论前身（接下来的第五章会建立严格等价）；
2. 它提供了第一个"无反向传播的测试时记忆更新"机制，是 TTT-Layer 的概念基础；
3. 它清晰地划分了"快学"与"慢学"两个时间尺度——这是理解 Titans（第九章）的关键。

4.2 最小例子：从查找表到连续向量的模糊检索

玩具问题 B：键值联想记忆（Associative Recall）

场景：你是一个序列模型，正在逐步处理以下 token 序列：

$$\underbrace{k_1 \to v_1}_{\text{写入对 1}},\quad \underbrace{k_2 \to v_2}_{\text{写入对 2}},\quad \underbrace{k_3 \to v_3}_{\text{写入对 3}},\quad \underbrace{q \approx k_2}_{\text{查询}}$$

任务：当看到查询 q（与 k_2 相似但不完全一样）时，快速输出 v_2。

朴素方案一：维护一个显式字典 $\{k_i \to v_i\}$，查询时做精确匹配。

- 优点：精确
- 缺点：如果 $q \neq k_2$（模糊查询），失败；且字典大小随历史线性增长。

朴素方案二：标准 RNN（$h_t = f(h_{t-1}, x_t)$），隐状态是向量。

- 优点：固定维度，在线更新
- 缺点：一个向量要同时记住多对关联，容量有限；”覆盖”问题严重。

快权重方案：维护一个**矩阵** $\boldsymbol{W}_t \in \mathbb{R}^{d\times d}$，用外积写入：$\boldsymbol{W}_t = \boldsymbol{W}_{t-1} + v_t k_t^\top$，用矩阵乘法查询：$\hat{v} = \boldsymbol{W}_t q$。
内积 $k_t^\top q$ 会给与 q 相似的键更大的权重，天然实现模糊检索。

为什么矩阵比向量好

一个向量 $h \in \mathbb{R}^d$ 有 d 个自由度，只能在一维的线上”定位”。一个矩阵 $\boldsymbol{W} \in \mathbb{R}^{d\times d}$ 有 d^2 个自由度，对于每对 (k, v)，外积 $vk^\top$ 贡献一个 rank-1 矩阵更新。如果键向量之间夹角足够大（近似正交），不同的键值对会被”存储”在矩阵的不同”方向”上，互相干扰的程度很小。

这就是快权重比标准隐藏向量表达力更强的根本原因。

4.3 外积更新规则的直觉与推导

Hebb 规则的矩阵版本

Hebb 规则（神经科学经典）：“cells that fire together, wire together”—如果 k 和 v 同时被激活，则它们之间的连接增强。

在矩阵语言中，这对应于对权重矩阵做外积加法：

$$\boldsymbol{W} \leftarrow \boldsymbol{W} + \eta \cdot v \cdot k^\top \tag{4.1}$$

其中 η 是写入步长（fast weight learning rate）。本节引用 Hebb 规则是因为它在**计算角色和算法组织**上与快权重外积更新有形式对应；两者在实现机制和学习信号来源上有根本差异（见本节末尾的类比边界盒子）。

验证检索：

$$\boldsymbol{W}k = \left(\sum_s \eta\, v_s k_s^\top\right) k \tag{4.2}$$

$$= \eta \sum_s v_s (k_s^\top k) \tag{4.3}$$

$$= \eta \sum_s v_s \cdot \langle k_s, k\rangle \tag{4.4}$$

如果 $k = k_2$，且键两两正交（$\langle k_i, k_j\rangle = 0$ 对 $i \neq j$，$\langle k_i, k_i\rangle = 1$），则 $\boldsymbol{W}k_2 = \eta v_2$。这就是精确检索。

遗忘因子：控制时间尺度

纯加法（(4.1)）会无限积累历史，可能被遥远的旧记忆干扰。加入指数遗忘因子 $\lambda \in (0, 1]$：

$$\boxed{\boldsymbol{W}_t = \lambda \boldsymbol{W}_{t-1} + \eta\, v_t k_t^\top} \tag{4.5}$$

展开后：

$$\boldsymbol{W}_t = \eta \sum_{s=1}^{t} \lambda^{t-s} v_s k_s^\top \tag{4.6}$$

查询：

$$\hat{v} = \boldsymbol{W}_t q = \eta \sum_{s=1}^{t} \lambda^{t-s} v_s \underbrace{(k_s^\top q)}_{\text{相似度}} \tag{4.7}$$

时间加权相似度检索

式 (4.7) 的直觉：查询 q 的输出是所有历史值 v_s 的加权和，权重 $= \lambda^{t-s} \cdot (k_s^\top q)$。

- λ^{t-s}：时间遗忘权重——越近的记忆权重越大。
- $k_s^\top q$：相似度权重——键越相似查询的记忆权重越大。

两个权重相乘：近期的、相关的记忆最被优先检索。这是一种极其自然的联想记忆机制。

神经科学直觉：短时突触可塑性与快权重的时间尺度

类比对象：我们把**短时突触可塑性（Short-Term Synaptic Plasticity, STP）**和**工作记忆（Working Memory）**作为帮助理解快权重机制的神经科学类比。

共同之处（计算角色 + 算法组织层面）：

- 快权重矩阵 $\boldsymbol{W}_t$ 和工作记忆都承担**短暂的、任务内关联存储器**的角色：在当前任务内快速构建，任务结束后丢弃，不用于跨任务持久存储。
- Hebb 规则的外积更新和突触 STP 都基于**共现活动来增强连接**：键和值同时出现时（或突触前后神经元同时激发），关联强度上升。
- 两者都区分**快变量**（$\boldsymbol{W}_t$ 每步更新，STP 在毫秒至秒内变化）和**慢变量**（慢权重 W_K, W_V, W_Q 训练后固定；突触基线权重在更长时间尺度变化）。
- 遗忘因子 $\lambda < 1$ 使旧记忆指数衰减；STP 的时间常数 τ 决定突触增强的衰退速率——两者都在时间轴上编码了”近期优先”的偏好。

这个类比帮助理解什么：为什么快权重是矩阵而不是向量？工作记忆能同时维护多个独立对象，对应快权重矩阵 d^2 个自由度可以存储多个独立关联。为什么需要遗忘因子？系统应优先响应”当前任务中刚刚发生的事”，与工作记忆有限容量和快速刷新特性一致。

这个类比在哪里失效

一、更新规则的学习信号来源根本不同

快权重的外积更新由慢权重 W_K, W_V 生成的显式键值对驱动，是精确可微的矩阵代数，全局一致地执行。生物 Hebbian/STP 可塑性由局部突触电位、钙离子内流和时序相关性（STDP）驱动，没有全局目标函数，也没有反向传播。

二、原始 Hebb 规则不稳定，两者解法不同

纯外积累加（$\lambda = 1$）会无限增大权重范数，导致数值发散。工程实现通过遗忘因子 λ 和 ℓ_2 正则化保证稳定。生物系统通过突触缩放（Synaptic Scaling，一种稳态可塑性机制）防止过度增强。两者解决同一稳定性问题，但机制完全不同。

三、矩阵表示在生物中无直接类比

快权重矩阵 $\boldsymbol{W} \in \mathbb{R}^{d\times d}$ 是一个集中存储的、对所有输入共享的参数对象。生物的突触连接权重分布在解剖空间中，受神经元拓扑结构约束，无法被简化为一个单一共享矩阵。

读者不应推出：

- □ 快权重等价于短时突触可塑性

- □ 工作记忆就是一个矩阵
- □ 遗忘因子 λ 等同于突触时间常数 τ

类比在**计算角色**和**算法组织**层面成立；在**学习信号来源**、**稳定性机制**和**表征结构**层面失效。

4.4 慢权重 vs. 快权重：两个时间尺度的正式定义

定义 4.1 (慢权重与快权重)**.** 在一个具有快权重机制的系统中，存在两类参数：

- **慢权重（Slow Weights）** θ：通过在大规模训练集上的反向传播梯度下降学习，代表跨任务、跨序列的通用知识。在推断（inference）时固定不变。学习" 如何建立联想" 的元知识。
- **快权重（Fast Weights）** $\boldsymbol{W}_t$：在推断时由当前序列的输入动态生成，代表当前上下文的短期联想。每个时间步更新一次，序列结束后通常丢弃（不持久化）。存储" 这个序列里具体发生了什么关联"。

这个慢/快分离在后续所有架构中都会以不同形式出现：

章节	慢权重角色	快权重角色
第 四 章（本章）	投影矩阵 W_K, W_V, W_Q	记忆矩阵 $\boldsymbol{W}_t$
第 五 章	注意力参数	KV cache / 线性注意力状态 $\boldsymbol{S}_t$
第 八 章	外部参数 θ	隐状态权重矩阵 $\boldsymbol{S}_t$
第 九 章	主网络参数	Neural Memory 的 MLP 权重 Φ_t

4.5 快权重写入：一个完整的 Worked Example

例子 4.2 (3 键值对的精确检索)**.** 设 $d = 3$，$\lambda = 1$，$\eta = 1$。定义三个正交单位键向量：

$$k_1 = \begin{pmatrix}1\\0\\0\end{pmatrix}, \quad k_2 = \begin{pmatrix}0\\1\\0\end{pmatrix}, \quad k_3 = \begin{pmatrix}0\\0\\1\end{pmatrix}$$

和对应的值：

$$v_1 = \begin{pmatrix}1\\2\end{pmatrix}, \quad v_2 = \begin{pmatrix}3\\4\end{pmatrix}, \quad v_3 = \begin{pmatrix}5\\6\end{pmatrix}$$

（注：这里 $v \in \mathbb{R}^2$，实际应用中 $v \in \mathbb{R}^d$，但用不同维度简化展示。）

快权重矩阵 $\boldsymbol{W} \in \mathbb{R}^{2\times 3}$（$d_v = 2, d_k = 3$）按顺序写入：

写入 (k_1, v_1):

$$\boldsymbol{W}_1 = \mathbf{0} + v_1 k_1^\top = \begin{pmatrix} 1 \\ 2 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 0 & 0 \end{pmatrix}$$

写入 (k_2, v_2):

$$\boldsymbol{W}_2 = \boldsymbol{W}_1 + v_2 k_2^\top = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 0 & 0 \end{pmatrix} + \begin{pmatrix} 0 & 3 & 0 \\ 0 & 4 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 3 & 0 \\ 2 & 4 & 0 \end{pmatrix}$$

写入 (k_3, v_3):

$$\boldsymbol{W}_3 = \begin{pmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{pmatrix}$$

查询 $q = k_2$:

$$\hat{v} = \boldsymbol{W}_3 k_2 = \begin{pmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 3 \\ 4 \end{pmatrix} = v_2 \checkmark$$

精确检索成功!

模糊查询 $q = k_2 + 0.1 \cdot k_1$ **(轻微干扰)**:

$$\hat{v} = \boldsymbol{W}_3 q = \begin{pmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{pmatrix} \begin{pmatrix} 0.1 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0.1 + 3 \\ 0.2 + 4 \end{pmatrix} = \begin{pmatrix} 3.1 \\ 4.2 \end{pmatrix}$$

结果仍然接近 $v_2 = (3, 4)^\top$，只有 0.1 的干扰——说明快权重对模糊键具有鲁棒性。干扰量来自 k_1 方向的内积 $0.1 \cdot k_1^\top k_2 = 0$……不对，$k_1^\top q = 0.1$，所以 v_1 分量的贡献是 $0.1 \cdot v_1 = (0.1, 0.2)^\top$。观察：当键近似正交时，干扰与查询”对齐程度”成正比，可以被量化控制。

4.6 干扰分析：当键不正交时会发生什么

上面的例子用了正交键，真实情况往往不是这样。下面分析干扰的程度。

命题 4.3 (快权重的干扰界)**.** 设写入了 N 个键值对 $\{(k_i, v_i)\}_{i=1}^N$，$\lambda = 1$，$\eta = 1$。查询 $q = k_j$（其中 $\|k_j\| = 1$）时，输出为:

$$\hat{v} = \boldsymbol{W}_N k_j = v_j + \underbrace{\sum_{i \neq j} v_i (k_i^\top k_j)}_{\text{干扰项}}$$

干扰项的模满足 $\|\text{干扰}\| \le \left(\sum_{i\neq j} \|v_i\| \cdot |k_i^\top k_j|\right)$。

若值向量有界（$\|v_i\| \le V$），且键两两内积有界（$|k_i^\top k_j| \le \rho$ 对 $i \neq j$），则：

$$\|\text{干扰}\| \le (N-1)V\rho$$

证明（简化版）. 直接展开 $\boldsymbol{W}_N k_j = \sum_i v_i(k_i^\top k_j)$，分离 $i = j$ 项即可。由三角不等式得到上界。 □

注. 这个界说明：如果键向量来自高维随机分布（d 很大），则任意两个单位向量的内积期望约为 $O(1/\sqrt{d})$，干扰量平均为 $O(N/\sqrt{d})$。当 $d \gg N$ 时，干扰可以忽略不计——这正是高维联想记忆工作良好的原因。实际 Transformer 中通常 $d \in \{64, 128, 256\}$，这给了联想记忆相当大的" 有效容量"。

4.7 慢权重学习什么：读写参数 (W_K, W_V, W_Q)

在实际应用中，我们不直接用原始 token 向量作为键值查询，而是先通过可学习的投影矩阵变换：

$$k_t = W_K x_t \tag{4.8}$$

$$v_t = W_V x_t \tag{4.9}$$

$$q = W_Q x_{\text{query}} \tag{4.10}$$

这里 $W_K, W_V, W_Q \in \mathbb{R}^{d\times d}$ 是慢权重（通过训练学习），$\boldsymbol{W}_t$ 是快权重（在推断时动态生成）。

慢权重学习什么?

慢权重学习的是" 什么样的投影使得快权重联想记忆最有效"：

- W_K：将原始 token 映射为" 作为键应该长什么样"——近似正交的方向
- W_V：将原始 token 映射为" 被存储的信息"——高信息密度
- W_Q：将查询 token 映射为" 如何检索"——与相关键的内积大

这个元知识（" 如何建立联想"）是从训练数据中通过元学习/常规训练获得的。

4.8 Ba et al. (2016) 的快权重注意器

Ba et al. (2016) 提出了一个明确的快权重序列模型，解决" 如何把快权重与神经网络隐状态结合" 的问题。

设置 在每个时步，除了标准 RNN 隐状态 h_t，还维护一个快权重矩阵 $A_t \in \mathbb{R}^{d_h\times d_h}$。

写入规则

$$A_t = \lambda A_{t-1} + \eta\, h_t h_t^\top \tag{4.11}$$

（注：这里用 h_t 自身的外积写入，一种自关联写入。实际更泛化版本用两个不同的投影。）

快速注意力读取 （Slow-to-fast 交互）

标准 RNN 产生候选隐状态 h'_t，用快权重”精炼”它：

$$h'_t = f(W_s h_{t-1} + W_x x_t) \tag{4.12}$$

$$\tilde{h}_t^{(0)} = h'_t \tag{4.13}$$

$$\tilde{h}_t^{(l+1)} = f\left(h'_t + A_t \tilde{h}_t^{(l)}\right), \quad l = 0, 1, \ldots, L-1 \tag{4.14}$$

$$h_t = \tilde{h}_t^{(L)} \tag{4.15}$$

式 (4.14) 是一个简短的迭代精炼过程（通常 $L = 1$ 或 $L = 2$）：在快权重矩阵的”记忆场”中，通过几步迭代让当前状态向相关联的过去记忆靠拢。

注. 式 (4.14) 本质上是在做”快权重引导的最近邻检索”：$A_t \tilde{h}^{(l)}$ 产生当前状态在记忆空间中的”相关历史方向”，$h'_t + A_t h^{(l)}$ 将候选状态向这个方向拉近。

在几步学习（Few-shot Learning）任务上的实验结果 Ba et al. 在 Omniglot few-shot 分类上验证了快权重注意器：对比纯 LSTM（无快权重）vs 快权重 LSTM，快权重版本在 1-shot 和 5-shot 提升显著，说明快权重机制成功地帮助模型在测试序列内”学习”新类别。

4.9 HyperNetworks：快权重的参数生成推广

Ha et al. (2016) 把快权重的”上下文决定矩阵”思路推广到了更一般的框架：一个网络（**超网络 / HyperNetwork**）的输出是另一个网络的权重矩阵。

定义 4.4 (HyperNetwork)**.** 设主网络 f_W，超网络 g_θ。HyperNetwork 的设置是：

$$W = g_\theta(z)$$

其中 z 是某个”条件向量”（可以是任务描述、序列 embedding、层编号等），θ 是超网络的参数（慢权重），W 是被生成的主网络权重（快权重的推广）。

与快权重的联系 快权重 $\boldsymbol{W}_t = \sum_s \lambda^{t-s} v_s k_s^\top$ 本质上是”当前上下文 → 权重矩阵”的映射。HyperNetwork 把这个映射参数化为一个神经网络，使得生成更复杂的、非线性的权重成为可能。

在本书主线中的位置 HyperNetworks 直接启发了" 用一个学习好的程序来动态生成序列模型权重" 的思路，这是 TTT-Layer（第 八 章）的概念先驱：TTT-Layer 中，外部参数 θ 学习" 如何为每个序列生成一个好的内部 MLP 状态"。

4.10 连接 Memory-Augmented Neural Networks

Santoro et al. (2016) 的 Memory-Augmented Neural Networks（MANN）走了一条不同的路：把外部记忆实现为一个显式的**内存矩阵** $M \in \mathbb{R}^{N\times d}$（$N$ 个 d 维记忆槽），通过一组可微的读写头进行访问。

特性	快权重（Fast Weights）	外部记忆矩阵（MANN）
存储结构	$d \times d$ 矩阵	$N \times d$ 矩阵
写入机制	外积（不能精确擦除）	可寻址写入（可删改）
读取机制	矩阵-向量乘法	内容寻址（注意力）
容量	$O(d^2)$，隐式	N 个显式槽
梯度	可微（外积）	近似可微（软寻址）

两者都是在服务同一个目标：让模型能在前向传播中写入和读取序列历史信息，而不依赖梯度更新持久参数。它们的区别更多在于工程取舍而非原理上的对立。

4.11 和前后章节的关系

- **来自第 三 章**：元学习用内/外循环框架来区分" 快学" 与" 慢学"。本章的慢权重/快权重正是把这个框架嵌入到序列模型的**架构层面**，而不只是训练算法层面。
- **通向第 五 章**：Transformer 的线性注意力在数学上精确等价于快权重的外积积累 + 矩阵-向量查询（第 五 章将严格推导）。理解了快权重，线性注意力就不是" 去掉 softmax 的奇怪变体"，而是" 联想记忆的标准实现"。
- **通向第 八 章**：TTT-Layer 把快权重进一步升级——记忆矩阵不再用外积直接写入，而是通过梯度下降更新，使得记忆更新更" 有目的性"。

Lab 4：手写外积联想记忆并分析干扰

目标：实现快权重联想记忆，验证干扰界，理解容量限制。

1. **生成数据**：随机生成 $N = 10$ 个键值对，键 $k_i \in \mathbb{R}^{64}$（标准正态后 L2 归一化），值 $v_i \in \mathbb{R}^{64}$（标准正态）。
2. **写入**：用式 (4.5)（$\lambda = 0.9, \eta = 1$）按顺序写入所有键值对，每步保存

$\boldsymbol{W}_t$。

3. **精确查询**：用 $q = k_5$ 查询，计算 $\hat{v} = \boldsymbol{W}_{10}q$，度量 $\cos(\hat{v}, v_5)$（余弦相似度）。
4. **模糊查询**：用 $q = k_5 + 0.3 \cdot k_1$ 后 L2 归一化，观察结果变化。
5. **容量实验**：固定 $d = 64$，让 N 从 1 增加到 100，记录精确查询的余弦相似度曲线。在哪个 N 附近出现明显下降?
6. **高维救赎**：重复上述实验但用 $d = 512$，观察 N 的容忍上限如何变化。

预期结果：

- $d = 64, N \leq 5$：余弦相似度 > 0.95，检索基本精确
- $d = 64, N = 50$：余弦相似度明显下降（0.5–0.8）
- $d = 512$：相同 $N = 50$ 时依然高于 0.9

常见误解

常见误解 1：快权重就是 RNN 的隐藏状态。
纠正：隐藏状态 $h \in \mathbb{R}^d$ 是**向量**，随时间覆盖更新；快权重 $\boldsymbol{W} \in \mathbb{R}^{d\times d}$ 是**矩阵**，通过外积**积累**更新（不覆盖旧内容，而是叠加）。矩阵的容量（d^2 个自由度）远大于向量（d 个自由度），可以同时存储多个独立的键值联想。

常见误解

常见误解 2：快权重是一种”特殊技巧”，在现代网络中已经过时。
纠正：快权重不是种特殊技巧，而是一个**基本计算原语**。现代的 Transformer KV Cache、线性注意力状态矩阵、TTT-Layer 隐状态矩阵、Titans 神经记忆，归根结底都是快权重思想的不同实例化。理解外积写入和矩阵读取是理解这一切的基础。

常见误解

常见误解 3：快权重只能存储”近期”输入，无法保留长期历史。
纠正：当 $\lambda = 1$ 时，快权重对所有历史等权保留，理论上无遗忘。$\lambda < 1$ 则引入指数遗忘，这是设计选择，不是固有限制。Titans（第九章）通过门控机制实现了”选择性遗忘”，能在超长序列上同时保留关键的遥远记忆。

4.12 练习题

练习 4.1. 精确检索条件：设快权重矩阵为 $\boldsymbol{W} = \sum_{i=1}^{N} v_i k_i^\top$，查询 $q = k_j$（$\|k_j\| = 1$）。证明当被且仅当所有键两两正交时，$\boldsymbol{W} k_j = v_j$（精确检索）。如果 $\|k_j\|^2 = c \neq 1$，结果如何？

练习 4.2. 容量计算：设键向量由 d 维 Rademacher 随机变量（每维 $\pm 1/\sqrt{d}$）生成。两个键的内积期望 $\mathbb{E}[k_i^\top k_j]$（对 $i \neq j$）等于多少？方差？据此，用切比雪夫不等式估计：要让 N 个键的所有配对内积都小于 ϵ 的概率超过 $1-\delta$，需要 d 满足什么条件？

练习 4.3. 写入顺序的影响：设 $\lambda = 0.9$，按 k_1, k_2, k_3（$t = 1, 2, 3$）顺序写入。之后查询 $q = k_1$，输出是多少？再查询 $q = k_3$，输出是什么？比较两者中”目标值”的权重，说明为什么说快权重”偏好近期记忆”。

练习 4.4. 与 Gradient Descent 的联系：设我们想学习一个线性映射 $\hat{v} = \boldsymbol{W} k$，使得对所有 (k_t, v_t) 有 $\boldsymbol{W} k_t \approx v_t$。将外积更新 $\boldsymbol{W}_t = \boldsymbol{W}_{t-1} + \eta v_t k_t^\top$ 改写为对以下损失函数做一步梯度下降：$\ell(\boldsymbol{W}) = \frac{1}{2}\|\boldsymbol{W} k_t - v_t\|^2$。验证两者完全一致，即快权重更新 = 在线最小二乘的一步梯度下降。这一发现在第 十 章中有重要意义。

4.13 本章小结

- **快权重**是介于激活向量和持久参数之间的第三类存储：$d \times d$ 矩阵，在前向传播中通过外积积累，随用随弃。
- 外积写入：$\boldsymbol{W}_t = \lambda \boldsymbol{W}_{t-1} + \eta v_t k_t^\top$；遗忘因子 λ 控制时间尺度：$\lambda = 1$ 均等历史，$\lambda < 1$ 偏重近期。
- 矩阵查询：$\hat{v} = \boldsymbol{W}_t q = \sum_s \lambda^{t-s} v_s (k_s^\top q)$；相似度加权 + 时间加权的联合检索。
- 干扰量 $\leq (N-1)V\rho$（非正交键情形），在高维下 $\rho \to 0$，容量问题自然缓解。
- 慢权重/快权重的二分法是本书一个核心组织原则，贯穿第 4–9 章。
- **关键预告**：下一章将证明，线性注意力层的前向计算**数学等价于**快权重的外积积累 + 查询，这使得”注意力 = 联想记忆”不再只是比喻，而是可以写成等式。

读完本章，你应该能够：

- 解释为什么快权重矩阵比隐藏向量有更大的存储容量
- 推导外积写入规则（包括遗忘因子）

- 计算简单情况下的联想检索输出，并分析干扰
- 说明慢权重/快权重在序列模型中分别对应什么
- 把快权重更新识别为在线最小二乘的一步梯度步

第五章　Attention、Linear Attention 与递推视角

> "The attention mechanism is, at its heart, a differentiable key-value store."

5.1　本章想回答什么问题

上一章建立了快权重联想记忆的数学基础：外积写入 $\boldsymbol{W}_t \mathrel{+}= v_t k_t^\top$，矩阵乘法读取 $\hat{v} = \boldsymbol{W}_t q$。

本章做三件事：

1. 复习 Transformer 的标准 Softmax 注意力，建立统一符号（§5.3）
2. 推导线性注意力（去掉 Softmax），证明它可以被写成精确的递推形式（§5.4）
3. 建立线性注意力与快权重之间的**严格数学等价**（§5.5）

这三件事放在一起，说明了一个深刻结论：**注意力层的前向计算等价于在线更新的联想记忆**——这不只是比喻，而是可以写成等式。

理解这个等价是通向第 六–八 章所有内容的关键。

5.2　前置概念：内积、核函数与相似度度量

在进入注意力之前，我们先统一一个概念：**相似度函数**。

给定两个向量 $q, k \in \mathbb{R}^d$，常用的相似度函数有：

- **内积**：$\langle q, k \rangle = q^\top k$（未归一化，快，但数值随维度增大）
- **缩放内积**：$q^\top k / \sqrt{d}$（防止 softmax 梯度消失）
- **核函数**：$\kappa(q, k) \geq 0$，可以分解为 $\phi(q)^\top \phi(k)$（某些核）

定义 5.1 (正定核与特征映射)**.** 函数 $\kappa : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$ 是正定核，如果对任意有限点集，对应的 Gram 矩阵是正半定的。

若 $\kappa(q,k) = \phi(q)^\top \phi(k)$，则称 $\phi : \mathbb{R}^d \to \mathbb{R}^{d'}$ 为该核对应的特征映射。

关键例子：指数核与线性核

- **指数核**：$\kappa(q,k) = e^{q^\top k/\sqrt{d}}$。这个核不能被有限维 ϕ 精确表示（需要无穷维特征空间）。Softmax 注意力用的就是这个核（除了归一化）。
- **线性核**：$\kappa(q,k) = \phi(q)^\top \phi(k)$，其中 ϕ 是某个有限维映射。最简单的情形：$\phi = \mathrm{id}$（恒等映射），则 $\kappa(q,k) = q^\top k$。线性注意力用的就是这个核。

5.3 标准 Softmax 注意力的推导与理解

问题设置

给定一个长度为 N 的序列，每个位置有 token 表示 $x_i \in \mathbb{R}^d$（$i = 1, \ldots, N$）。自注意力的目标：让位置 i 的输出 y_i 能够”关注”所有（或部分）历史位置的信息。

三线性变换

首先，用可学习矩阵（慢权重）把每个 x_i 投影为三个角色：

$$q_i = W_Q x_i \in \mathbb{R}^{d_k} \qquad \text{（查询）} \tag{5.1}$$

$$k_i = W_K x_i \in \mathbb{R}^{d_k} \qquad \text{（键）} \tag{5.2}$$

$$v_i = W_V x_i \in \mathbb{R}^{d_v} \qquad \text{（值）} \tag{5.3}$$

注意力权重的计算

位置 i 对位置 j 的未归一化注意力分数：

$$s_{ij} = \frac{q_i^\top k_j}{\sqrt{d_k}} \tag{5.4}$$

除以 $\sqrt{d_k}$ 是为了稳定初始化时的梯度流：若 q_i, k_j 的各分量独立 $\mathcal{N}(0,1)$，则 $q_i^\top k_j$ 的方差为 d_k；除以 $\sqrt{d_k}$ 后方差固定为 1，防止 softmax 进入梯度消失的”尖峰”区域。

Softmax 归一化注意力权重（因果版本，只关注过去 $j \le i$）：

$$\alpha_{ij} = \frac{e^{s_{ij}}}{\sum_{j' \le i} e^{s_{ij'}}}, \quad j \le i \tag{5.5}$$

加权聚合输出

$$y_i = \sum_{j=1}^{i} \alpha_{ij} v_j \in \mathbb{R}^{d_v} \tag{5.6}$$

矩阵形式（全序列一次计算） 令 $Q = [q_1, \ldots, q_N]^\top \in \mathbb{R}^{N \times d_k}$（类似 K, V），则：

$$Y = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M_{\text{causal}}\right) V \tag{5.7}$$

其中 M_{causal} 是因果掩码矩阵（$i < j$ 的位置设为 $-\infty$，使其在 softmax 后为 0）。计算量：$O(N^2 d_k)$（矩阵乘法 $QK^\top$），内存 $O(N^2)$（存储注意力矩阵）。

为什么 $O(N^2)$ 是问题

当 $N = 100{,}000$（长文档推断），$N^2 = 10^{10}$——现代 GPU 存不下。这驱动了对" 避免显式计算 $N \times N$ 注意力矩阵" 的需求，即线性注意力。

5.4 从 Softmax 到线性注意力：去掉归一化的代价与收益

核心变换

用一个特征映射 $\phi : \mathbb{R}^{d_k} \to \mathbb{R}^{d'}$（$d'$ 可以等于 d_k）替代指数相似度：

$$\alpha_{ij}^{\text{linear}} \propto \phi(q_i)^\top \phi(k_j) \tag{5.8}$$

归一化后（暂时不写分母，先关注非归一化版）：

$$y_i^{\text{unnorm}} = \sum_{j \le i} \phi(q_i)^\top \phi(k_j) \cdot v_j \tag{5.9}$$

矩阵结合律：从 $O(N^2)$ 到 $O(Nd^2)$

关键一步：利用矩阵乘法结合律，改变括号位置：

$$y_i^{\text{unnorm}} = \sum_{j \le i} \phi(q_i)^\top \phi(k_j) \cdot v_j \tag{5.10}$$

$$= \phi(q_i)^\top \sum_{j \le i} \phi(k_j) v_j^\top \quad (\text{提出}\phi(q_i)) \tag{5.11}$$

$$= \phi(q_i)^\top \underbrace{\left[\sum_{j \le i} \phi(k_j) v_j^\top \right]}_{\boldsymbol{S}_i \in \mathbb{R}^{d' \times d_v}} \tag{5.12}$$

矩阵 $\boldsymbol{S}_i$ 可以递推计算：

$$\boldsymbol{S}_i = \boldsymbol{S}_{i-1} + \phi(k_i) v_i^\top \tag{5.13}$$

输出：

$$\boxed{\begin{array}{ll} \boldsymbol{S}_i = \boldsymbol{S}_{i-1} + \phi(k_i) v_i^\top & (\text{写入/状态更新}) \\ y_i = \phi(q_i)^\top \boldsymbol{S}_i & (\text{读取/输出}) \end{array}} \tag{5.14}$$

计算量分析：

- 每步状态更新：$O(d'd_v)$（外积）
- 每步输出计算：$O(d'd_v)$（向量-矩阵乘法）
- 总计：$O(Nd'd_v)$，线性于序列长度！
- 推断时内存：$O(d'd_v)$（只需存 $\boldsymbol{S}_t$），不随 N 增长！

常用特征映射 ϕ 的选取

方法	$\phi(x)$	维度 d'	特性
恒等	x	d_k	最简单，内积核
ELU+1	$\text{ELU}(x) + 1$	d_k	非负，近似 softmax
ReLU	$\max(0, x)$	d_k	稀疏
随机特征（RF）	$\cos(Rx + b)$	d'	近似高斯核

本书大多数场合用 $\phi = \text{id}$ 简化推导。真实系统会根据任务选择保证非负性的映射。

线性注意力失去了什么

Softmax 归一化有一个重要作用：让注意力权重 α_{ij} 对所有 j 加和为 1。这提供了隐式的” 注意力集中” 效果——模型可以选择只关注少数位置。

线性注意力的输出是所有历史值向量的**标量加权和**，没有强制稀疏，对于需要精确 token 匹配的任务（如 MQAR，多查询联想召回）可能表现较差。这是线性注意力与标准注意力主要的**定性**差异（不只是近似）。

5.5 线性注意力 = 快权重：严格的数学等价

现在我们建立本章最核心的结论。

定理 5.2 (线性注意力 ≡ 快权重序列模型 — Schlag et al., 2021)**.** *设线性注意力的参数 $W_K, W_V \in \mathbb{R}^{d\times d}$，特征映射 $\phi = id$，状态更新为式 (5.13)，学习率 $\eta = 1$，遗忘因子 $\lambda = 1$。*

则：

1. 状态矩阵 $\boldsymbol{S}_t = \sum_{j=1}^{t} k_j v_j^\top$（其中 $k_j = W_K x_j, v_j = W_V x_j$）。

2. 设快权重矩阵为 $\boldsymbol{W}_t = \sum_{j=1}^{t} v_j k_j^\top$（式 (4.5)，$\lambda = 1$）。

则 $\boldsymbol{S}_t = \boldsymbol{W}_t^\top$——线性注意力的状态矩阵是快权重矩阵的转置。

3. 两者的输出等价：

$$y_i = \boldsymbol{S}_i^\top \phi(q_i) = \boldsymbol{W}_i q_i = \hat{v}_i \tag{5.15}$$

（输出形式因矩阵转置约定而有所不同，但数学上代表同一计算。）

证明. 展开线性注意力状态：$\boldsymbol{S}_t = \sum_{j=1}^{t} k_j v_j^\top$。

展开快权重（$\lambda = 1, \eta = 1$）：$\boldsymbol{W}_t = \sum_{j=1}^{t} v_j k_j^\top$。

对比：$\boldsymbol{S}_t = (\boldsymbol{W}_t)^\top$（由于 $(AB^\top)^\top = BA^\top$，代入即得）。

输出：线性注意力 $y_i = \phi(q_i)^\top \boldsymbol{S}_i = q_i^\top \sum_j k_j v_j^\top$，快权重 $\hat{v}_i = \boldsymbol{W}_i q_i = \left(\sum_j v_j k_j^\top\right) q_i = \sum_j v_j (k_j^\top q_i)$。

两者等价（$q_i^\top \sum_j k_j v_j^\top$ 是行向量，其转置 $= \sum_j v_j (k_j^\top q_i)$）。 □

注. 转置方向的差异源于矩阵乘法的”行/列”约定不同，本质上是同一计算。重要的是：**两者都实现了相似度加权值召回**，区别仅在约定。

等价的实际含义

定理 5.2 告诉我们：

- Transformer 研究者从”注意力”这个词语出发设计的机制，和 Hinton 小组早在 1980–90 年代研究的”快权重联想记忆”，在数学上是同一个东西（在线性核下）。
- 这不是巧合，而是揭示了一个深层结构：**有效的上下文信息压缩，在某种意义下，必然收敛到外积积累 + 内积检索这种形式。**
- 后续的 TTT-Layer 进一步把外积更新替换为梯度下降更新，这是一个严格的”泛化”方向。

5.6 带遗忘因子的线性注意力与衰减位置偏置

在式 (5.14) 中加入遗忘因子 $\lambda \in (0,1)$：

$$\boldsymbol{S}_t = \lambda \boldsymbol{S}_{t-1} + \phi(k_t) v_t^\top \tag{5.16}$$

$$y_t = \phi(q_t)^\top \boldsymbol{S}_t \tag{5.17}$$

展开：$\boldsymbol{S}_t = \sum_{j \le t} \lambda^{t-j} \phi(k_j) v_j^\top$，所以

$$y_t = \sum_{j \le t} \lambda^{t-j} \cdot \phi(q_t)^\top \phi(k_j) \cdot v_j \tag{5.18}$$

这等价于对注意力权重加上了一个**时间衰减位置偏置** λ^{t-j}——越近的 token 权重越大。

这与 **RetNet**（Retentive Network，Sun et al.，2023）的设计思路完全对应：RetNet 在线性注意力上引入了衰减因子来提升序列建模效果。式 (5.18) 正是 RetNet 中”保留（Retention）”机制的核心公式。

5.7 Worked Example：用线性注意力做 MQAR

例子 5.3 (多查询联想召回（MQAR）)**. 序列**（$d = 2$，$\phi = \mathrm{id}$）：

$$(k_1 = e_1, v_1 = a), \quad (k_2 = e_2, v_2 = b), \quad q = e_1$$

（$e_1 = (1,0)^\top, e_2 = (0,1)^\top$，$a, b$ 是任意值向量）

状态更新：

$$\boldsymbol{S}_1 = e_1 a^\top = \begin{pmatrix} a^\top \\ 0^\top \end{pmatrix} \quad (\text{写入}(k_1, v_1)) \tag{5.19}$$

$$\boldsymbol{S}_2 = \boldsymbol{S}_1 + e_2 b^\top = \begin{pmatrix} a^\top \\ b^\top \end{pmatrix} \tag{5.20}$$

查询 $q = e_1$：

$$y = q^\top \boldsymbol{S}_2 = e_1^\top \begin{pmatrix} a^\top \\ b^\top \end{pmatrix} = a^\top$$

输出精确等于 $a = v_1$！

关键：正交键（$e_1 \perp e_2$）保证了无干扰检索。真实应用中键不完全正交，因此有残余干扰。

5.8 标准注意力为什么不能被递推

现在我们可以精确回答：为什么去掉 Softmax 才能得到递推？

Softmax 注意力：

$$y_i = \frac{\sum_{j\le i} e^{q_i^\top k_j/\sqrt{d}} v_j}{\sum_{j\le i} e^{q_i^\top k_j/\sqrt{d}}} \tag{5.21}$$

分子 $\sum_j e^{q_i^\top k_j} v_j$ 中的指数项是 q_i 的函数，**不能从**求和中提取出来。即 $e^{q_i^\top k_j} \neq \phi(q_i)^\top \phi(k_j)$ 对任何有限维 ϕ。

因此，对每个新查询 q_i，必须重新对所有 $j \le i$ 计算 $e^{q_i^\top k_j}$——这是 $O(N)$ 操作，总体 $O(N^2)$。

去掉 Softmax 用线性核 $\kappa(q,k) = \phi(q)^\top \phi(k)$，则可以替换矩阵结合律提取 $\phi(q_i)$，把 $\sum_j \phi(k_j) v_j^\top$ 预先计算为状态矩阵，避免重复计算。

视角（非定论）

完整 Softmax 注意力也可以从”联想记忆”视角理解——例如通过 Hopfield 网络的能量函数分析，或随机特征近似方法。但这些分析是近似的，而不是像线性情形那样的严格等价。

在本书中，凡是说”注意力 = 快权重”的地方，均专指线性注意力（无 Softmax）情形的严格等价。对于完整 Softmax 注意力，这是一个富有启发性的类比，但需要额外的近似或框架才能建立形式联系。

5.9 代表论文精读指南

Vaswani et al. (2017) —Attention Is All You Need 建立了多头自注意力（Multi-Head Self-Attention）的标准形式，并证明纯注意力机制可以取代 RNN 达到 SOTA 翻译效果。

本教材用处：提供标准符号和计算方案，是式 (5.4)–(5.6) 的原始来源。

Katharopoulos et al. (2020) —Transformers are RNNs 第一次明确推导线性注意力的递推形式（等价于我们的式 (5.14)），并实验验证其在长序列上的速度优势和性能权衡。

本教材用处：证明了递推写入 = 线性注意力，连接了 Transformer 和 RNN 两个社区。

Schlag et al. (2021) —Linear Transformers Are Secretly Fast Weight Programmers 建立了线性注意力与快权重之间的严格数学等价（定理 5.2），并提出了多种增强线性注意力能力的方案（如引入归一化、负权值等）。

本教材用处：本章定理 5.2 的主要来源。

5.10 和前后章节的关系

- **来自第 四 章**：快权重提供了数学基础——外积写入 + 矩阵读取。本章证明了线性注意力正是这个基础在神经网络中的自然实现。
- **通向第 六 章**：一旦我们知道线性注意力 = 快权重 = 在线最小二乘梯度步（见练习 4 of Ch.4），就可以问：Transformer 做 ICL 时，是不是在通过注意力计算" 运行梯度下降"？这正是第 六 章的主题。
- **通向第 八 章**：线性注意力的外积写入 $\boldsymbol{S}_t \mathrel{+}= k_t v_t^\top$ 不是唯一的写入方式——如果用梯度步替换，就得到 TTT-Layer 的核心更新规则。

Lab 5：实验对比线性注意力的并行实现与递推实现

目标：验证等价性，理解递推的推断优势。

1. 生成随机序列：$N = 256, d = 64$ 的 Q, K, V 矩阵（标准正态）
2. **方法 A（并行）**：计算 $Y = (QK^\top \odot \text{Causal_Mask}) \cdot V$ （不含 softmax，下三角掩码）
3. **方法 B（递推）**：写循环，按时步更新 $\boldsymbol{S}_t \mathrel{+}= k_t v_t^\top$，计算 $y_t = q_t^\top \boldsymbol{S}_t$
4. 验证 $\|Y_A - Y_B\|_\infty < 10^{-5}$（数值等价性）
5. 使用 `time.perf_counter()` 测量两种方法在 $N = 1024, 4096, 16384$ 时的运行时间
6. 绘制" 序列长度 vs 运行时间" 的双对数图，观察两者的增长速率
7. 测量 $N = 16384$ 时的峰值内存占用（并行版需存 N^2 注意力矩阵，递推版只需 $d \times d$）

预期结果：

- 数值等价性：$< 10^{-4}$（浮点误差）
- 时间：并行版 $\propto N^2$，递推版 $\propto N$（斜率不同的直线）
- 内存：并行版在 $N = 16384$ 时约 $16384^2 \times 4$ 字节 ≈ 1 GB，递推版约 $64 \times 64 \times 4$ 字节 = 16 KB

常见误解

常见误解 1：线性注意力只是" 没有 softmax 的弱化 Transformer"。
纠正：线性注意力是一套不同的权衡，不是单纯的弱化版本：(a) 对于推断效

率极为敏感的场景（长序列实时部署、移动端）线性注意力是关键；(b) 增加门控、衰减、选择机制后（RetNet、Mamba、xLSTM），与 Softmax 注意力的性能差距可以大幅缩小；(c) 对于"需要联想记忆"的任务，线性注意力有时反而是更合适的归纳偏置。

常见误解

常见误解 2：定理 5.2 说明 Softmax 注意力也等价于快权重。
纠正：定理明确说明这只在**线性核（去掉 Softmax）**下成立。Softmax 注意力使用指数核，无法写成有限维内积形式，因此不存在严格的快权重等价。这个区别很重要，不能混淆。

5.11 神经科学直觉与类比边界

神经科学直觉：注意力作为内容寻址记忆

类比对象：我们可以用**海马体的内容寻址记忆（Content-Addressable Memory）**来帮助理解注意力机制的核心计算。
海马体 CA3 区的循环连接可以实现**模式补全（Pattern Completion）**：给定部分线索（query），通过与存储记忆（keys）的相似度匹配，回忆出完整的记忆（values）。
共同之处（算法层面）：

海马体 CA3	注意力机制
存储模式 = 突触连接强度	键值缓存 = K, V 矩阵
线索输入 = 部分模式	查询 = q
模式补全 = 循环网络收敛	输出 = 相似度加权聚合
存储容量 $\propto$ 神经元数	KV cache $\propto N$（序列长度）

这个类比帮助理解什么：为什么"用相似度选择性检索"是一种有效的处理信息方式——因为它允许以线索（query）为索引，从大量存储中选择性地提取相关信息，而不是对所有信息做无差别处理。

这个类比在哪里失效

一、存储与检索的分离（实现层）

注意力机制	存储（写入 KV cache）和检索（query 乘法）是**分离的**线性代数操作。存储是精确的、无损的。
海马体 CA3	存储和检索通过**同一组**循环连接实现。存储是分布式的、有干扰的，检索依赖于网络动力学的收敛（吸引子）。

二、容量限制的本质不同

注意力的容量限制是**计算性的**（$O(N^2)$ 复杂度），而非存储性的——N 个 KV 对都被完整保留。海马体的容量限制是**表征性的**——由于干扰效应，存储的模式数不能超过约 0.14× 神经元数。

读者不应从这个类比推出：

- **[X]** “注意力等价于海马体的内容寻址”
- **[X]** “线性注意力的容量问题类似海马体的干扰”
- **[X]** “多头注意力对应海马体的多列结构”

类比成立的层次：计算目标——两者都实现了“基于相似度的选择性检索”。
类比失效的层次：存储/检索机制、容量限制的本质、动态行为均不同。

5.12 练习题

练习 5.1. 推导递推：从式 (5.9) 出发，利用矩阵结合律，完整地推导出式 (5.14)，给出每一步的代数理由。特别说明：为什么矩阵乘法结合律能够应用于这里，而 Softmax 计算中不能？

练习 5.2. 等价性验证：手动计算一个 $d = 2$，$N = 3$ 的例子（键正交），分别用并行公式 $y_i = \sum_{j \le i} q_i^\top k_j \cdot v_j$ 和递推公式 (5.14)，验证输出相同。

练习 5.3. 遗忘因子与位置偏置：设线性注意力有遗忘因子 λ，展开式 (5.18)，说明 token j（距当前位置 t）的有效注意力权重为 $\lambda^{t-j} \cdot q_t^\top k_j$。如果把 λ^{t-j} 理解为一种”位置偏置”，它与常见的 RoPE（旋转位置编码）有何不同？各有什么优缺点？

练习 5.4. 多头线性注意力：标准 Transformer 用了多头注意力（Multi-Head Attention，MHA）：将 q, k, v 分割为 H 个头，分别计算注意力后连接。对线性注意力同样可以用多头版本。设 $H = 8, d = 512, d_k = d_v = 64$，每个头独立维护一个 64×64 状态矩阵。推断时总的状态内存是多少？与单头 $d = 512$ 版本相比如何？

5.13 本章小结

- **Softmax 注意力**（$O(N^2)$）：相似度 = 指数核，需全序列矩阵乘法，无法递推。
- **线性注意力**（$O(Nd^2)$ 推断）：相似度 = 线性核，可递推：状态矩阵 $\boldsymbol{S}_t = \boldsymbol{S}_{t-1} + k_t v_t^\top$，输出 $y_t = q_t^\top \boldsymbol{S}_t$。
- **等价关系**（定理 5.2）：线性注意力状态矩阵 $\boldsymbol{S}_t^\top$ = 快权重矩阵 $\boldsymbol{W}_t$——在线性核下严格等价，不只是类比。
- 引入遗忘因子 λ 等价于加时间衰减位置偏置，是 RetNet 的核心设计之一。
- 这个等价链（快权重 ↔ 线性注意力 ↔ 在线最小二乘梯度步）是第 六–八 章所有分析的数学基础。

读完本章，你应该能够：

- 从头推导线性注意力的递推公式
- 证明线性注意力与快权重的数学等价（不依赖" 这只是类比"）
- 解释为什么 Softmax 阻碍了递推，而线性核不阻碍
- 分析线性注意力在 MQAR 任务中的精确检索条件
- 代码实现并验证并行版与递推版的数值等价性

第六章　上下文学习（ICL）到底是不是学习?

> "In-context learning is not a magical emergent property; it is the natural consequence of meta-learning an optimization algorithm within the forward pass."
>
> — 一种具有影响力的观点

6.1　本章想回答什么问题

自从 GPT-3 提出以来，**上下文学习（In-Context Learning, ICL）**成为了大模型的标志性能力：只需在 prompt（提示词）中给定几个输入-输出示例，模型就能对新输入做出正确预测，**这期间不改变任何模型参数**。

这引发了一个长期的争论：ICL 到底是不是真正的" 学习"?

- **观点 A（模式召回）**：模型在预训练时就已经见过类似任务。ICL 只是通过提供格式，帮助模型把已经知道的知识" 提取" 或" 定位" 出来。

- **观点 B（隐式优化）**：模型前向计算的本质是**在线运行了一个梯度下降（或其他优化）算法**，通过注意力机制用上下文示例构造了一个损失函数的梯度步。

本章将回答：在理论最清晰的设定（例如线性回归任务）下，Transformer 的前向传播**在数学上到底对 prompt 做了什么**？这一章将利用第 五 章建立的工具（注意力 = 快权重 = 联想记忆），展示注意力是如何等同于微步梯度下降的。这为后续所有动态测试时学习（TTT）提供了强有力的支持。

6.2 最小例子：在上下文中做线性回归

玩具问题 C：ICL 处理线性函数

考虑一个简单的任务类：所有可能的线性函数 $f_w(x) = w^\top x$。模型在预训练时，没见过具体的 w。预训练数据是一系列随机生成的序列：

$$P = (x_1, y_1, x_2, y_2, \ldots, x_k, y_k, x_{\text{query}})$$

其中 $w \sim \mathcal{N}(0, I)$，$x_i \sim \mathcal{N}(0, I)$，$y_i = w^\top x_i$。
测试时任务：给定一个新的序列（其蕴含一个全新、未知的 w^*）：

$$(x_1, y_1), (x_2, y_2), (x_3, y_3), q$$

Transformer 接收该序列的前向计算，输出 $\hat{y}$ 能否逼近 ${w^*}^\top q$？这是 Garg et al. (2022) 提出的经典实验。结果发现，纯 Transformer 在这个任务上的表现，精度甚至接近或持平最优的理论解（最小二乘法 / 岭回归）。它是怎么做到的？

6.3 直觉解释：从归纳头到梯度步

要理解 Transformer 的前向传播是在做什么，我们从最简单的机械操作入手。

6.3.1 感性解释 1：归纳头（Induction Heads）

如果任务是纯离散复制，Olsson et al. (2022) 发现了所谓的”归纳头”。它是一对注意力头，能执行这样的逻辑：

> “寻找过去是否出现过和当前 token A 相同的 token；如果出现过，看看它后面的 token 是 B；然后把预测结果推向 B。”

这类似于 $A \to B$ 的简单关联映射，是联想记忆的离散版本。但这种方法不足以解决连续的回归问题。

6.3.2 感性解释 2：一步梯度下降的构想

如果我们想拟合 w，我们需要最小化误差：$L(w) = \frac{1}{2}\sum_i (y_i - w^\top x_i)^2$。假设我们初始权重 $w_0 = 0$。通过一步梯度下降（学习率为 η）：

$$w_1 = w_0 - \eta \nabla L(w_0) = 0 + \eta \sum_i (y_i - 0) x_i = \eta \sum_i y_i x_i$$

对新输入 q 的预测就是：

$$\hat{y} = w_1^\top q = \eta \sum_i y_i (x_i^\top q)$$

请仔细看上式！预测结果是所有历史 y_i 的加权求和，权重是 x_i 和查询 q 的内积。这难道不正是我们在第 五 中推导的**线性注意力（即快权重）**的公式吗？

因此，只要把注意力矩阵的点积看作是在计算**样本相似度**（或者说内积核），多加几层注意力就是在执行**多步梯度下降**。

6.4 数学形式化：一层线性注意力等价于一步梯度下降

我们现在用严格的数学证明 Oswald et al. (2023b) 提出的一个核心推论。

设定结构

设输入序列包含格式化的 token，每个 token e_i 交替包含 x_i 或 y_i。为了方便分析，我们可以构造一种统一的" 数据 token" 格式，将 x_i 和预测残差放在同一个向量中：

$$e_i = \begin{pmatrix} x_i \\ y_i \\ 1 \end{pmatrix} \in \mathbb{R}^{d+2}$$

为了方便推导，我们考虑一个特定的前向网络操作。假设当前要预测查询 q 的输出值 y_q。

让模型维持一个" 当前参数估计" $W_0 = 0$（可以编码在额外的状态或者直接用 0 开始）。平方损失为 $\ell(W) = \frac{1}{2}\sum_i (Wx_i - y_i)^2$。其梯度为：

$$\nabla \ell(W) = \sum_i (Wx_i - y_i) x_i^\top$$

注意力层的前向操作

将线性注意力的参数手动设置为某种结构：键矩阵设为 $K = \mathrm{Diag}(I_d, 0, \ldots)$，也就是提取出 x_i：$k_i = x_i$。值矩阵设为 $V = \mathrm{Diag}(0, I_1, \ldots)$ 且组合误差项，用于提取误差信息。具体地，如果模型能通过某种残差连接提取 $(y_i - W_0 x_i)$：值向量设为 $v_i = y_i - W_0 x_i$（当 $W_0 = 0$ 时，就是 y_i）。

则线性注意力的输出状态矩阵（快权重矩阵）为：

$$\Delta W = \sum_i v_i k_i^\top = \sum_i (y_i - W_0 x_i) x_i^\top = -\nabla \ell(W_0) \tag{6.1}$$

也就是：**线性注意力层的权重矩阵 S，恰好等于负的损失函数梯度。**

对于查询 q，应用注意力的输出为：

$$\text{Output} = \Delta W \cdot q = \left(\sum_i y_i x_i^\top \right) q \tag{6.2}$$

如果在模型架构中加上残差连接 $W_1 = W_0 + \eta \Delta W$，则预测结果即为：

$$y_{\text{pred}} = W_1 q = (W_0 - \eta \nabla \ell(W_0)) q \tag{6.3}$$

Transformer 的前向计算是在线优化

这个构造性证明说明了：只需要构造简单的参数映射 W_K, W_V, W_Q，一层（线性）注意力层的前向传递操作，在数学上可以精确执行一步带动量的梯度下降；叠加 L 层这样的注意力层，就可以执行 L 步梯度下降。这就是为什么 Transformer 不需要显式反向传播，就能在上下文里完成模型的更新：**它的架构自然是一套并行前向执行梯度下降的物理引擎。**

定理 6.1 (ICL 的一步梯度下降等价性)**.** *设一层线性注意力层的投影矩阵为 $W_K, W_V \in \mathbb{R}^{(d+2)\times(d+2)}$，查询为线性变换后的 q。存在 W_K, W_V 的设置，使得对于任意上下文 $\{(x_i, y_i)\}_{i=1}^n$ 和查询 x_q，该层的输出等于以下一步梯度下降的结果：*

$$y_{pred} = W_1 x_q, \quad W_1 = W_0 - \eta \nabla_W \frac{1}{2} \sum_{i=1}^n \|W x_i - y_i\|_2^2 \Big|_{W=W_0} \tag{6.4}$$

其中 $W_0 = 0$，η 由注意力层的缩放因子控制。

注. 该定理的关键限定条件是 “线性注意力”（不使用 Softmax）和 “线性回归任务”（目标函数是二次的）。在一般的非线性任务上，ICL 的行为可能偏离精确的梯度下降，但实验表明仍保持 “近似梯度下降” 的特征。

多层推广：L 层注意力 $\approx$ L 步梯度下降

如果堆叠 L 层线性注意力，每一层都执行式 (6.1) $W_{l+1} = W_l - \eta_l \nabla \ell(W_l)$ 的操作，则 L 层叠加等价于 L 步梯度下降。

更进一步，Ahn et al. (2023) 证明了：如果允许注意力层之间插入前馈网络（FFN），前馈网络可以近似计算 Hessian 矩阵的逆 H^{-1}，使得每一步变为**预处理梯度下降**：

$$W_{l+1} = W_l - \eta_l H_l^{-1} \nabla \ell(W_l) \tag{6.5}$$

这解释了为什么实际的深层 Transformer 比简单的多步 SGD 收敛更快——它学到了一种隐式的二阶优化算法。

Worked Example：$d = 2$ 的 ICL 等价性验证

例子 6.2 (手算验证 ICL = 一步 GD). 设 $d = 2, W_0 = 0$，上下文为两个样本：$(x_1, y_1) = ((1,0)^\top, 3)$ 和 $(x_2, y_2) = ((0,1)^\top, 5)$。

方法 A：一步梯度下降

$$\nabla \ell(0) = \sum_{i=1}^{2} (0 \cdot x_i - y_i) x_i^\top = -3 \cdot (1,0) - 5 \cdot (0,1) = (-3, -5) \tag{6.6}$$

$$W_1 = 0 - \eta \cdot (-3, -5) = (\eta \cdot 3,\ \eta \cdot 5) \tag{6.7}$$

查询 $x_q = (1,0)^\top$：$y_{\text{pred}} = 3\eta$。

方法 B：线性注意力

设 $k_i = x_i$，$v_i = y_i$，查询 $q = x_q$。注意力输出：

$$y_{\text{attn}} = \frac{\sum_i (q^\top k_i) \cdot v_i}{\sum_i (q^\top k_i)} = \frac{(1 \cdot 3) + (0 \cdot 5)}{1 + 0} = 3 \tag{6.8}$$

取 $\eta = 1$ 时，$y_{\text{pred}} = 3\eta = 3 = y_{\text{attn}}$。**两者完全一致。**

查询 $x_q = (0,1)^\top$：$y_{\text{pred}} = 5\eta = 5$，线性注意力也给 5。

当键正交时（如本例），线性注意力精确实现了一步梯度下降的结果。当键非正交时，两者仍然在一阶近似下匹配，但会产生干扰项。

6.5 代表论文精读

Garg et al. (2022) —What Can Transformers Learn In-Context? 这篇论文通过控制实验证明：经过大规模预训练的 Transformer，仅凭上下文中的输入-输出示例，就能对未见过的线性回归、稀疏逻辑回归、甚至小型 MLP 任务达到接近理论最优的预测精度。这不是模式匹配——模型在预训练中从未见过这些具体的回归系数。

Oswald et al. (2023b) —Transformers learn in-context by gradient descent 本文给出了本章核心的构造性证明：在线性注意力和线性回归任务下，存在 W_K, W_V 的设置使得前向传播等价于一步梯度下降。同时在实证上验证：训练过的 Transformer 的内部权重更新模式与显式运行梯度下降的轨迹高度相似。

Ahn et al. (2023) 和 Zhang et al. (2024) 后续工作将等价性从一阶 SGD 扩展到**预处理梯度下降**和**二阶牛顿法**：通过在注意力层之间插入 FFN 来近似 Hessian 逆，Transformer 可以实现比普通多步 SGD 更快的收敛。这解释了为什么深层 Transformer 在 ICL 中的表现常常优于简单的最小二乘估计。

6.6 三层证据体系：ICL 的科学地位

这一节是本章最重要的教学内容。前面的推导说明了 “ICL 可以等价于梯度下降”，但读者必须理解这个主张的**精确适用范围**。我们把当前的证据分为三个清晰的层次。

第一层：在简化设定中已被严格证明的等价性

以下主张有构造性数学证明支持：

主张	证据
一层线性注意力可以精确执行一步 GD	定理 6.1：构造性证明
L 层线性注意力 $\approx L$ 步 GD	逐层归纳，每层维护残差
FFN + 注意力可以实现预处理 GD	Ahn et al. (2023)：Hessian 逆近似

关键限制条件：这些证明全部依赖于（a）线性注意力（无 Softmax），（b）线性回归任务（二次损失），（c）特定的 W_K, W_V 构造。

第二层：有方向性但不构成严格证明的机制证据

1. **归纳头（Induction Heads）** (Olsson et al., 2022)：在训练过的 Transformer 中发现了执行 “模式匹配-复制” 的注意力头对。这证明了模型确实学到了结构化的前向计算策略，但归纳头只能解释离散复制任务，不能解释连续回归中的 ICL。

2. **训练后的权重与 GD 轨迹的相似性**：Oswald et al. (2023b) 的实验表明，训练过的 Transformer 的内部表示与显式 GD 的中间结果高度相关。但这是行为相似性（behavioral similarity），不是构造性证明——模型可能通过不同的计算路径达到相似的输出。

3. **非线性任务上的 ICL 表现**：Garg et al. (2022) 展示了 Transformer 在 MLP 回归任务上也能 ICL，但此时没有严格的 “等价于 GD” 的证明，只有 “表现与 GD 近似一致” 的实验观察。

第三层：有启发性但仍属开放的解释性主张

视角（非定论）

以下主张是当前研究社区中有影响力的观点，但尚未被证明为一般性结论：

1. **“大型 LLM 在做自然语言 ICL 时，内部全局地执行了类似 GD 的算法”**——目前没有证据表明 GPT-4 规模的模型在处理翻译、推理等复杂任务时仍然在“运行梯度下降”。
2. **“ICL 是贝叶斯推断的近似”**——另一个有力的理论框架认为 ICL 是在隐式地进行后验推断（识别上下文对应的任务分布）。这个视角与优化视角在线性高斯设定下可以统一，但在更一般的情形下二者给出不同的预测。
3. **“预训练等价于元学习”**——功能上相似（外循环学习“如何做内循环”），但预训练的训练信号是下一 token 预测，不是显式的元学习目标。

6.7 等价性在哪里失效

本节明确列出定理 6.1 的每一条假设，并说明去掉该假设后等价性如何被破坏。

限制 1：Softmax 注意力打破严格等价

定理 6.1 的证明依赖于**线性注意力**（注意力权重是 $q^\top k_i$，无归一化）。标准的 Softmax 注意力对权重做了非线性归一化：

$$\alpha_i = \frac{\exp(q^\top k_i/\sqrt{d})}{\sum_j \exp(q^\top k_j/\sqrt{d})}$$

归一化使得注意力权重之间产生**竞争关系**（增大一个必然减小另一个），这在一般情形下不等价于任何线性优化算法的梯度步。

实际影响：这就是为什么大多数“ICL = GD”的理论论文都在线性注意力的假设下工作。对于使用 Softmax 的实际 Transformer，等价性是**近似的**，而非严格的。

限制 2：学习到的特征表示 $\neq$ 原始输入

证明中假设 $k_i = x_i$（键就是原始输入）。在训练过的 Transformer 中，$k_i = W_K x_i$ 是一个**学习到的投影**。这意味着模型可能在一个变换后的特征空间中“做 GD”，而这个特征空间本身是由预训练决定的。等价性仍然成立，但“等价于在哪个空间中的 GD”需要额外分析。

限制 3：有限深度限制优化步数

一个 L 层 Transformer 最多可以执行 L 步 “GD”。对于需要大量迭代才能收敛的任务（如高维稀疏回归），L 步可能远远不够。这与在线学习理论中 “需要 $O(d)$ 步才能达到统计最优” 的已知结果一致。

限制 4：离散 token 和非线性操作

真实的语言模型处理离散 token（通过嵌入层映射为向量），中间层包含 LayerNorm、ReLU/GELU 等非线性操作。这些操作在线性回归的分析框架中没有对应物。当前没有理论能证明 “包含所有这些操作的完整 Transformer 等价于某种优化算法”。

6.8 “学习” 在这里到底指什么

本书的后续章节将反复区分三种不同意义上的 “测试时学习”。本节建立这个关键分类法。

类型	机制	代表
参数更新	对模型参数 θ 执行显式梯度步。需要 `.backward()`。	TTT 2020（第二章）
状态更新	更新一个独立的隐状态 $\boldsymbol{S}_t$，θ 保持不变。状态在序列结束后丢弃。	TTT-Layer（第八章）、Titans（第九章）
隐式算法执行	前向传播的计算过程等价于某种优化算法。没有任何参数或状态被 “更新”。	ICL（本章）

ICL 的独特之处在于：它不改变任何参数，也不维护显式的可更新状态——它通过**前向传播的计算结构**隐式地实现了优化算法的效果。

这种区分为什么重要？因为它决定了系统的**可扩展性**：

- **参数更新**需要完整的反向传播计算图，代价最高。
- **状态更新**只需维护固定大小的状态，代价中等。
- **隐式执行**不需要任何额外机制，但依赖 KV cache 保存所有上下文。

后续章节（第七–十一章）的核心主题就是：如何在这三种 “学习方式” 之间找到最佳权衡。

6.9 和前后章节的关系

- **来自第 五 章**：第 5 章证明了“注意力 = 快权重（外积积累）”。本章在此基础上证明“注意力 = 在线梯度下降”（因为梯度的数学形式也是外积）。

- **来自第 三 章**：MAML 的外循环学习初始化，内循环做适应。类比地，Transformer 预训练（外循环）把“如何在前向传播中执行 GD”的规则编码进了 W_K, W_V, W_Q（慢参数）。

- **通向第 七 章**：ICL 的隐式 GD 依赖于 KV cache 保存所有上下文（$O(N^2)$ 计算、$O(N)$ 内存）。下一章的问题是：能否通过**显式的状态更新机制**把学习到的信息压缩进固定大小的参数矩阵，从而突破 KV cache 的内存瓶颈？

Lab 6: 验证 ICL 的内部梯度下降等价性

目标：手动验证一层带有残差连接的线性自注意力产生的结果，与明确对相同数据点跑 1 步梯度下降的结果相吻合。

1. 随机生成 $N = 20$ 对 $(x_i, y_i) \in \mathbb{R}^5 \times \mathbb{R}$，构建上下文矩阵 $X \in \mathbb{R}^{20\times 5}$ 和 $Y \in \mathbb{R}^{20\times 1}$。另生成查询 x_{query}。
2. **优化器方式**：从 $W = 0$ 开始求损失 $L = \frac{1}{2}\|XW - Y\|_F^2$。手动更新 $W' = 0 - \eta(-X^\top Y) = \eta X^\top Y$。预测为 $\hat{y}_{\text{GD}} = W'^\top x_{\text{query}} = \eta Y^\top X x_{\text{query}}$。
3. **注意力方式**：将 x_i 作为键 $(Q = x_{\text{query}}, K = X)$，$y_i$ 作为值 $(V = Y)$。实现纯线性注意力机制，且将学习率缩放比例放到投影权重里。
4. **比较**：观察两边的输出向量，在 η 对齐时它们应当毫无二致（误差限制在浮点截断内 $<$ 1e-6）。
5. **进阶探究（可选）**：如果在注意力前加入一个对 K 的特定线性变换 S_{pre}，能不能让两者的结果对应于加入 S_{pre} 的自适应学习率算法？

常见误解

常见误解 1：因为 ICL 等价于梯度下降，所以 Transformer 的 ICL 就是在做线性或逻辑回归。

纠正：虽然我们用线性回归作为构建解来证明等价性，但这不代表 Transformer 只能做这么基础的任务。这只证明了神经网络里存在表达“执行下降步”的物理机制（Existence proof）。在拥有庞大非线性 ReLU、门控网络且具有多层的真实模型里，它很可能是多级分层非线性投影中的一种高级优化。

常见误解

常见误解 2：“ICL 就是训练”，所以我们不再需要明确用 SGD 写训练循环。
纠正： ICL 只有能力对极其特定的短时适应任务生效（也就是“内循环”）。如果没有外循环利用海量真实人类文本作为自监督的负反馈（即传统的反向传播训练），它就不会获得能把自注意力组织成梯度下降运算器的这个本领。

常见误解

常见误解 3：ICL 到底是不是学习，在这个领域有唯一定论。
纠正： 本章所展示的主要是“优化视角”（Optimizer View）的研究议程，说明 ICL 在某些任务上**可以被等同对待**为微调或学习。但在复杂语言任务（特别是翻译、提取）中，贝叶斯推断视角（Bayesian Inference 认为是在隐式识别上下文对应的元任务概率）也同样有力并有效。两者在更深层的数学上并不矛盾。在阅读时应保持一定的批判和边界感。

6.10 神经科学直觉与类比边界

神经科学直觉：元学习与大脑的“学会如何学习”

类比对象： 前额叶皮层（PFC）的**元学习能力**可以帮助理解 Transformer 如何“学会在上下文中做梯度下降”。
Wang et al. (2018) 的研究表明，递归神经网络在大量不同任务上的元训练后，能够在 PFC 类的循环层中发展出 **隐式的强化学习算法**——网络的前向传播本身等价于执行 Bayes-optimal RL。
共同之处（算法层面）：

PFC 的元学习	Transformer 的 ICL
外循环 = DA 驱动的突触学习	外循环 = 大规模预训练
内循环 = PFC 活动动态	内循环 = 前向传播中的 GD
慢权重 = 突触强度	慢权重 = W_K, W_V, W_Q
快变量 = 瞬时活动模式	快变量 = 注意力权重 / KV cache

这个类比在哪里失效

一、学习算法的实现不同（实现层）

Transformer ICL	前向传播通过矩阵乘法精确执行梯度下降。“学会的算法” 编码在投影矩阵的参数值中。
PFC 元学习	学到的 “内循环算法” 分布在循环连接的动态中，以活动模式的形式存在。PFC 不做矩阵乘法，也不计算显式梯度。

二、“学会什么” 的验证标准不同

Transformer 的 ICL-as-GD 等价性可以被**构造性地证明**（在线性任务上，可以直接写出 W_K, W_V 的解析形式）。PFC 的元学习只有间接的行为证据——我们无法打开生物网络验证它 “内部在运行什么算法”。

读者不应从这个类比推出：

- **[X]** “Transformer 是 PFC 的计算模型”
- **[X]** “ICL 解释了人类的快速学习能力”
- **[X]** “预训练等价于多巴胺驱动的突触学习”

类比成立的层次：外循环学习 “如何做内循环” 的两层结构。
类比失效的层次：实现机制、可验证性、学习规模均不同。

6.11 练习题

练习 6.1. 注意力实现二阶更新：为了实现单步的普通梯度，值向量必须存入 $y_i - W_t x_i$（残差）。假设要实现具有二阶信息加速的迭代（比如加入 H^{-1} 作为类似预处理），请论述如果在注意力层的网络中间引入前馈全连接网络（FFN），是否能提供表达局部海森矩阵拟合的能力？

练习 6.2. Softmax 与线性：在本章定理证明的推断里，假设我们如果不使用线性注意力（去掉 Softmax），而是用原版的 Softmax，式 (6.1) 的权重更新还会等同于损失函数的精确梯度吗？这解释了为什么“ICL 作为严格梯度下降”的大多理论论文都会放宽或者去除 Softmax 约束条件？

练习 6.3. 时间与深度的代换：假定用 L 层的 Transformer 来处理一个上下文回归任务，这等效于进行多少步的隐式梯度下降？每步梯度使用的输入数据是什么（是全批量还是小批量）？

练习 6.4. 多变量解耦：如果在任务中，键 x_i 的维度是 100，但有效的相关特征只有 5 维。元学习外循环应该如何设定投影权重 W_K 来使得 ICL 更加抗噪？

练习 6.5. 元学习链接：将第 三 章中学过的基于模型不可知元学习（MAML）的概念，用至少三句话对应到在 Transformer 推断通过前向生成 ICL 的过程中去。如果 ICL 是 MAML 内循环，那什么是外循环？

6.12 本章小结

- **ICL 之谜**：仅靠前向计算，模型展现出了仿佛经过反复调优后的性能。
- **归纳头机制**：即使是复制或关联这样的机械操作，通过匹配前序 token，也能解释基础表现。
- **梯度下降构造**：研究证明：一个设计得当的（线性）自注意力层可以通过设置键 $= x_i$、值 $=$ 预测残差 的方式直接实现一步负梯度步的加权更新。
- **理论启示**：Transformer 的层级结构提供了一种原生的、高并发的在上下文中实施基于梯度的微调框架——这消除了传统上“训练与推理”泾渭分明的绝对壁墙。
- **解释层明确区分**：证明了“等价梯度机制的存在”，不代表当前的所有大语言模型就仅仅只在运行这个规则。“它能在数学上实现优化算法”是强解释视角的共识基石。

读完本章，你应该能够：

- 清晰地描述为什么自注意力不仅等价于快权重矩阵，还具有梯度更新同构。
- 手动推导和复现如何构建带有负梯度向量的注意力权重。
- 对比“模式提取”和“基于梯度在线微调”这两个关于 ICL 理性的两极化理论概念。
- 理解 Transformer 多层架构实际上是深度可展平的多步自适应学习器系统。

第七章　Learning to (Learn at Test Time)：通过架构学习微调

> "The architecture itself is an instantiation of the inner learning algorithm. Instead of running Python code to update parameters, we pass tokens through a neural network layer whose math is isomorphic to gradient descent."
>
> — 架构等同于优化算法

7.1　本章想回答什么问题

上一章（第六章）带来了一个令人着迷的视角：**Transformer 实际上是在前向传播中运行了隐式的梯度下降**。

但是，这立刻引出了一个关键的工程局限：为了让隐式梯度下降发挥作用，所有的"用于梯度的历史样本"（KV cache）都必须保存在显存中！Transformer 无法把学习到的历史真正的"融合"进某个小型状态里，它只能被动地持有整个序列（$O(N^2)$ 计算瓶颈）。为了处理极长上下文的上下文学习，这种显存暴涨是不可接受的。

本章探讨桥梁性的问题：

> 如果我们既然已经在进行"测试时的内部循环学习"（*Test-Time Training/Learning*），为什么不明确地把它固化或者设计为一种**具备常数级大容量内部参数**的网络层？能否通过"*Learning to (Learn at Test Time)*"这个明确目标，设计一种网络结构，其隐藏状态本身就是在测试时更新的学习参数？

本章将介绍这一重要转化。我们将讲解如何把原本需要数以万计甚至几十万个 token 外挂驻留的显存瓶颈，转化为用一个**动态学习网络**来完成任务；这将为下一章真正的 TTT Layers（从 RNN 的扩展发展而来）提供核心逻辑基础。

7.2 最小例子：在序列中背诵密码

玩具问题 D：长序列中的隐性规则修改

设想一个超长文档（例如一篇百万字的小说），其中零星地提到："顺便提一句，房间的门密码是 [4927]"。然后在 10 万个 token 之后，文章突然问："要进入该房间输入的密码是？"
如果模型依靠传统 Transformer：- 它必须保留这 10 万个 token 的全部键值对（KV cache）。- 因为它只能通过"查找"前文来提取 [4927]。
如果依靠本章讨论的自适应参数系统结构：- 当模型读到"密码是 [4927]"时，它的前向层（隐性具有梯度调节功能）触发了一次对**内部状态参数的写入**（把它内化成权重连接的一小步偏移）。- 此后 10 万个 token，哪怕它不再关注那句话的内存向量，这把隐藏参数的变化已经伴随整个模型的前向结构一直保留了下来。- 最终提问时，模型不再是通过海量搜索来寻找，而是直接由带有此偏差的"微型模型"通过常数运算提取出 [4927]。

这个玩具问题区分了两种截然不同的序列处理范式：提取式搜索 vs 状态吸收式学习。

7.3 直觉解释：模型内部的俄罗斯套娃

在经典的监督学习中，我们可以把它当做是在做双层嵌套优化（见 MAML，第三章）。外层处理整体数据，内层处理每个样本或任务。"Learning to (Learn at Test Time)"这个思想就是要把这个嵌套过程**压缩到一个时序过程内部**。

设想有一个主模型（相当于人脑的主网络），在流式接收文本时，主网络旁边挂着一本**可以反复被擦写的小记事本**（隐状态）。每读一个字，主网络不仅仅是要输出下一个字，它同时还要决定：1. 本字里的什么要素值得被记录？2. 向小记事本（以矩阵更新或内层前向反向步计算形式）写入多大程度的知识？3. 在未来的生成中，如何从记事本中读出关键信息作为补充依据？

这个小记事本，它本身就可以被理解为一个**具有自我更新能力的小型学习器（Inner Learner）**。在测试阶段，大模型的全局权重被冻结（不会有梯度发到主网络从而占用海量 GPU 集群运算量），但大模型在前向推演时，发出了对"子网络参数"（记事本矩阵）的步进优化指令。

因此，原本庞大的注意力 KV Cache 操作，就被置换为了对于一个小得多的独立优化器（或隐状态参数）的动态累积操作。

7.4 数学形式化：把隐状态作为学习器参数

我们从文献 Sun et al. (2023) 和后续发展出发，把 RNN 的状态视为参数来构建理论等价。

7.4.1 标准序列结构的隐藏层

在传统 RNN 中，状态更新极为简单：

$$h_t = f(Wh_{t-1} + Ux_t) \tag{7.1}$$

这里的 $h_t \in \mathbb{R}^d$ 是一个向量，其表达能力随时间序列暴增会迅速枯竭。这是为什么我们有了 LSTM，也有了最终摒弃 RNN 而走向 Transformer 的过去。

7.4.2 隐状态升级为一个微型模型

在第 五 线性注意力的推导中我们已经知道：去 softmax 后的状态矩阵实质上是：

$$\boldsymbol{S}_t = \lambda \boldsymbol{S}_{t-1} + k_t v_t^\top \in \mathbb{R}^{d\times d} \tag{7.2}$$

现在我们在理论层面上彻底打破限制，把状态升级为一个多层微型参数网络，例如它包含了一个线性映射算子（甚至是多层 MLP！）的权重，我们记作 W_t^{inner}。在前向推断过程（即测试时应用）到达时刻 t 时：

我们定义内部模型的输出预测机制为：

$$\hat{y}_t = W_{t-1}^{\text{inner}} x_t \tag{7.3}$$

内部状态学习的目标则是要最小化历史信息导致的内部误差：

$$\mathcal{L}_{\text{inner}}(W^{\text{inner}}) = \frac{1}{2}\left\|W_{t-1}^{\text{inner}} x_t - v_t\right\|^2 \tag{7.4}$$

那么通过微网络计算一次或多次的一步隐式随机梯度下降更新，状态转移变成了：

$$W_t^{\text{inner}} = W_{t-1}^{\text{inner}} - \eta \nabla_W \mathcal{L}_{\text{inner}}(W_{t-1}^{\text{inner}}) \tag{7.5}$$

$$= W_{t-1}^{\text{inner}} - \eta(W_{t-1}^{\text{inner}} x_t - v_t)x_t^\top \tag{7.6}$$

双重参数界限

注意这之中的奇妙区别：η, x_t, v_t 都是如何生成的？它们是由**全局慢参数（Slow Weights，大模型的投影矩阵等，在此处冻结）**经过预训练决定和输出的！而 W_t^{inner} 是**快参数（Fast Weights，或叫隐状态参数）**，在每输入一个 token 的同

时，由外围慢参数所输出的“梯度指令”自动进行了实时的更新！

定义 7.1 (Learning to (Learn at Test Time) 框架)**.** 设全局慢参数 $\theta = \{W_K, W_V, W_Q, \eta, \ldots\}$ 在预训练阶段通过外循环学习。对于推断时的每个 token x_t，慢参数产生键值对 $(k_t, v_t) = (W_K x_t, W_V x_t)$，快参数（内层模型）$W_t^{\text{inner}}$ 执行在线更新：

$$W_t^{\text{inner}} = W_t^{\text{inner}} - \eta \nabla_W \ell_{\text{inner}}(W_{t-1}^{\text{inner}}, k_t, v_t) \tag{7.7}$$

外循环的优化目标：

$$\min_{\theta} \sum_{\text{seq}} \sum_{t=1}^{T} \mathcal{L}_{\text{outer}}(y_t, f(x_t; W_t^{\text{inner}}(\theta))) \tag{7.8}$$

其中 $W_t^{\text{inner}}(\theta)$ 依赖于 θ，因为 θ 决定了 (k_t, v_t, η) 的生成方式。

关键对比：ICL vs. Learning to TTT

理解 Learning to TTT 的最重要方式是与第 六 章的 ICL 进行对比：

	ICL（第 六 章）	**Learning to TTT**
更新对象	无显式更新——编码在注意力权重中	显式更新 W^{inner}
状态复杂度	$O(N)$ 的 KV cache（随序列线性增长）	$O(d^2)$ 的参数矩阵（固定大小）
如何“学到”学习	W_K, W_V, W_Q 在预训练中使前向传播等价于 GD	外循环端到端训练内循环的所有超参
长序列处理	$O(N^2)$（Softmax）或 $O(Nd^2)$（线性）	$O(Td^2)$，与序列长度线性

ICL 的核心局限是：它依赖于把所有上下文保持为 token 供注意力检索，因此内存随上下文长度线性增长。Learning to TTT 的核心创新是：通过把信息**压缩进固定大小的参数矩阵**，实现了与序列长度无关的内存占用——代价是压缩过程中不可避免的信息损失。

注 (与快权重的关系). 比较式 (7.7) 和第 四 章的快权重更新 $\boldsymbol{W}_t = \boldsymbol{W}_{t-1} + v_t k_t^\top$，可以发现：快权重是 Learning to TTT 在 $\ell_{\text{inner}} = \frac{1}{2}\|Wk_t - v_t\|^2$ 且忽略预测反馈（$W_{t-1}k_t \approx 0$）时的特例。Learning to TTT 保留了预测反馈项，因此能够实现更精确的记忆和更好的干扰抑制。这也是为什么 TTT-Layer 在长序列上常常优于线性注意力。

伪代码：单序列的前向推断

以下伪代码展示了一个 Learning to TTT 层在推断时处理长度为 T 的序列的完整流程。注意：θ 在推断时完全冻结，所有的 “学习” 都发生在 W^{inner} 的更新中。

输入：序列 $x_1, x_2, \ldots, x_T$，慢参数 $\theta = \{W_K, W_V, W_Q, \eta\}$
初始化：$W_0^{\text{inner}} \leftarrow 0_{d\times d}$ （或由 θ 提供的初始化）

for $t = 1$ **to** T **do**：

1. **投影**：$k_t = W_K x_t$，$v_t = W_V x_t$，$q_t = W_Q x_t$
2. **内层损失**：$\ell_t = \frac{1}{2}\|W_{t-1}^{\text{inner}} k_t - v_t\|_2^2$
3. **内层梯度**：$g_t = (W_{t-1}^{\text{inner}} k_t - v_t) k_t^\top$
4. **状态更新**：$W_t^{\text{inner}} = W_{t-1}^{\text{inner}} - \eta \cdot g_t$
5. **输出**：$y_t = q_t^\top W_t^{\text{inner}} k_t + x_t$ （残差连接）

end for

关键观察：

- 步骤 1–5 全部是前向计算，没有调用 `.backward()`。
- 步骤 3 的 “梯度” 是对 W^{inner} 的解析表达式，不需要构建计算图。
- 步骤 4 是唯一的 “学习” 操作——它修改了隐状态 W^{inner}，等价于在线回归的一步 SGD。
- 每一步的计算量为 $O(d^2)$，与序列长度无关。

Worked Example：$d = 2$ 的 3 步状态追踪

例子 7.2 (手动追踪 3 步 Learning to TTT)**.** 设 $d = 2$，$\eta = 0.5$，$W_0^{\text{inner}} = 0$。假设慢参数已经将输入投影为以下键值对：

t	k_t	v_t	目标：$W^{\text{inner}} k_t \approx v_t$
1	$(1,0)^\top$	$(3,0)^\top$	第一列应趋近 $(3,0)$
2	$(0,1)^\top$	$(0,5)^\top$	第二列应趋近 $(0,5)$
3	$(1,1)^\top$	$(3,5)^\top$	应输出两列之和

Step 1（$t = 1$）：

$$\text{预测} = W_0 k_1 = 0 \cdot (1,0)^\top = (0,0)^\top \tag{7.9}$$

$$\text{残差} = (0,0)^\top - (3,0)^\top = (-3,0)^\top \tag{7.10}$$

$$g_1 = (-3,0)^\top (1,0) = \begin{pmatrix} -3 & 0 \\ 0 & 0 \end{pmatrix} \tag{7.11}$$

$$W_1 = 0 - 0.5 \cdot g_1 = \begin{pmatrix} 1.5 & 0 \\ 0 & 0 \end{pmatrix} \tag{7.12}$$

Step 2（$t = 2$）：

$$\text{预测} = W_1 k_2 = \begin{pmatrix} 1.5 & 0 \\ 0 & 0 \end{pmatrix} (0,1)^\top = (0,0)^\top \tag{7.13}$$

$$\text{残差} = (0,0)^\top - (0,5)^\top = (0,-5)^\top \tag{7.14}$$

$$g_2 = (0,-5)^\top (0,1) = \begin{pmatrix} 0 & 0 \\ 0 & -5 \end{pmatrix} \tag{7.15}$$

$$W_2 = \begin{pmatrix} 1.5 & 0 \\ 0 & 0 \end{pmatrix} - 0.5 \cdot \begin{pmatrix} 0 & 0 \\ 0 & -5 \end{pmatrix} = \begin{pmatrix} 1.5 & 0 \\ 0 & 2.5 \end{pmatrix} \tag{7.16}$$

Step 3（$t = 3$）：

$$\text{预测} = W_2 k_3 = \begin{pmatrix} 1.5 & 0 \\ 0 & 2.5 \end{pmatrix} (1,1)^\top = (1.5, 2.5)^\top \tag{7.17}$$

$$\text{目标} = (3,5)^\top \tag{7.18}$$

$$\text{相对误差} = \frac{\|(1.5,2.5) - (3,5)\|}{\|(3,5)\|} = \frac{\sqrt{1.5^2 + 2.5^2}}{\sqrt{34}} \approx 0.50 \tag{7.19}$$

观察：经过 3 步，W^{inner} 已经“学到了”部分目标映射（对角矩阵的一半）。如果用 $\eta = 1.0$，第 3 步的预测误差将更小。如果继续更多步或使用更大的内层模型（MLP），误差会进一步降低——这就是 TTT-MLP 优于 TTT-Linear 的原因。

对比固定 RNN：一个传统 GRU 在同样的 3 步中只会对 $d = 2$ 的隐状态向量执行门控操作。它的“记忆容量”是 $d = 2$ 个标量。而 Learning to TTT 的 W^{inner} 有 $d^2 = 4$ 个参数。这就是“隐状态是参数”带来的容量优势。

从双层优化到架构设计的映射

本章的核心概念转变可以通过以下对应关系总结：

双层优化概念	架构对应物	来源章节
内层参数 ϕ	隐层状态 W_t^{inner}	本章
内层损失 ℓ_{inner}	序列局部重建损失	本章
内层优化器	状态更新规则（SGD/外积）	Ch. 5, 8
外层参数 θ	全局投影矩阵 W_K, W_V, W_Q	Ch. 3
外层梯度	通过展开的内循环反向传播	Ch. 3

这个映射揭示了 Learning to TTT 的方法论意义：它把第 三 章中“令人费解的嵌套优化”转化为了“可以直接在 GPU 上高效执行的网络层”。双层优化不再是一种训练技巧，而是成为了**序列模型的设计原理**。

7.5 代表论文精读

Sun et al. (2023) —Learning to (Learn at Test Time) 本文是重新唤醒现代研究者关注“大网络结构自身就是优化算法”的代表性文章之一。由于在海量序列建模场景下无法承担自注意力的平方式代价，文章重新梳理了内部循环（如何构造小模型的测试时间更新目标）和外部循环（如何预训练那些能生成内部所需变量特征的全局网络权重）。文章不仅统一了对上下文能力和泛化的追求，并实际上宣告了一种可无限扩展的深度状态重构 RNN（或状态机）的架构范式。

RNNs with Expressive Hidden States (2024) 后续研究认为，既然内循环被确定为了一个独立的测试时多级优化任务，那么我们为什么要限制它的规模？如果只用简单的一层线性网络当作 W^{inner} 并不具有高容量记忆，如果我们**直接用一个前馈神经网络层（MLP）作为隐状态去被执行在线学习**，那么模型的记忆容量和表达能力能随其 MLP 宽度指数增长——这是通向下一章（TTT Layers）的关键概念奠基。

7.6 和前后章节的关系

- **来自第 四 章和第 六 章**：我们分别推导出了外积写入作为记忆矩阵与隐藏于 Transformer 的梯度下降过程。本章则通过确立“Learning to Learn”的工程原则，解释了如何将上述理念融合进一种脱离了固定序列缓存结构（KV Cache）的“测试态在流学习”网络中。

- **通向第 八 章**：本章指明了：内部隐状态可以而且应当是一个可经由内部损失函数定义、用梯度在运行状态中迭代的“微型子模型”。下一章我们就将直接

详细拆解它的终极实现形态："基于实际可微网络的 TTT-Layers"，去了解隐状态是如何最终脱胎换骨的。

Lab 7: 通过定义内部损失反推网络状态更新层

目标：手动证明基于规则更新的状态机，可以重构并实现成内部损失下的一步梯度层。

1. 使用 numpy 进行操作。设输入数据为一个维度是 $d = 8$ 的序列。
2. **外部参数部分**：生成一个固定的 8×8 慢参数投影矩阵，将当前的输入序列转换出当前拟要被记忆的样本和拟用于驱动误差的目标。
3. **内部优化循环**：定义 $\ell = \mathrm{MSE}(Sh_{\text{proj}}, v_{\text{target}})$。使用 `loss.backward()` 手动实施一次基于负梯度的状态 S （内部模型）累积相加修改。
4. 把它和之前手动拼凑并写死的 Hebb 规则进行数值比较。
5. **反思**：在实际的 GPU 上，为了这**哪怕仅仅一步小的模型学习**都要拉起完整的后向优化算法图会导致怎样极高的同步开销与资源消耗？引发出后续提出 TTT Done Right 论文的工程优化缘由所在。

常见误解

常见误解 1：这里所谓的 Test-Time Training 就是本书第一二章里用自监督学习在遇到分布外数据时的整体测试修正框架。

纠正：尽管精神一致，但实现层面有天壤之别！第一二章的普通 TTT 是一种算法技巧，依赖 PyTorch 中实际构建出的大型反向传播后计算图并调用真实硬件里的 Optimizer 修正模型全局参数 θ。而本章（和以后的 TTT 架构系列），是把这一过程物理化缩小为由前向连接构成的信息累加，且直接嵌在一个或每一个网络层内部，作为原生的"隐状态修正方式"，这是架构层面的革命！它是硬件友好的推断态执行，并且不需要额外调出梯度反算功能（或者说是把特定形式内嵌固化为了不需要图结构的特殊运算而已）。

常见误解

常见误解 2：内部模型越复杂，外网络对它的把控力和实际表现肯定就等比例越来越好。

纠正：这面临极大的优化脆弱期。作为内层目标的梯度如果在训练初中期稍有不适当（例如外层因为对数步长参数分配不均等给出了太剧烈的目标驱动梯度要求），内部模型的更新会迅速失控并输出爆炸级的特征，导致灾难性的崩溃。因此内循环的学习结构在当前极力追求工程稳定（限制参数复杂性、强加规范化）。

7.7 神经科学直觉与类比边界

神经科学直觉：突触元可塑性——“学习如何学习”的生物对应

类比对象：神经科学中的**突触元可塑性（Synaptic Metaplasticity）**可以帮助理解 “Learning to (Learn at Test Time)” 的核心思想。

元可塑性是指**学习规则本身可以被调控**——突触不仅可以改变强度（可塑性），其可塑性的**阈值和速率**也可以被历史活动改变。Abraham & Bear (1996) 将此称为 “可塑性的可塑性”。

共同之处（算法层面）：

突触元可塑性	Learning to TTT
突触可塑性 = 内层学习	W^{inner} 的在线更新
元可塑性 = 调控学习速率	外层慢参数设定 η, W_K, W_V
历史活动调控阈值	预训练决定内循环的 “如何学习”

这个类比帮助理解什么：为什么需要 “学习如何学习”——固定的学习规则无法适应所有情境，需要一个更高层级的过程来调节学习行为本身。

这个类比在哪里失效

一、调控的精度不同（实现层）

Learning to TTT	外层通过端到端反向传播精确优化内层的所有超参数（学习率、投影矩阵、目标函数系数）。
突触元可塑性	调控通过局部的生化信号（钙离子浓度、受体磷酸化状态等）实现，精度远低于梯度优化，且无法做端到端调控。

二、“内模型” 的概念不同

在 Learning to TTT 中，内层状态 W^{inner} 是一个明确的参数化模型，有输入、输出和目标函数。在生物系统中，没有 “突触矩阵作为一个模型” 的对应概念——突触可塑性是分布式的、局部的。

读者不应从这个类比推出：

- **[X]** “内层网络等价于一组突触”
- **[X]** “元可塑性通过梯度实现”
- **[X]** “Learning to TTT 解释了大脑的元学习”

类比成立的层次：“学习规则本身可被调节” 的两层结构。
类比失效的层次：调控精度、内模型的抽象程度均不同。

7.8 练习题

练习 7.1. 内外变量区分：当构建一个隐状态其实是模型参数本身的序列处理网络时。详细列举从预处理 $t = 0$ 到计算进入时刻 $t = 1$ 时，哪些结构变量将被系统外环（在数以 PB 计的数据中心训练期间）永久性进行更改；哪些将被推断过程中丢弃、但需要在输入时以毫秒级时间进行频繁的更小范围更新？并回答这样层叠的性能和存储占用的界是分别如何分布的？

练习 7.2. 泛化：从线性内模型到 MLP：如果内部的网络状态不只是一个参数矩阵 $\boldsymbol{S}$，而是一个带激活函数的的两层网络构型（例如 $Y_t^{\text{inner}} = S_2(\sigma(S_1 X_t))$）。使用偏导数的链式法则大概展示如果要对其由输入损失产生的一步更新，其表达式将产生多少个非线性截断状态。为什么在此时我们无法像线性回归下轻易替换为简单的直接状态积相加项？

7.9 本章小结

- **容量与显存的矛盾**：传统的 ICL 由于严重依赖保存一切旧有键值导致注意力呈平方消耗无法支持极长文本。解决的核心是将信息压缩（或说化解为）更复杂容量更大的权重状态。
- **微模型即状态**：隐状态不应当只是一种降维抽取映射或单维度向量累加机，它实质上应当被设定为具备自我闭环纠偏驱动能力的微型学习参数组（例如完整的可调网络）。
- **内外目标的时序固化**：通过把外部全局数据集用于培养稳定大尺度输出模型特征参数网络，把处理单次生成生命周期内的高频流数据分配给以梯度累积规则工作、具有多梯度方向性的内置网络块，实现了“通过学习（架构上的元调整），使得大模型获得了它具有直接以动态微调作为预测基础底座”的能力。
- **桥梁预备**：这就是为何我们会把这类网络称为”Learning to (Learn at Test Time)” 系列的前奏。下文将进一步将这个“内部隐状态即隐模型”真正全面落实。

读完本章，你应该能够：

- 把传统的预训练-微调分离模型转移到能在一个推理过程中体现为双层操作流的复杂结构视角。

- 解释为什么测试时不需要再跑外部的反向传播代码或梯度系统即能够被评价为“微网络层上的测试时自适应训练”。
- 对比“隐状态是一个可微模型层”这个观点相较于“隐状态就是一个特征浓缩集合”所具备的理论包覆优势及存储上的常数复杂度优势。

第八章　TTT Layers：隐藏状态就是学习器

> "Do not compress history into a vector. Compress it into the weights of a neural network layer. Let the hidden state be a learner."
>
> — RNN 架构的现代觉醒

8.1　本章想回答什么问题

通过第 七 章的学习，我们已经理解了核心理念：与其用庞大且缓慢的 Transformer 通过上下文硬背（把所有信息全挂在注意力的缓存里），不如将这部分信息提纯后作为局部修正，用在推断流程内部微调一个随时间更新的网络层（Inner Learner）。

这正是现代 TTT-Layer（Test-Time Training 序列预测架构）的核心主张。那么，既然这也是一种带有隐状态随序列流式变迁的架构：

> *为什么不直接叫它普通 RNN 呢？TTT 层和经典的 LSTM 或 GRU 有什么本质的区别？到底是怎么把整个隐状态变成一个完整的模型的？以及它背后的计算是如何在硬件资源限制下真正"做对"（Done Right）的？*

本章的主旨是展示如何从理论模型正式向具备工业潜力的主流高极算力算法设计迈入。我们将通过明确区分隐状态为"参数堆"而非"特征集"带来的容量雪崩式增长，以及从 TTT-Linear 到 TTT-MLP 的演进，来真正解答 TTT-Layer 是如何革新序列学习体系的。

8.2 最小例子：需要不断刷新规则的过滤筛选

玩具问题 E：隐态参数容量的极限测试

假设你是一个审核助手，你正在阅读一本 1000 万字不断改变时空规则的架空日记。在第 1 章，规则是："红色石头代表有毒，绿色石头代表大补。"但在第 30 章突然一句话说："由于世界线变动，从现在起，绿石头有毒，红色是解药。但是蓝石头对火系法师有增益且带弱毒性附加……"（这是一系列非常复杂的高阶条件嵌套规则）

传统 LSTM 模型：它的内部状态是一个有限长度的 d 维向量。即使经过精巧的遗忘门和记忆门。把一套复杂的非线性逻辑挤压进一个线性表示极度吃力，它的表示必定趋向于最近频繁出现的统计平均。

TTT-Layer：它的内部状态不再是向量。它可能直接是一层由 $d \times d$ 元素组成的权重矩阵，有的更甚至是一个深度特征分离的双层网络 W_1 和 W_2 及中间的非线性激活。当遇到新的规则描述时，模型前向利用类似损失梯度的调整方程对这些 W_1 进行写入操作。在这个过程中，TTT 层内保留的是一个"小型网络"，能够容纳足够切分不同特征空间（如分辨是说火法的还是水法的）的高阶非线性信息，这就是所谓隐态扩容。

8.3 直觉解释：为什么是"就是学习器"

在经典的深度学习语境中：- **参数（Parameters/Weights）**指的是经过长训练定格，带有跨序列、普适性世界规则的静止记忆体。- **激活与隐藏层（Activations/Hidden States）**指的是输入数据和静止参数进行乘法激荡后产生的临时流动信息集。

TTT-Layer 打破了这两者的物理隔离。它认为：对于一个足够长的序列片段而言，这段独立子空间内同样有着高度自洽和需深度处理的映射函数结构需要被保持。它将传统层间的流转的隐藏状态 h_t 形式上升级成了带有容量存储性质的网络权重形态 $\boldsymbol{W}_t$。在这个子微粒环境内：- 隐藏状态 $\boldsymbol{W}_t$ 对特定输入的映射充当了前向层产生表征作用。- 对每个新的局部语料产生新梯度用来调整 $\boldsymbol{W}_t$ 就是它自带的学习吸收消化系统。

所以，"隐藏状态就是学习器"是在说，不要只把输出带下去，把处理输出的算法参数块本身带着在时间序列里一步一步往前走！

8.4 数学形式化：从 TTT-Linear 到 TTT-MLP

8.4.1 TTT-Linear 的规范方程

设定外部全局参数包含一系列慢权重用来投影：将输入的信号转换出查询（q_t），键（k_t）和值（v_t）。隐状态设定为具有同等维度的线性变换矩阵 $\boldsymbol{S}_t \in \mathbb{R}^{d\times d}$。

在此时刻，隐状态学习器的目标被定义为重建或映射误差：

$$\ell_t = \frac{1}{2}\|\boldsymbol{S}_{t-1}k_t - v_t\|^2 \tag{8.1}$$

隐状态由该时刻数据的负梯度更新，加上固定遗忘因子与规范化：

$$\nabla_{\boldsymbol{S}}\ell_t = (\boldsymbol{S}_{t-1}k_t - v_t)k_t^\top \tag{8.2}$$

$$\boldsymbol{S}_t = \boldsymbol{S}_{t-1} - \eta\nabla_{\boldsymbol{S}}\ell_t \tag{8.3}$$

对当前输入的处理并输出则是：

$$o_t = \boldsymbol{S}_t q_t \tag{8.4}$$

这基本复刻和加强了第 四 章所介绍由于外积导致写入和查询一致性关联的原则，但将更新动机正式包裹在损失梯度的定义内，而不仅仅是一个粗略的 Hebb 法则。

8.4.2 向高容量跃迁：TTT-MLP

因为线性映射的容量天花板是显而易见的（哪怕你增加 d，只能拟合高度线性的关联）。Sun et al. (2024b) 提出了极具理论野心的变体：如果 $\boldsymbol{S}$ 不仅仅是个大矩阵，如果隐状态是一个包含了两级独立变换，和非线性 $\sigma(\cdot)$ 乃至层归一化等更为复杂的隐性超微型神经网络子模块呢？此时内部参数为两个独立权重集合：$\mathbf{W}^{(1)}, \mathbf{W}^{(2)}$ 内部损失变为对微型网络的非线性反向传播：

$$\ell_t = \frac{1}{2}\|\mathbf{W}_{t-1}^{(2)}\sigma(\mathbf{W}_{t-1}^{(1)}k_t) - v_t\|^2 \tag{8.5}$$

更新时，利用经典的链式反向传播原理：

$$\mathbf{W}_t^{(i)} = \mathbf{W}_{t-1}^{(i)} - \eta\nabla_{\mathbf{W}^{(i)}}\ell_t \quad \text{for } i \in \{1,2\} \tag{8.6}$$

至此，TTT-Layer 的形态达成了完全进化：它在测试数据流面前推断推进的过程中，实实在在地于图节点级拉起并运行了一个包含多网络和独立反向链式的训练操作，这等同于把一套完全功能的深度神经网络装入进了隐态的包裹中！

表达力突破与 TTT-MLP

普通的线性关注能够记忆 $O(d)$ 个正交键。这在长程语料中几乎不能支撑大规模检索任务。引入多层和非线性处理的意义，是在没有将维度指数放大的前提下，指数放大了所能折叠区分的局部特征几何空间，在不使用 Attention 的条件下，真正达到和超越了无限注意力能产生的映射极值。

神经科学直觉：任务内可塑性与感觉-运动校准

类比对象：我们把**任务内在线可塑性（within-task fast plasticity）**和**感觉-运动校准（sensorimotor calibration）**作为帮助理解 TTT-Layer 的神经科学类比。

共同之处（计算角色层面）：

- TTT-Layer 的内层参数 $\boldsymbol{S}_t$ 在处理单个序列的过程中实时调整，任务结束后状态不持久化；感觉-运动系统在执行一项新任务（如适应棱镜偏移）时，在单次任务内迅速调整内部映射，不依赖跨任务的长期训练。
- 两者都体现了**在线误差驱动的快速适应**：TTT-Layer 用 $\ell_t = \frac{1}{2}\|\boldsymbol{S}_{t-1}k_t - v_t\|^2$ 的梯度来调整隐状态；感觉-运动校准由预测误差（感觉结果与期望结果之差）驱使小脑等结构快速修正内部模型。

这个类比帮助理解什么：为什么要在推断时执行梯度步？因为当前序列的统计结构与训练分布不同，就像人在陌生光照或重力环境下需要当场调整感觉-运动映射，而不是依赖出厂时烧录的固定参数。两者的设计直觉都源于”当前情境需要当场适配，而不是调用固定规则”。

这个类比在哪里失效

一、学习信号的来源完全不同（学习信号层）

TTT-Layer 的内层更新依赖**显式构造的损失函数** $\ell_t = \frac{1}{2}\|\boldsymbol{S}_{t-1}k_t - v_t\|^2$ 及其精确梯度 $\nabla_{\boldsymbol{S}}\ell_t$，后者通过标准反向传播计算，需要维护整个计算图。生物感觉-运动校准依赖**局部误差信号**（如小脑浦肯野细胞接收到的攀爬纤维信号），是高度局部化的，没有全局目标函数，也没有反向传播。

二、TTT-Layer 的”任务”边界是人为设定的

TTT-Layer 的隐状态随序列流重置，这是工程上的人为切分；感觉-运动校准的任务边界由解剖结构、注意和情境来界定，不是人为设计的参数。

三、”以梯度步作为学习规则”不是生物合理的（实现机制层）

内层梯度下降依赖全局反向传播；生物可塑性的规则（如 STDP、调制性可塑性）是局部的，实现代价完全不同，不存在”链式法则”在生物中的直接对应。

读者不应推出：

- □ TTT-Layer 实现了大脑的感觉-运动校准机制
- □ 大脑的在线适应就是在执行梯度下降
- □ TTT-Layer 的 ℓ_t 等同于神经预测误差信号

类比在**计算角色**层面（在线误差驱动的快速适应）成立；在**学习信号来源**和**实现机制**层面失效。

8.5 代表论文精读与发展

Sun et al. (2024b) —Learning to (Learn at Test Time): RNNs with Expressive Hidden States 这篇长文通过细致地消融实验论述了既然可以利用隐性优化，为什么使用简单的特征更新不足以取代自注意力机制的根源（因为容量局限）。这篇论文正式介绍了 TTT-Linear 到 TTT-MLP 的过渡结构，并定义了这个架构为何被称为广义 RNN 的一种究极演变。

Sun et al. (2025) —Test-Time Training Done Right 虽然上述愿景完美，在实际硬件中，“为每一步序列做微型反向传播”的实现让现成的 PyTorch 后端和 GPU 利用率在并行化流水线断裂的情况下极其低下。因为反向计算对于每个时间步骤强依赖。这篇重要论文通过对 TTT 的设计架构优化（比如通过重构其多头并行投影计算图）并使用低阶的定制并行前缀和扫面算子与张量重组技术，实现了 $O(\log N)$（甚至是常数感知内）并行时间，从底层突破了“TTT 因为自回归计算极度缓慢”导致无法进行万亿参数级并行预训练的枷锁，这是把理想拉进实用大厂模型的转折之作。

8.6 与经典序列模型的系统对比

隐状态的容量对比

不同序列模型的核心差异在于隐状态的结构和容量。

模型	隐状态	更新规则	容量	可否在线
GRU	$h_t \in \mathbb{R}^d$	门控插值	$O(d)$	是
LSTM	(c_t, h_t)	遗忘门 + 输入门	$O(d)$	是
线性注意力	$\boldsymbol{S}_t \in \mathbb{R}^{d\times d}$	外积累积	$O(d^2)$	是
TTT-Linear	$\boldsymbol{S}_t \in \mathbb{R}^{d\times d}$	在线 SGD	$O(d^2)$	是
TTT-MLP	$W_t^{(1)}, W_t^{(2)}$	非线性反传	$O(d'^2)$	是
Transformer	KV cache	追加存储	$O(Nd)$	部分

关键观察：

- GRU/LSTM 的 $O(d)$ 容量瓶颈意味着所有历史信息必须压缩到一个向量中。当 $N \gg d$ 时信息损失不可避免。
- TTT-Linear 和线性注意力都使用 $d \times d$ 矩阵，但更新规则不同（见第十章的严格分析）。
- Transformer 的 KV cache 容量线性增长，延迟和内存代价为 $O(N)$。

计算复杂度对比

模型	每步计算	N 步总计	可并行
标准注意力	$O(Nd)$	$O(N^2d)$	完全并行
线性注意力	$O(d^2)$	$O(Nd^2)$	递推式
TTT-Linear	$O(d^2)$	$O(Nd^2)$	递推 *
TTT-MLP	$O(d'^2)$	$O(Nd'^2)$	递推 *
LSTM / GRU	$O(d)$	$O(Nd)$	递推式

*：通过前缀扫描技术可实现 $O(\log N)$ 并行深度。

伪代码：TTT-Linear 推断流程

输入：序列 $x_1, \ldots, x_T$，慢参数 $\theta = \{W_K, W_V, W_Q, \eta\}$
初始化：$\boldsymbol{S}_0 \leftarrow 0_{d\times d}$

for $t = 1$ **to** T **do**：

1. $k_t = W_K x_t,\ v_t = W_V x_t,\ q_t = W_Q x_t$
2. $\text{pred}_t = \boldsymbol{S}_{t-1} k_t$ // 当前预测
3. $e_t = \text{pred}_t - v_t$ // 预测误差
4. $\boldsymbol{S}_t = \boldsymbol{S}_{t-1} - \eta \cdot e_t k_t^\top$ // SGD 步
5. $o_t = \boldsymbol{S}_t q_t$ // 读取输出

end for

与线性注意力的区别在步骤 2–3：线性注意力忽略 pred_t，直接执行 $\boldsymbol{S}_t = \boldsymbol{S}_{t-1} + v_t k_t^\top$。TTT-Linear 保留预测反馈项，避免重复写入。

8.7 工程实现：从理论到硬件

将 TTT-Layer 从论文变为系统级组件需要解决三个工程挑战：

1. **串行依赖**：$\boldsymbol{S}_t$ 依赖于 $\boldsymbol{S}_{t-1}$，天然递推，无法直接并行。解决方案：将更新公式重写为关联运算，使用前缀扫描算法在 $O(\log T)$ 并行深度内完成。
2. **内存管理**：TTT-MLP 的内层反向传播需保存中间激活。长序列时 T 个时间步的激活可能超出显存。使用**梯度检查点**：只保存每隔 B 步的 $\boldsymbol{S}_{tB}$，需要时从最近检查点重新前向计算。
3. **训练时的双重反传**：预训练 TTT-Layer 需对内层学习过程求外层梯度，即通过展开的 T 步 SGD 反向传播。当 T 大时，梯度可能消失或爆炸。截断反传（truncated BPTT）和 stop-gradient 是常用缓解手段。

8.8 和前后章节的关系

- **相对第 七 章**：第 7 章提供为什么这样设计的架构元学习观念。本章着眼于物理计算结构的严谨公式以及容量的跃迁（MLP 结构取代普通线性投影矩阵）。
- **对立与继承于经典 RNN**：在形式上，它们依然维护时间切片的状态。但这个状态不再是传递信息的桥梁，而是处理信息的功能模块本身。
- **引出下一章的矛盾（第 九 章 Titans）**：不管这个随时间步连续更新的网络层（Inner Learner）容量做多大，它仍然面临一个时间跨度带来的客观现实——它只有**一个整体记忆系统**。随着梯度在时间轴上无限向前跑去，无论多么抗干扰的设计，百万序列后仍会存在“覆盖与遗忘（catastrophic forgetting）”的危机。所以接下来，必须将记忆进行短视窗和长远时间的切割体系。

Lab 8: TTT-Linear 一级矩阵手动更新与序列遗忘追踪

目标：手动编写一层具有一步负梯度在线纠偏更新能力的简化层，并探究它的遗忘表现。

1. **数据初始化**：选取序列长 $N = 50$，隐层 $d = 32$。
2. 在早期输入 k_{10}, v_{10}，并观察在这之后不再投入类似特征。
3. 结合标准梯度迭代，每有一轮新数据经过，就针对 $\boldsymbol{S}_t$ 以恒定的步长更新。
4. 到了最后第 50 步，向系统中发出 $q = k_{10}$ 作为查询输入；观察输出预测值与当初所存 v_{10} 之间的关联衰退情况。
5. 尝试加入遗忘乘法因子 $\lambda = 0.98$ 来调整矩阵更新方式。

通过实验能够深刻意识到“虽然矩阵容量很高，可以被更新修正并给出高拟合分数；但连续使用无选择的梯度擦写带来的信号不可逆覆盖是亟需管理的大问题。”

常见误解

常见误解 1：因为有“MLP 层被反向传播修改权重去了”，这就一定会耗尽主存空间，并且时间也需要翻番再循环。

纠正：这与外层网络使用全局海量节点的数据分布式反向传播不是一回事。隐藏态微模型的容量往往只有几 K 到最高几十 K 参数规模，它的优化是在局部单一算子栈内并发瞬间结算完毕的。尤其采用 TTT Done Right 思想将其与预编译的多重计算分离后，并不阻碍全网络的宏观显存和总体 throughput 速度。

常见误解

常见误解 2：TTT-Layer 取代了整个预训练基础。也就是说可以直接通过这个机制，不用任何长达几个月的语料处理也能进行逻辑推演。

纠正：绝对不能。驱动微模型参数修改的核心素材——q_t, k_t, v_t 本身是极度浓缩的高度结构化语义表示。这些高质量语义张量的生成，全部由固定在外部的主 Transformer 特征抽离模块主理并给出。没有外层浩瀚语料进行几十轮甚至更久的强逻辑元知识提取，“内部学习器”只会拿到完全无意义的噪声并瞎乱更新自己，彻底崩溃。这再度回到“外循环产生学习表示方法，内循环解决新适配”的本质。

8.9 练习题

练习 8.1. 计算复杂度比较：设上下文长度为 N，特征维度为 d。对于标准的全局自我注意力机制框架，它的前向推断关于序列长度耗时是什么量级？对比于具有内部线性更新态的单层的 TTT-Linear 模型（在不需要回溯更新历史只执行推演的前提下），其执行消耗量级是什么样的？两者差距对于 $N > 10^5$ 意味着什么？

练习 8.2. MLP 与梯度的关系：如果隐式优化模块是带有简单 ReLU 的 $\mathbf{W}^{(2)} \max(0, \mathbf{W}^{(1)} k_t)$，写出根据链式求偏导后产生的参数梯度项公式要素，并说明为了获取这个要素，外部大计算系统在预训练规划这个架构时，计算图需要留存多少个跨步特征作为基础导数链支持以维持整个模型的预微调不报错（即使在最后使用特定并行内核代替常规调用）？

练习 8.3. 过早吸收：假定外部学习到了一条“不好的提取规则”，导致在处理语句时产生了一个噪声峰值键和值对被投喂给 TTT-MLP，根据学习规则这一噪声会被大幅度地消化写入状态本身，从而由于权重的持续保留从而污染之后几千词的判断。为了反制这种情况（相比于自注意力机制因为没有固定写死能够更加灵活改变关注点分配的优势），TTT 框架需要在慢网络（外层投影层）学得额外的什么强能力设计以过滤或防止高误信传播？你能联想到基于此引发的哪些改进思路（如阈值拦截等）？

8.10 本章小结

- **重铸隐状态**：摒弃传统状态的有限单级压缩特质，让每个状态位本质上升格为微型的参数库，它是用来承担并解决长上下文中无尽特定知识关联适配的关键模块结构。
- **公式级集成**：建立起了具有特定损失规范和反向学习迭代更新支持功能的内置状态机数学准则：ℓ_t 及随之而来的 $\nabla\ell_t$ 实证了架构中实施全微观模型实时部署的可能性。
- **MLP 的指数破局**：简单的二次扩展并不能真正具备和无限计算抗衡的高度，真正让能力产生变现的跨度是要让具有包含非线性的层结构被允许内置为学习更新单元，以此让子空间中实现真正的强聚类记忆化特征保持和隔离提取。
- **实施障碍解套**：硬件约束始终是核心矛盾，只有借助于前沿的流计算处理（扫描式快速核运算与矩阵前置处理分割等“Done Right”式的解法）才真正意义上让全时动态学习具有并跑甚超越常规注意力的经济及实际实施价值。

读完本章，你应该能够：

- 准确辨识使用常规注意力、线性注意力和本章使用 TTT 参数层所维护的具体推进行为与内存持有形态上的主要代际差异。
- 理解把线性提取公式延伸为具两级复合神经深度的内部优化层这一演化的核心理论动力（表达与容量边界的跨越）。
- 用双层框架解释为什么外部固定全局参数的存在不能被舍弃而是前提。
- 熟知伴随全局持续性参数推移所固生附带不可避免的过载失控或严重遗忘关联趋势（这将直接催生多周期门控设计的登场）。

第九章　Titans 与多时间尺度记忆：如何在长河中钉住一根针

> "Memory is not a single thing. It is a family of mechanisms operating at different speeds, capacities, and timescales."
>
> — 互补学习系统理论（CLS），McClelland et al., 1995

9.1　本章想回答什么问题

第 八 章给了我们一个能力强大的机制：把隐藏状态升级为一个实时更新的微型神经网络，使模型在处理序列的过程中能够持续" 消化" 新出现的信息。

但这带来了一个不可回避的矛盾：

> 如果这个微型网络对每一个到来的 *token* 都做梯度更新，那么经过足够长的序列（比如一百万字），早期写入的信息早就被后续的无数次更新覆盖殆尽了。这就是**灾难性遗忘（*Catastrophic Forgetting*）**——一个系统在学新东西的同时不可逆地忘掉旧东西。

本章的问题是：**能否在一个模型内同时维护" 紧密追踪当前上下文的短时记忆" 和" 在超长序列上稳定保留关键信息的长时记忆"?**

Titans (Behrouz et al., 2025c) 给出的回答是：让这两种记忆机制在架构上分离，并设计一个叫做**惊讶度（Surprise）**的可微门控算子，使系统只在遇到" 高度出乎意料的输入" 时才触发对长时记忆的大幅度写入，从而集中有限的记忆容量，保护已存储的信息不被平凡噪声覆盖。

读完本章，你将能够：

- 推导 Surprise 门控写入规则，说明它为何减少平凡输入对记忆的干扰
- 解释 Titans 的三类记忆如何组合输出
- 用互补学习系统（CLS）理论作为直觉，但同时指出模型与生物系统的明确差异

- 识别”灾难性遗忘”问题在哪些架构设计假设下会出现，在哪些设计下得到缓解

9.2 最小例子：大海捞针

玩具问题 F：百万 Token 草垛寻针（Needle in A Haystack）

设置：序列总长 $T = 10^6$ tokens。在位置 $t = 100$ 处出现一条唯一的高价值信息，称之为”针”：

$$(k_{\text{needle}}, v_{\text{needle}}) = (\text{”暗号”}, \text{”红狐狸跳墙”})$$

此后 $t = 101$ 到 $t = 10^6$ 的所有输入均为统计上平凡的噪声（”草垛”）。任务：在 $t = 10^6$ 处给定查询 $q \approx k_{\text{needle}}$，输出 $\hat{v} \approx v_{\text{needle}}$。

基准方案（朴素 TTT-Layer）：每个 token 到来时都对内层参数 $\boldsymbol{S}_t$ 执行一步梯度更新：

$$\boldsymbol{S}_t = \boldsymbol{S}_{t-1} - \eta \nabla_{\boldsymbol{S}} \ell(\boldsymbol{S}_{t-1}, k_t, v_t)$$

问题：$10^6 - 100 = 999{,}900$ 次对噪声的梯度更新，会把 $t = 100$ 时写入的 $(k_{\text{needle}}, v_{\text{needle}})$ 完全覆盖。$t = 10^6$ 时查询，输出将接近噪声分布的均值，”针”已不可追回。

Titans 方案：设计一个标量门控 $s_t \in [0, 1]$，称为”惊讶度”。对噪声输入，$s_t \approx 0$，长时记忆模块的参数基本不变；对”针”输入，$s_t \approx 1$，执行大幅度写入。

结果：10^6 次更新中，只有极少数高惊讶度时刻真正改变长时记忆，大量噪声被屏蔽，”针”在 $t = 10^6$ 时依然可以被检索出来。

这个玩具问题揭示了核心矛盾：*无差别的在线更新会把重要信息淹没在噪声中*。解决方案的关键是**选择性写入**——但如何使”选择”本身可微、可学、可训练?

9.3 直觉解释：三种记忆的分工

Titans 的设计基于一个简单的原则：**不同类型的信息应当由不同的存储机制处理**。

具体来说，Titans 并联三类模块：

1. **短时记忆（Short-term Memory，STM）**：标准的滑动窗口注意力。只看最近的 W 个 token。精确度极高，但容量有限（$O(W)$）。适合处理精细的语法结构和局部因果关系。

2. **长时记忆（Long-term Neural Memory，$\mathcal{M}$）**：一个权重矩阵 Φ_t，随序列流式在线更新，通过 Surprise 门控保护关键信息。适合存储跨越长距离的高价值事实。

3. **持久记忆（Persistent Memory）**：少量固定的可学习 token，与输入无关。可理解为模型从预训练中固化下来的通用先验知识，推断时不更新。

最终输出是三类模块输出的加权融合，权重通过可学习的门控参数 α 控制：

$$y_t = \alpha_1 \cdot \text{output}_{\text{STM}}(x_t) + \alpha_2 \cdot \text{output}_{\mathcal{M}}(x_t) + \alpha_3 \cdot \text{output}_{\text{Persistent}}(x_t)$$

设计直觉：不同信息的" 保质期" 不同

短时精确信息（" 刚才那个词是什么？ "）需要立刻取用又立刻丢掉——短时记忆模块胜任。长期重要信息（" 这个序列开头出现的特殊标记是什么？ "）需要穿越大量噪声被保留——长时记忆模块负责。通用世界知识（"Paris 是 France 的首都"）随预训练固化，推断时不应改动——持久记忆负责。

9.4 数学形式化：惊讶度门控写入

本节给出长时记忆模块 $\mathcal{M}$ 的完整形式化。

Step 1：定义问题

长时记忆模块的任务是：给定当前键值对 (k_t, v_t)，维护一个映射 $k \mapsto v$ 的参数化函数 $M_\Phi(\cdot)$，使得 $M_{\Phi_t}(k_t) \approx v_t$。

这是一个**在线回归问题**（参见第十章的统一视角）：隐式的损失函数是

$$\ell_t(\Phi) \;=\; \tfrac{1}{2}\big\|M_\Phi(k_t) - v_t\big\|_2^2 \tag{9.1}$$

如果直接用梯度步更新（朴素 TTT-Layer）：

$$\Phi_t^{\text{naive}} = \Phi_{t-1} - \eta\, \nabla_\Phi \ell_t(\Phi_{t-1}) \tag{9.2}$$

这在噪声主导的情况下会导致灾难性遗忘，原因在 Section 9.2 中已说明。

Step 2：引入惊讶度的形式化

定义 9.1 (惊讶度（Surprise）). 在时刻 t，给定当前参数 Φ_{t-1} 和输入 (k_t, v_t)，惊讶度 $s_t \in [0, 1]$ 定义为模型预测误差的归一化度量：

$$s_t \;=\; \sigma\big(\gamma \cdot \big\|M_{\Phi_{t-1}}(k_t) - v_t\big\|_2 - \beta\big) \tag{9.3}$$

其中 $\sigma(\cdot)$ 是 sigmoid 函数，$\gamma > 0$ 是缩放系数，$\beta \in \mathbb{R}$ 是偏置。

变量含义：

- $M_{\Phi_{t-1}}(k_t)$：当前参数对 k_t 的预测输出 $\hat{v}_t$
- $\left\|\hat{v}_t - v_t\right\|_2$：预测误差的 ℓ_2 范数
- γ, β：可通过训练学习的超参数（或固定为常数）

如何读这个公式：当 $\hat{v}_t \approx v_t$（模型已经"知道"这条信息）时，$\left\|\hat{v}_t - v_t\right\|_2 \approx 0$，因此 $s_t \approx \sigma(-\beta) \approx 0$（偏置 $\beta > 0$ 时）。当 $\hat{v}_t$ 与 v_t 相差较大（高度出乎预料）时，$s_t \approx 1$。这使得 s_t 成为一个天然的"选择性写入强度控制器"。

Step 3：Surprise 门控更新规则

将惊讶度作为梯度步长的乘性系数：

$$\boxed{\Phi_t \;=\; \Phi_{t-1} \;-\; \eta \cdot s_t \cdot \nabla_\Phi \ell_t(\Phi_{t-1}) \;+\; \mu\, \Delta\Phi_{t-1}} \tag{9.4}$$

变量含义：

- $\eta > 0$：基础学习率
- s_t：Surprise 门控系数（见公式 (9.3)）
- $\nabla_\Phi \ell_t(\Phi_{t-1})$：损失对参数的梯度
- $\mu\, \Delta\Phi_{t-1}$：动量项，平滑历史梯度方向

为什么这个公式能解决灾难性遗忘?

当输入为噪声时，$s_t \approx 0$，方程 (9.4) 退化为 $\Phi_t \approx \Phi_{t-1}$（加上小幅动量修正）——参数几乎不变。当输入为高价值信息时，$s_t \approx 1$，方程退化为标准梯度步——参数显著更新以吸收新信息。

因此，噪声不累积对参数的影响，关键信息才能持久写入。

Step 4：读取（Retrieval）

给定查询 q_t（由慢权重投影 $q_t = W_Q x_t$ 得到），长时记忆的输出为：

$$\hat{v}_t^{\mathcal{M}} \;=\; M_{\Phi_t}(q_t) \tag{9.5}$$

即直接用更新后的参数 Φ_t 对查询做前向计算。

注. $M_\Phi(\cdot)$ 的具体形式在 Titans 中为一个小型的多层感知机（MLP），与第 八 章的 TTT-MLP 结构类似。这使得 M_Φ 具有非线性逼近能力，能够存储比线性矩阵更复杂的关联关系。

9.5 Worked Example：三步追踪 Surprise 的行为

例子 9.2 (Surprise 门控行为的数值追踪)**.** 设 M_Φ 为一个线性模型（简化），$\Phi_0 = \mathbf{0}$，$\eta = 1$，$\mu = 0$，$\beta = 0$，$\gamma = 5$。

时刻 $t = 1$（写入针）：$(k_1, v_1) = (e_1, 2)$（e_1 为第一个标准基向量）

$$\begin{aligned}
\hat{v}_1 &= \Phi_0 k_1 = 0 \\
s_1 &= \sigma(5 \cdot |0 - 2| - 0) = \sigma(10) \approx 1 \\
\Phi_1 &= \Phi_0 - 1 \cdot 1 \cdot (-2 \cdot e_1^\top) = 2\, e_1 e_1^\top \quad \text{（写入成功）}
\end{aligned}$$

时刻 $t = 2$（噪声）：$(k_2, v_2) = (e_2, \epsilon)$，$\epsilon = 0.01$

$$\begin{aligned}
\hat{v}_2 &= \Phi_1 k_2 = 2\, e_1 e_1^\top e_2 = 0 \quad (e_1, e_2 \text{ 正交}) \\
s_2 &= \sigma(5 \cdot |0 - 0.01|) = \sigma(0.05) \approx 0.51 \\
\Phi_2 &= \Phi_1 - 1 \cdot 0.51 \cdot \nabla \ell_2
\end{aligned}$$

由于 $k_2 = e_2$ 方向与 $k_1 = e_1$ 正交，噪声更新只作用在 e_2 方向，对 e_1 方向（针所在）无干扰。这正是高维键向量近似正交时的保护效果。

时刻 $t = 3$（查询针）：$q_3 = e_1$

$$\hat{v}_3^{\mathcal{M}} = \Phi_2\, e_1 \approx 2 \checkmark$$

针成功检索。

关键观察：即使 $s_2 \approx 0.51$（噪声的惊讶度不为零），只要键向量的高维正交性成立，噪声对”针”方向的干扰仍然有限。参见第 4.6 章的干扰界分析。

9.6 与前后章的关系

- **来自第 八 章**：TTT-Layer 证明了”隐状态是可学习参数而非静态向量”的可行性。本章的长时记忆 $\mathcal{M}$ 直接继承这一思路，并通过 Surprise 门控解决 TTT-Layer 的灾难性遗忘问题。

- **来自第 四 章**：Surprise 门控 s_t 在功能上与遗忘因子 λ 互补：λ 是时间维度的全局衰减，s_t 是内容维度的选择性门控。第 四 章的干扰界分析直接适用于本章 Worked Example 的正交键假设。

- **通向第 十 章**：方程 (9.1) 是标准在线回归损失。第 十 章将说明，Titans 的长时记忆在温和条件下是一种带选择性权重的在线岭回归问题。

- **通向第 十一 章**：Titans 展示了在一个前向传播中同时运行三个时间尺度的记

忆组件：这是”多层嵌套优化”思想的架构实例，在 Nested Learning 中将被形式化为三级优化层次。

9.7 代表论文

Behrouz et al. (2025c) —Titans: Learning to Memorize at Test Time 提出三类记忆并联架构与 Surprise 门控机制，在长达 2×10^6 tokens 的 Needle-in-a-Haystack 基准上验证了长时记忆模块在极长上下文下的检索性能，并通过消融实验分别了解各模块的贡献。

9.8 神经科学直觉与类比边界

神经科学直觉：互补学习系统（CLS）理论

类比对象：我们把 **互补学习系统（Complementary Learning Systems, CLS）理论**（McClelland, McNaughton & O'Reilly, 1995；Kumaran et al., 2016）视为帮助理解 Titans 三类记忆架构的一个类比。
CLS 理论的核心主张是：哺乳动物大脑需要两种互补的记忆系统来避免灾难性遗忘——

- **海马体（Hippocampus）**：用稀疏编码快速存储情景记忆（episodic memory），可以在单次呈现后记住一个新事件（one-shot learning）。容量有限，存储的是具体的情景。
- **新皮质（Neocortex）**：通过大量重复和缓慢的突触修改，整合统计规律，构建泛化的语义记忆（semantic memory）。容量大，但更新慢，抵抗单次输入的干扰。

共同之处（计算角色上的相似）：Titans 的设计与 CLS 的分工在**算法组织层面**有对应：

CLS 理论	Titans 架构
海马体：快速写入，高可塑性，情景特定	长时记忆模块 $\mathcal{M}$：Surprise 门控高频写入
新皮质：缓慢整合，低可塑性，泛化	持久记忆（Persistent Memory）：预训练固化，推断不更新
工作记忆：维持当前任务相关信息	短时记忆（STM / Window Attention）：滑动窗口

这个类比帮助理解什么：为什么架构需要分离三个时间尺度？CLS 理论提供了一个来自认知神经科学的设计动机：*不同保质期的信息不能由同一个机制来处理*，混用会导致新学的东西覆盖旧学的东西。这给 Titans 的多模块设计提供了直觉背书。

这个类比在哪里失效

一、更新规则根本不同（实现机制层）

Titans 的 Surprise 门控	一个精确的可微标量函数：$s_t = \sigma(\gamma\|\hat{v}_t - v_t\|_2 - \beta)$，通过标准反向传播端到端训练，$\gamma, \beta$ 可学习。
生物的新异刺激驱动强化	依赖多巴胺奖励预测误差（多脑区环路）、去甲肾上腺素（新异信号），通过局部突触标签（Synaptic Tagging）和生化级联实现，无全局目标函数，无反向传播。

二、海马体的功能比"快速写入器"复杂得多（时间与机制层）

CLS 理论中海马体的功能包括：情景记忆的**编码**、基于关系的**索引**、睡眠期的**重放（Replay）**以引导皮层巩固。Titans 的长时记忆模块 $\mathcal{M}$ 只是一个更新频率受门控的参数矩阵，它没有稀疏编码、索引化、或任何类似睡眠重放的机制。将 $\mathcal{M}$ 直接对等于海马体会掩盖这些关键功能差异。

三、皮层巩固的时间尺度与模型完全不同（时间尺度层）

生物的系统巩固（Systems Consolidation）历经数天至数年，涉及跨脑区的物理迁移和离线重放。Titans 的持久记忆（Persistent Memory）在推断时静态固定，在预训练阶段通过标准梯度下降学习；两者在时间尺度和信息迁移机制上没有真实对应。

读者不应从这个类比推出：

- **[X]** "Titans 实现了大脑的 CLS 机制"
- **[X]** "长时记忆模块 $\mathcal{M}$ 等同于海马体"
- **[X]** "Surprise 门控和多巴胺 RPE 是同一机制"
- **[X]** "生物系统在解同一优化问题"

类比成立的层次：*算法组织上的相似*——两者都有快/慢两类子系统分工处理不同保质期的信息，快的容易遗忘，慢的稳定。**类比失效的层次**：实现机制、学习信号、时间尺度均不同。

9.9 Lab 9：手写 Surprise 门控并观察遗忘-保留权衡

Lab 9：Surprise 门控对长序列遗忘的缓解

目标：通过代码实验，直接观察”有无 Surprise 门控”对灾难性遗忘的影响。
数据构造：

1. 生成序列长度 $T = 5000$，特征维度 $d = 64$。
2. 在 $t = 10$ 处写入一个高价值对：k_{needle} = 随机单位向量，$v_{\text{needle}} = 1.0$（标量）。
3. 对 $t = 11$ 到 T，生成服从 $\mathcal{N}(0, 0.1)$ 的随机键 k_t 和对应值 $v_t \sim \mathcal{N}(0, 0.1)$。

实验 A（无门控）：用标准梯度步更新线性模型 Φ，学习率 $\eta = 0.01$。在每 100 步检查 $\Phi \cdot k_{\text{needle}}$ 与 v_{needle} 的余弦相似度。
实验 B（Surprise 门控）：在梯度步前乘以 $s_t = \sigma(20 \cdot |v_{\text{pred}} - v_t| - 2)$，其余设置相同。
方法对比表（预期结果）：

	$t = 100$	$t = 1000$	$t = 5000$
无门控（A）	> 0.9	≈ 0.4	≈ 0.1
有门控（B）	> 0.9	> 0.8	> 0.7

思考题：如果噪声值 v_t 的方差与针值 v_{needle} 相近，门控的效果会减弱还是增强？为什么？

9.10 常见误解

常见误解

误解 1：只要加了 Surprise 门控，长时记忆就永远不会遗忘重要信息。
纠正：Surprise 门控缓解的是*平凡噪声对记忆的稀释*。如果序列中出现了大量高惊讶度的”真实”新信息，门控会持续开启，累积的写入同样可能覆盖早期写入的内容。此外，长时记忆的容量受参数矩阵的秩限制（与第 四 章的干扰界同原理）。Surprise 门控是缓解手段，不是完全解决方案。

常见误解

误解 2：加入了神经科学类比，就说明这个架构在模拟大脑机制。

纠正：CLS 理论在本章中是**设计直觉的来源**，不是架构正确性的证明。Titans 的性能保证来自数学推导和实验结果，与生物机制无关。如果实验证明某个不受 CLS 理论启发的架构性能更好，该架构应当被选择——生物类比不是评判标准。

常见误解

误解 3：s_t 等于零时长时记忆完全”冻结”，$\Phi_t = \Phi_{t-1}$。
纠正：由于动量项 $\mu\Delta\Phi_{t-1}$ 的存在，即使 $s_t = 0$，参数依然受历史动量影响而略有变化。若设 $\mu = 0$，则在 $s_t = 0$ 时参数确实不变——但这是一个超参数选择，不是 Surprise 机制本身的必然行为。

9.11 练习题

练习 9.1. Surprise 的敏感性分析：设 $\gamma = 5$，$\beta = 2$，σ 为标准 sigmoid。计算以下情况下的 s_t：(a) $\|\hat{v}_t - v_t\|_2 = 0$，(b) $\|\hat{v}_t - v_t\|_2 = 0.5$，(c) $\|\hat{v}_t - v_t\|_2 = 1.0$。根据你的计算，说明 γ 和 β 分别控制门控行为的哪个方面。

练习 9.2. 输出融合的权衡：假设三个记忆模块的输出分别为 $y_{\text{STM}} = 0.9$，$y_{\mathcal{M}} = 0.1$，$y_{\text{Persistent}} = 0.5$，且 $\alpha_1 = 0.5, \alpha_2 = 0.3, \alpha_3 = 0.2$（已归一化）。(a) 计算最终输出 y_t。(b) 如果任务要求精确恢复序列开头出现的一个特殊值（即典型的”针”任务），α_2 应该更高还是更低？请给出直觉理由。

练习 9.3. 与 TTT-Layer 的对比：TTT-Layer（第八章）的参数更新规则为 $\boldsymbol{S}_t = \boldsymbol{S}_{t-1} - \eta\nabla_{\boldsymbol{S}}\ell_t$，Titans 的更新规则为公式 (9.4)。(a) 写出两者更新规则的差异（数学形式）。(b) 若输入序列中 99% 为平凡噪声，1% 为高价值信息，当 s_t 设置为恰当的阈值后，Titans 对长时记忆的有效写入次数相比 TTT-Layer 减少多少（用百分比估计）？(c) 这个减少对计算开销有何影响？

练习 9.4. CLS 类比的边界检验：CLS 理论中，海马体通过睡眠期重放将信息迁移到新皮质。(a) Titans 架构中有没有对应”睡眠期重放”的机制？(b) 如果要在 Titans 中设计一个类似”离线巩固”步骤，形式上应该是什么操作？(c) 这个操作在工程上的代价是什么？

9.12 本章小结

- **核心问题**：无差别在线更新导致灾难性遗忘；解决方向是选择性写入——只在输入”出乎意料”时才大幅更新长时记忆。
- **Surprise 门控**：$s_t = \sigma(\gamma\|\hat{v}_t - v_t\|_2 - \beta)$，是一个可微的误差幅度归一化函数，在端到端训练中与主网络一同优化。

- **三类记忆并联**：短时（窗口注意力）+ 长时（Surprise 门控更新的 MLP）+ 持久（预训练固化），各司其职，通过可学习权重融合输出。
- **CLS 类比**：互补学习系统理论提供了" 为什么需要快/慢双系统" 的设计直觉，相似性在**算法组织层面**成立；但更新规则、学习信号、时间尺度均与生物系统根本不同。
- **通向下一章**：Titans 是" 多时间尺度嵌套优化" 的架构实例。第 十 章将从在线回归的角度统一解释所有这些机制，第 十一 章将给出形式化的理论框架。

读完本章，你应该能够：

- 推导 Surprise 门控更新规则，并解释公式中每一项的作用
- 解释为什么 Surprise 门控能缓解灾难性遗忘，以及它的局限性
- 将 Titans 的三类记忆与 CLS 理论进行对照，并**显式说明对照在哪个层面成立，在哪个层面失效**
- 设计 Lab 9 的数值实验，预测结果并解释差异来源

第十章　统一视角：Test-Time Regression 与联想记忆

> "All these models are solving the same optimization problem. They just don't know it yet."
>
> — 一种统一的解读

10.1　本章想回答什么问题

我们在这本书中走过了一段漫长的旅程：

- 第 四 章：快权重用外积 $\boldsymbol{S}_t = \boldsymbol{S}_{t-1} + v_t k_t^\top$ 写入联想记忆。
- 第 五 章：线性注意力的递推公式与快权重严格等价。
- 第 六 章：ICL 可以被理解为隐式梯度下降。
- 第 八 章：TTT-Layer 把隐状态变成通过梯度下降更新的学习器。
- 第 九 章：Titans 用 Surprise 门控选择性写入神经记忆。

现在的问题是：**这些模型真的互不相干吗？**

本章将展示：**它们都是在近似求解同一个优化问题——Test-Time Regression（TTR，测试时回归）**。区别仅在于使用的优化算法、截断策略和正则化方式。

理解了这个统一视角，你就不再是"学习了十种新架构"，而是"掌握了一个能生成所有变体的方程族"。

10.2 最小例子：在线联想回忆的统一性

玩具问题 F：从同一个目标函数推导出不同模型

设一个序列模型依次观测 $(k_1, v_1), (k_2, v_2), \ldots, (k_t, v_t)$，目标是维护一个状态矩阵 $\boldsymbol{S}_t$，使得对任意查询 q，输出 $\hat{v} = \boldsymbol{S}_t q$ 能尽可能准确地回忆出对应的值。定义**带遗忘的岭回归**目标：

$$\mathcal{L}_t(\boldsymbol{S}) = \frac{1}{2}\sum_{i=1}^{t} \lambda^{t-i} \|\boldsymbol{S}k_i - v_i\|_2^2 + \frac{\gamma}{2}\|\boldsymbol{S}\|_F^2 \tag{10.1}$$

现在观察以下三种 "求解策略" 的结果：

1. **一步 SGD，忽略预测反馈**（$\boldsymbol{S}_{t-1}k_t \approx 0$）：$\boldsymbol{S}_t = \boldsymbol{S}_{t-1} + v_t k_t^\top \Rightarrow$ **快权重 / 线性注意力**。
2. **一步 SGD，保留预测反馈**：$\boldsymbol{S}_t = \boldsymbol{S}_{t-1} - \eta(\boldsymbol{S}_{t-1}k_t - v_t)k_t^\top \Rightarrow$ **TTT-Linear**。
3. **一步 SGD + Surprise 门控**（仅当 $\|\boldsymbol{S}_{t-1}k_t - v_t\|$ 大时更新）：$\Rightarrow$ **Titans 的长时记忆**。

结论：三个看似不同的 "模型架构"，其实都是对**同一个回归目标**的不同近似求解！

10.3 直觉解释：优化的物理实体化

理解这个统一视角需要一个范式转换：

> **不要把序列模型理解为 "一种层结构"，而要理解为 "一个未被充分跑完的优化算法在硬件上的截断展开"。**

具体来说：

1. 我们面临的核心问题是**在线联想回忆**：用有限的内存（$d \times d$ 的状态矩阵）存储不断到来的键-值对，并在查询时高效检索。
2. 这天然是一个**在线优化问题**：每来一个新数据点，无法回顾所有历史（内存不够），只能 "就着当前状态" 做修补。
3. 不同的 "修补策略"（优化算法的选择）导致了不同的 "层结构"——外积写入、梯度步、门控写入……

所以我们其实不是在 "设计层"，而是在**选择一个在线优化器的单步更新公式**，然后把它包装成一个 "网络层"。

这个观察带来了方法论上的根本转变。在传统的深度学习研究中，序列模型的设计往往依赖于直觉和经验式的架构搜索——研究者先提出一种新的层结构，然后在实验中验证其效果。而在 TTR 框架下，设计过程变成了：**先写下一个明确的回归目标函数，然后根据计算预算和硬件约束选择一种近似求解策略**。不同的求解策略自然导出不同的层更新公式，其性能和效率的权衡可以在优化理论的框架内被分析和预测。这意味着未来的序列模型创新可能不再来自于“灵机一动想出新结构”，而是来自于“对目标函数或求解器的系统性改进”。

10.4 数学形式化：Test-Time Regression

10.4.1 基石定义

定义 10.1 (Test-Time Regression (TTR))**.** 设隐态参数为 $\boldsymbol{S} \in \mathbb{R}^{d_v \times d_k}$。在时间步 t，输入 token x_t 通过慢参数（投影矩阵）转换为 $k_t = W_K x_t$ 和 $v_t = W_V x_t$。

理想状态 $\boldsymbol{S}_t^*$ 应最小化以下加权岭回归：

$$\mathcal{L}_t(\boldsymbol{S}) = \frac{1}{2}\sum_{i=1}^{t} \alpha_i \|\boldsymbol{S}k_i - v_i\|_2^2 + \frac{\gamma}{2}\|\boldsymbol{S}\|_F^2 \tag{10.2}$$

其中 $\alpha_i \geq 0$ 是每个时间点的权重（可用于实现遗忘 $\alpha_i = \lambda^{t-i}$），γ 是 L2 正则系数。

10.4.2 闭式解：Recursive Least Squares (RLS)

目标函数 (10.2) 是关于 $\boldsymbol{S}$ 的二次函数，存在闭式解：

$$\boldsymbol{S}_t^* = V_t K_t^\top \left(K_t K_t^\top + \gamma I\right)^{-1} \tag{10.3}$$

其中 $K_t = [\sqrt{\alpha_1}k_1, \ldots, \sqrt{\alpha_t}k_t] \in \mathbb{R}^{d_k \times t}$，$V_t$ 类似定义。

这个闭式解可以通过 **Sherman–Morrison 公式**递推计算：

$$P_t = P_{t-1} - \frac{P_{t-1}k_t k_t^\top P_{t-1}}{1 + k_t^\top P_{t-1} k_t} \tag{10.4}$$

$$\boldsymbol{S}_t = \boldsymbol{S}_{t-1} + (v_t - \boldsymbol{S}_{t-1}k_t)k_t^\top P_t \tag{10.5}$$

其中 $P_t \in \mathbb{R}^{d_k \times d_k}$ 是精度矩阵的逆（初始化 $P_0 = \gamma^{-1}I$）。

注. RLS 在每步需要 $O(d_k^2)$ 的矩阵运算来维护 P_t。这虽然比重新计算逆矩阵（$O(d_k^3)$）高效，但在 d_k 很大时仍然不实用。正是这个成本驱动了我们转向更粗糙但更便宜的近似。

10.4.3 一阶近似：在线 SGD

不求精确解，而是对每个新数据点 (k_t, v_t) 只做一步 SGD：

$$\nabla_{\boldsymbol{S}} \ell_t = (\boldsymbol{S}_{t-1} k_t - v_t) k_t^\top \tag{10.6}$$

$$\boldsymbol{S}_t = \boldsymbol{S}_{t-1} - \eta (\boldsymbol{S}_{t-1} k_t - v_t) k_t^\top \tag{10.7}$$

这就是 TTT-Linear 的状态更新公式（第 八 章式 (8.2)）。

10.4.4 极端截断：忽略预测反馈

如果进一步假设 $\boldsymbol{S}_{t-1} k_t \approx 0$ （当前状态对新键的预测约为零——在 $\boldsymbol{S}$ 初始化为零且键近似正交时这是合理的），则梯度退化为：

$$\nabla_{\boldsymbol{S}} \ell_t \approx -v_t k_t^\top \tag{10.8}$$

代入更新公式（取 $\eta = 1$）：

$$\boldsymbol{S}_t = \boldsymbol{S}_{t-1} + v_t k_t^\top \tag{10.9}$$

这就是快权重 / 线性注意力的递推公式（第 五 章式 (5.13)）。

线性注意力是在线回归的 “最粗暴” 近似

线性注意力不是一种 “独立发明的机制”，而是对在线岭回归目标的一步 SGD 求解中，进一步忽略了预测反馈项 $\boldsymbol{S}_{t-1} k_t$ 后得到的极端截断。它之所以有时性能不足，正是因为缺少了这个闭环纠偏项。TTT-Layer 通过保留这个项来获得更好的记忆质量。

理解这个退化关系的意义在于：它揭示了线性注意力的**根本性局限**不是 “设计得不够精巧”，而是来自一个明确的数学近似——忽略当前状态对新键的预测。当输入序列的键向量彼此近似正交时（例如在信息检索任务中），这个近似是合理的，线性注意力能表现良好。但当键向量高度相关时（例如自然语言中相邻 token 的语义重叠），忽略预测反馈会导致严重的记忆干扰。这解释了为什么线性注意力在长文本建模中常常不如 TTT-Layer 或标准 Softmax 注意力。

这条从 “精确闭式解” 到 “极端截断近似” 的连续谱，为序列模型的设计提供了一个清晰的地图：研究者可以根据目标场景（长文本理解、信息检索、对话生成）和硬件约束（内存限制、延迟要求、训练预算），在这条谱上选择最合适的近似层级。现有的模型不是一个个孤立的发明，而是同一个优化地图上不同坐标点处的采样。

10.5 统一表格：所有模型作为 TTR 特例

模型	求解策略	保留反馈?	额外机制
快权重	1 步 SGD	否	无
线性注意力	1 步 SGD	否	= 快权重
RetNet	1 步 SGD	否	衰减 λ^{t-i}
TTT-Linear	1 步 SGD	**是**	无
TTT-MLP	1 步 SGD	**是**	非线性内层模型
Titans LTM	1 步 SGD	**是**	Surprise 门控
RLS (假想)	精确闭式解	**是**	P_t 精度矩阵

每一行的区别仅在于：对**同一个** TTR 目标函数 (10.2) 选择了不同的（a）优化算法、（b）截断策略、（c）附加门控。

推导谱：从精确解到极端截断

以下展示了从同一个 TTR 目标函数出发，不同的求解策略如何生成不同的模型。这不是“分类”（每个模型有各自的来源），而是“推导谱”（所有模型从同一个方程分叉）。

1. **精确闭式解**（无截断）：$\boldsymbol{S}_t^* = V_{1:t}K_{1:t}^\top(K_{1:t}K_{1:t}^\top + \gamma I)^{-1} \Rightarrow$ **在线最小二乘 / RLS**
   - 优点：最优回忆精度
   - 代价：需要维护 $d \times d$ 的精度矩阵 P_t 并用 Sherman-Morrison 增量更新。计算量 $O(d^2)$/步，但常数大。
2. **一步 SGD，保留预测反馈**：$\boldsymbol{S}_t = \boldsymbol{S}_{t-1} - \eta(\boldsymbol{S}_{t-1}k_t - v_t)k_t^\top \Rightarrow$ **TTT-Linear**
   - 近似程度：忽略了历史样本的贡献，只用当前样本更新
   - 优点：硬件友好，$O(d^2)$/步且不需要维护 P_t
3. **一步 SGD，忽略预测反馈**（令 $\boldsymbol{S}_{t-1}k_t \approx 0$）：$\boldsymbol{S}_t = \boldsymbol{S}_{t-1} + v_t k_t^\top \Rightarrow$ **快权重 / 线性注意力**
   - 近似程度：进一步丢弃了当前状态对预测的反馈
   - 优点：实现最简单，并行效率最高
   - 代价：无法抑制旧记忆对新查询的干扰
4. **一步 SGD + 门控**（仅在 Surprise 高时大步更新）：$\Rightarrow$ **Titans 长时记忆**
   - 近似程度：与 (2) 相同的 SGD，但乘以 Surprise 门控

- 优点：保护已存信息不被平凡噪声覆盖

5. **多步 SGD / 非线性内层模型：⇒ TTT-MLP**

- 近似程度：用更多的计算换取更高的近似精度
- 代价：内层反向传播使每步计算量显著增加

扩展主表：假设、增益与代价

模型	核心假设	增益	代价	目标场景
快权重 / 线性注意力	$\boldsymbol{S}_{t-1}k_t \approx 0$（键近似正交）	实现最简单，高并行	键相关时记忆干扰	信息检索，短序列
TTT-Linear	当前梯度近似精确方向	更准确的记忆更新	每步 $O(d^2)$ 且不可并行	长序列，分布偏移
Titans LTM	Surprise 可以识别重要信息	保护已存关键信息	需要调参门控阈值	超长序列，needle-in-haystack
TTT-MLP	内层 MLP 的表达能力足够	非线性记忆容量	内层反传代价高	复杂任务，高记忆需求
RLS	二阶信息值得维护	最优回忆精度	需要 $d \times d$ 精度矩阵	理论参考/小 d 场景

10.6 Worked Example：从 TTR 推导 TTT-Linear

例子 10.2 (从 TTR 推导 TTT-Linear 的更新规则)**. 设定**：$d_k = d_v = 3$，$\gamma = 0$（无正则），$\eta = 0.1$。

时间步 $t = 1$：$k_1 = (1, 0, 0)^\top$，$v_1 = (3, 1, 2)^\top$，$\boldsymbol{S}_0 = 0_{3\times 3}$。

TTR-SGD 更新：

$$\text{预测}: \boldsymbol{S}_0 k_1 = 0 \tag{10.10}$$

$$\text{残差}: v_1 - \boldsymbol{S}_0 k_1 = (3,1,2)^\top \tag{10.11}$$

$$\text{梯度}: -(v_1 - \boldsymbol{S}_0 k_1) k_1^\top = -(3,1,2)^\top (1,0,0) = \begin{pmatrix} -3 & 0 & 0 \\ -1 & 0 & 0 \\ -2 & 0 & 0 \end{pmatrix} \tag{10.12}$$

$$\boldsymbol{S}_1 = \boldsymbol{S}_0 - 0.1 \times \text{梯度} = \begin{pmatrix} 0.3 & 0 & 0 \\ 0.1 & 0 & 0 \\ 0.2 & 0 & 0 \end{pmatrix} \tag{10.13}$$

时间步 $t = 2$：$k_2 = (0,1,0)^\top$，$v_2 = (0,5,0)^\top$。

TTR-SGD 更新（注意保留了预测反馈！）：

$$\text{预测}: \boldsymbol{S}_1 k_2 = (0,0,0)^\top \quad (\text{因为} k_2 \text{ 的第 1 分量为 0}) \tag{10.14}$$

$$\text{残差}: v_2 - \boldsymbol{S}_1 k_2 = (0,5,0)^\top \tag{10.15}$$

$$\boldsymbol{S}_2 = \boldsymbol{S}_1 + 0.1 \times (0,5,0)^\top (0,1,0) = \begin{pmatrix} 0.3 & 0 & 0 \\ 0.1 & 0.5 & 0 \\ 0.2 & 0 & 0 \end{pmatrix} \tag{10.16}$$

查询：$q = k_1 = (1,0,0)^\top$：$\hat{v} = \boldsymbol{S}_2 q = (0.3, 0.1, 0.2)^\top = 0.1 \times v_1$。回忆不完美（因为 $\eta = 0.1 < 1$），但方向正确。

对比线性注意力（忽略预测反馈，$\eta = 1$）：$\boldsymbol{S}_2^{\text{LA}} = v_1 k_1^\top + v_2 k_2^\top$，查询 $q = k_1$ 得 $\hat{v} = v_1$——精确回忆（因为 $k_1 \perp k_2$）。

对比 TTT-Linear（保留预测反馈，$\eta = 1$）：$\boldsymbol{S}_2^{\text{TTT}}$ 的计算与上述相同但 $\eta = 1$，也得到精确回忆。

结论：当键正交时，TTT-Linear 和线性注意力给出相同结果。它们的差异在键**非正交**时体现——TTT-Linear 的预测反馈项可以抑制干扰。

10.7 神经科学直觉与类比边界

神经科学直觉：Hebbian 学习与误差驱动学习的统一

类比对象：我们可以用 **Hebbian 学习**和**误差驱动学习（Error-Driven Learning）**这两种神经科学范式来帮助理解 TTR 框架中的两端特例。

在神经科学中，突触可塑性有两个经典模型：

- **Hebb 规则**："共同激活的神经元加强连接"——$\Delta w \propto v \cdot k$（外积形式）。这是一种**无监督的**、**正反馈的**学习规则。

- **Delta 规则 / LMS**：$\Delta w \propto (v - w^\top k) \cdot k$（预测误差形式）。这是一种**有闭环纠偏的**学习规则。

共同之处（算法层面）：
TTR 框架中，快权重/线性注意力的更新 $\boldsymbol{S}_t = \boldsymbol{S}_{t-1} + v_t k_t^\top$ 在形式上等同于 Hebb 规则（外积写入）。而 TTT-Linear 的更新 $\boldsymbol{S}_t = \boldsymbol{S}_{t-1} - \eta(\boldsymbol{S}_{t-1} k_t - v_t) k_t^\top$ 在形式上等同于 Delta 规则（误差驱动写入）。
TTR 框架把这两者统一到一个连续谱上：它们是同一个目标函数在不同截断程度下的产物。

这个类比在哪里失效

一、学习信号的来源不同（信息流层）

TTR 框架	(k_t, v_t) 由全局慢参数 W_K, W_V 从输入 x_t 投影得到。学习目标是全局端到端训练的结果，已经包含了对 “什么信息值得记住” 的高层判断。
Hebb / Delta 规则	学习信号是**局部的**——仅依赖突触前后的活动，没有全局目标函数。神经元不知道自己的学习是否有助于 “整体任务”。

二、“闭环” 的含义不同（实现层）
TTT-Linear 的 “预测反馈” $\boldsymbol{S}_{t-1} k_t$ 是通过矩阵乘法精确计算的。生物的误差信号（如小脑的攀缘纤维信号）是通过专门的神经回路传递的，具有延迟、噪声和容量限制。

读者不应从这个类比推出：

- **[X]** “线性注意力就是 Hebbian 学习”
- **[X]** “TTT-Linear 实现了大脑的误差驱动学习”
- **[X]** “TTR 框架解释了大脑如何学习”

类比成立的层次：数学形式——外积 vs. 误差修正的二元性在两个系统中都存在。
类比失效的层次：信号来源（全局 vs. 局部）、误差传递机制、学习的目的性均不同。

10.8 这个统一视角的边界

TTR 作为统一框架的价值是真实的，但它的适用范围有明确的边界。本节区分框架的 “威力范围” 和 “过度推广区域”。

TTR 框架在哪里有力

1. **作为分析语言**：给定一个新的序列模型，你可以问：“它对应 TTR 目标函数中的哪种求解策略？它做了什么近似？放弃了什么？” 这种问题的结构化能力是真正有价值的。

2. **作为设计原理**：如果你想设计一个新的序列模型层，不再需要从零开始 “搭积木”。你可以从 TTR 目标函数出发，修改正则化项、优化器、或截断策略，然后推导出更新规则。这把 “灵感驱动的架构搜索” 转化为 “在优化空间中的系统探索”。

3. **作为比较工具**：扩展主表使得不同模型的假设和代价可以在同一个坐标系中被并列比较。

TTR 框架不应被过度推广之处

1. **Softmax 注意力不在框架内**：标准的 Softmax 注意力由于其非线性归一化，不能被简单地归入 “TTR 的一种近似求解”。TTR 框架主要覆盖的是**线性**或**可线性化**的状态更新。

2. **离散操作超出范围**：Token embedding、argmax 解码、位置编码等离散操作没有对应的在线优化解读。

3. **“统一 $\neq$ 解释”**：TTR 说明了多种架构可以被放入同一个数学框架中，但它没有解释为什么某些架构在实践中比其他架构表现更好。性能差异来自近似质量和硬件适配，而不是来自目标函数本身。

4. **预训练不在框架内**：TTR 统一的是**推断时**的状态更新规则。“外层参数 θ 如何被预训练” 是一个独立的问题，TTR 框架对此没有发言权。

10.9 代表论文精读

Xiong et al. (2025) — Test-Time Regression: A Unifying Framework 这篇论文将过去几年的十多种序列模型架构通过在线凸优化的语言归约为同一个目标函数的不同近似。核心贡献：证明了通过改变 TTR 目标函数的正则化项、求解策略和门控参数，可以系统地推导出 RetNet、Mamba、TTT-Linear 等多种 “看似独立” 的模型层。这是序列模型设计从 “搭积木试错” 转向 “从目标函数推导” 的分水岭。

Behrouz et al. (2025a) — It's All Connected 从不同角度建立了类似的统一。文中证明了注意力偏置、序列提取器、在线内存优化之间的形式联系，进一步印证了 TTR 视角的普遍性。

10.10 和前后章节的关系

- **对第 四–九 章的综合**：本章是前面所有模型章节的统一解释。之前按历史顺序介绍的机制，在本章被归约为同一个目标函数下的不同求解策略。

- **通向第 十一 章（Nested Learning）**：TTR 统一了单层的状态更新规则。但如果多个层叠在一起，每层都在做 TTR，层间的参数传导如何组织？这正是 Nested Learning 要回答的问题——把 TTR 推广到多层嵌套的形式。

10.11 实验

Lab 10：从 TTR 目标函数推导并实验对比三种层

目标：不参考任何具体论文的架构名称，直接通过修改目标函数的求解策略推导出三种不同的状态更新层，并用在线回归实验对比它们的性能。

1. **数据**：生成 $T = 500$ 步的在线回归序列，$d = 16$，键非正交（随机高斯），每 100 步切换一次目标函数参数。
2. **方法 A**（无反馈）：$\boldsymbol{S}_t = \boldsymbol{S}_{t-1} + v_t k_t^\top$。记录累积误差。
3. **方法 B**（有反馈）：$\boldsymbol{S}_t = \boldsymbol{S}_{t-1} - \eta(\boldsymbol{S}_{t-1} k_t - v_t) k_t^\top$，$\eta = 0.01$。记录累积误差。
4. **方法 C**（有反馈 + 衰减）：$\boldsymbol{S}_t = \lambda \boldsymbol{S}_{t-1} - \eta(\boldsymbol{S}_{t-1} k_t - v_t) k_t^\top$，$\lambda = 0.99$，$\eta = 0.01$。记录累积误差。
5. **分析**：绘制 "时间步 vs. 累积 MSE" 曲线。在任务切换点附近，哪种方法恢复最快？

预期观察：方法 B 优于方法 A（闭环纠偏）；方法 C 在任务切换后恢复最快（衰减帮助遗忘旧信息）。

10.12 常见误解

常见误解

误解 1：既然所有模型都在解同一个优化问题，那么它们的实际性能应该差不多。

纠正：理论上同源，但“极其微小”的近似差异会导致实际性能和硬件效率的巨大差异。例如：线性注意力（无反馈）在 GPU 上吞吐量最高，但记忆质量差；TTT-MLP（有反馈 + 非线性）记忆质量好，但每步需要对内层 MLP 做反向传播，吞吐量显著下降。选择哪个“近似”取决于具体的硬件约束和任务需求。

常见误解

误解 2：TTR 框架完全解释了深度学习的一切。

纠正：TTR 在**机制层面**提供了一种“优化算子”的统一语言。但“统一了底层运算的数学形式”不等于“解释了复杂行为的涌现”。大模型的推理能力、多模态对齐、对抗鲁棒性等高层现象，至今没有被任何单一框架充分解释。TTR 是理解机制的有力工具，不是万能理论。

10.13 练习题

练习 10.1. 推导 RetNet：从 TTR 目标函数 (10.2) 出发，设 $\alpha_i = \lambda^{t-i}$（指数衰减权重），$\gamma = 0$，优化策略为“一步 SGD，忽略预测反馈”。推导出 RetNet 的递推更新公式 $\boldsymbol{S}_t = \lambda \boldsymbol{S}_{t-1} + v_t k_t^\top$。说明衰减因子 λ 在目标函数中的含义。

练习 10.2. RLS 的递推推导：使用 Sherman–Morrison 恒等式 $(A + uv^\top)^{-1} = A^{-1} - \frac{A^{-1}uv^\top A^{-1}}{1+v^\top A^{-1}u}$，从 $\boldsymbol{S}^*_{t-1} = V_{t-1}K_{t-1}^\top(K_{t-1}K_{t-1}^\top + \gamma I)^{-1}$ 推导出 $\boldsymbol{S}^*_t$ 的递推更新公式 (10.4)–(10.5)。说明为什么 RLS 虽然是“精确解”，但在实践中不如一步 SGD 常用（提示：P_t 的维护代价）。

练习 10.3. Surprise 门控的 TTR 解读：将 Titans 中的 Surprise 值 $s_t = \|\boldsymbol{S}_{t-1}k_t - v_t\|_2^2$ 对应到 TTR 目标函数 (10.2) 中的哪一项？解释为什么高 Surprise 表示“当前状态对新数据的预测很差”，因此应该加大更新步长。

10.14 本章小结

- **TTR 统一视角**：快权重、线性注意力、TTT-Linear、TTT-MLP、Titans 的长时记忆都可以被理解为对同一个在线岭回归目标的不同近似求解。区别仅在于优化算法、截断策略和附加门控。

- **核心方程**：TTR 目标函数 (10.2) → 一步 SGD 得到通用更新 $\boldsymbol{S}_t = \boldsymbol{S}_{t-1} - \eta(\boldsymbol{S}_{t-1}k_t - v_t)k_t^\top$ → 忽略反馈退化为外积写入 $\boldsymbol{S}_t = \boldsymbol{S}_{t-1} + v_t k_t^\top$。
- **方法论转变**：序列模型设计从 "搭积木试错" 转向 "从目标函数推导新变体"。通过修改正则化项、求解策略和门控参数，可以系统地推导出新的模型层。
- **闭式解 vs. 近似**：RLS 给出精确闭式解但计算代价高；一步 SGD 便宜但不精确；实际系统在两端之间权衡。

读完本章，你应该能够：

- 写出 TTR 目标函数，并从中推导出快权重、TTT-Linear 的更新公式
- 解释线性注意力为什么是 "最粗暴的近似" ——因为它忽略了预测反馈项
- 对一个新提出的序列模型层，判断它在 TTR 框架中对应的优化策略
- 用 Sherman–Morrison 公式推导 RLS 的递推更新

第十一章　Nested Learning：当架构本身就是优化

> "There is no architecture. Everything is just learning to learn to learn. Deep learning architectures as we know them are an illusion."
>
> — Behrouz, Zhong & Mirrokni, 2025

11.1　本章想回答什么问题

我们已经走完了一段完整的推演：

- 第 三 章：双层优化把"训练"分解为内循环（快适应）和外循环（元学习）。
- 第 四–五 章：快权重联想记忆与线性注意力在数学上严格等价——它们都在维护一个键-值存储矩阵 $\boldsymbol{S}_t$。
- 第 六 章：Transformer 的前向传播可以等价于执行梯度下降。
- 第 八–九 章：TTT-Layer 和 Titans 把隐状态升级为一个可学习的微型网络。
- 第 十 章：Test-Time Regression 把上述所有机制统一到一个在线回归目标下。

现在的问题是：

> 如果每一个"层"都可以被重新解读为一次在线优化的截断近似，那么现代深度网络中那些堆叠了几十甚至上百层的"固定架构"，到底是什么东西？它们是我们有意设计出来的结构，还只是多层嵌套优化过程在硬件约束下的一种残影？

Behrouz et al. (2025b) 提出的 **Nested Learning（嵌套学习）**框架试图回答这个问题。本章将：

1. 给出 Nested Learning 的形式化定义（§11.4）

2. 展示它如何把前面各章的模型统一为特例（§11.5）

3. 严格区分哪些主张属于“已被验证的数学机制”，哪些属于“极具号召力但尚未完全验证的研究议程”（§11.6）

读完本章，你将能够：

- 写出 L 层嵌套优化的递推公式
- 把线性注意力、TTT-Layer、Titans 各归约为 Nested Learning 的特例
- 清晰区分“机制等价”和“嵌套解释”之间的科学地位差异

11.2 最小例子：三层嵌套的回归

玩具问题 G：三层嵌套的在线回归代理人

设想三个代理人 A（快）、B（中）、C（慢），各自维护一个参数矩阵：$\boldsymbol{S}_t^{(1)}$（A 的状态）、$\boldsymbol{S}_t^{(2)}$（B 的状态）、θ（C 的参数，在此简化为固定常数）。

A（**最内层，最快时间尺度**）：对每个 token x_t，A 执行一步在线梯度更新：

$$\boldsymbol{S}_t^{(1)} = \boldsymbol{S}_{t-1}^{(1)} - \eta_1 \nabla_{\boldsymbol{S}^{(1)}} \ell_1(\boldsymbol{S}_{t-1}^{(1)}, k_t, v_t)$$

这正是第 八 章 TTT-Layer 的更新规则。

B（**中间层，中等时间尺度**）：每隔 T_2 个 token，B 根据 A 在最近 T_2 步中的累积表现，调整 A 的学习率 η_1 和键值投影矩阵：

$$\boldsymbol{S}_t^{(2)} = \boldsymbol{S}_{t-T_2}^{(2)} - \eta_2 \nabla_{\boldsymbol{S}^{(2)}} \left[\sum_{i=t-T_2+1}^{t} \ell_2(\boldsymbol{S}_i^{(1)}) \right]$$

C（**最外层，最慢时间尺度**）：C 在整个训练集上学习 θ，使得 B 和 A 从 θ 初始化后能快速适应各种任务。这就是标准的预训练。

观察：从外向内看，这是三层嵌套的优化结构。每一层的“变量”是下一层的“超参数或初始化点”。三个代理人的计算内容在数学上完全相同——都是“给定目标函数，执行梯度步”——唯一的区别是它们操作的**时间尺度**不同（每 token、每 T_2 个 token、整个训练集）。

这个玩具问题揭示了 Nested Learning 的核心主张：*深度网络的“层级结构”可以被还原为多个时间尺度嵌套在一起的优化过程。*

11.3 直觉解释：为什么 “架构” 可能只是一种截断

考虑一个传统的 L 层前馈网络：

$$h_0 = x, \quad h_l = \sigma(W_l h_{l-1}), \quad y = h_L$$

在标准理解中，每一层 l 是一个**固定的参数化变换**：参数 W_l 在训练时通过反向传播学习，推断时完全固定。

Nested Learning 提出一种替代性解读：每一层 l 实际上可以被理解为对某个**局部目标函数**进行**一步优化**的截断近似。

具体来说，如果第 l 层的 “理想行为” 是最小化

$$\ell_l(\boldsymbol{S}) = \frac{1}{2}\|\boldsymbol{S} \cdot \phi(h_{l-1}) - v_l^*\|_2^2 \tag{11.1}$$

其中 v_l^* 是该层应当产出的 “理想表示”，那么从 $\boldsymbol{S} = 0$ 出发做一步梯度下降：

$$\boldsymbol{S} = 0 - \eta_l \nabla_{\boldsymbol{S}} \ell_l(0) = \eta_l \cdot v_l^* \cdot \phi(h_{l-1})^\top \tag{11.2}$$

如果把 $\eta_l \cdot v_l^*$ 吸收进一个可学习矩阵 W_l，这就退化为 $h_l = W_l \cdot \phi(h_{l-1})$——**恰好是标准的全连接层**。

深度层

标准的全连接层可以被视为一个 “从零出发、做一步梯度、然后截断” 的优化过程的物理实体化。“层数” 对应 “优化步数”，“架构设计” 对应 “截断策略的选择”。这不是在描述一种实现方式，而是在提供一种**理解框架**。

11.4 数学形式化：L 层嵌套优化

Step 1：递归定义嵌套层级

定义 11.1 (L 层 Nested Learning 框架)**.** 设一个深度为 L 的模型包含 L 层嵌套的在线优化过程。第 l 层（$l = 1, \ldots, L$）维护一个状态 $\boldsymbol{S}_t^{(l)}$，在时间步 t 按如下规则更新：

$$\boldsymbol{S}_t^{(l)} = \mathrm{Opt}^{(l)}\Big(\boldsymbol{S}_{t-1}^{(l)},\ \ell^{(l)}\Big(\boldsymbol{S}_{t-1}^{(l)},\ k_t^{(l)},\ v_t^{(l)}\Big)\Big) \tag{11.3}$$

其中：

- $\mathrm{Opt}^{(l)}$ 是第 l 层使用的优化算法（如 SGD、Adam 或外积写入）
- $\ell^{(l)}$ 是第 l 层的局部目标函数
- $(k_t^{(l)}, v_t^{(l)})$ 是由**更高层级**传递下来的键和值

Step 2：层间的参数传导

嵌套结构的关键是**层间传导**：第 l 层的输入（键、值、学习率）由第 $l+1$ 层的状态决定：

$$(k_t^{(l)},\ v_t^{(l)},\ \eta^{(l)}) = f^{(l)}(\boldsymbol{S}_t^{(l+1)},\ x_t) \tag{11.4}$$

其中 $f^{(l)}$ 是由更高层级的参数化映射。

最高层（第 L 层）的状态 $\boldsymbol{S}^{(L)}$ 对应于 “慢权重”——在标准训练中通过整个数据集上的反向传播学习，推断时固定。

Step 3：时间尺度的分离

每一层操作在不同的时间尺度 $T^{(l)}$ 上：

$$T^{(1)} \ll T^{(2)} \ll \cdots \ll T^{(L)} \tag{11.5}$$

- 第 1 层（最内，最快）：每个 token 更新一次，对应 TTT-Layer 的逐 token 梯度步
- 第 2 层：每 $T^{(2)}$ 个 token 更新一次，对应 “调整内层超参数”
- 第 L 层（最外，最慢）：在整个训练集上更新，对应标准预训练

Step 4：从嵌套到展平——理解标准网络

如果对每一层都执行**恰好一步**优化就截断输出，并把优化的初始点吸收进可学习的静态参数 W_l，那么：

$$\boldsymbol{S}_t^{(l)} \approx W_l + \eta_l \cdot \Delta_t^{(l)} \tag{11.6}$$

其中 $\Delta_t^{(l)}$ 是来自局部目标函数的一步更新量。

当进一步令 $\Delta_t^{(l)} = 0$（忽略动态更新项）时，$\boldsymbol{S}^{(l)}$ 退化为固定参数 W_l——这就是标准前馈网络的每一层。

注. 这并不意味着标准前馈网络 “不好”。恰恰相反，标准网络通过（a）吸收初始点到参数中、（b）截断为一步、（c）在预训练中内化所有动态调整，用**固定参数有效地模拟了**嵌套优化的行为。截断带来的好处是硬件友好性和推断效率；代价是失去了测试时动态适应的能力。

11.5 统一：把前面所有模型归约为 Nested Learning 的特例

本书前面介绍的所有模型都可以被放入 Nested Learning 框架中，区别仅在于嵌套深度、优化算法、截断策略和时间尺度的选取。

模型	嵌套深度	内层 $\mathrm{Opt}^{(1)}$	截断/近似策略
快权重（Ch. 四）	$L=2$	外积写入 $\boldsymbol{S}+kv^\top$	一步、忽略预测反馈
线性注意力（Ch. 五）	$L=2$	外积写入 $\boldsymbol{S}+kv^\top$	$\equiv$ 快权重
TTT-Linear（Ch. 八）	$L=2$	SGD with residual	$\boldsymbol{S}_{t-1}k_t$ 项保留
TTT-MLP（Ch. 八）	$L=2$	多层 MLP 的 SGD	非线性，容量更大
Titans（Ch. 九）	$L=2$（+STM）	Surprise 门控 SGD	选择性写入
标准 Transformer	L_{stack}	前向传播即一步截断	忽略 Δ_t

例子 11.2 (从 Nested Learning 推导线性注意力)**.** 设 $L=2$，最外层参数 θ（学习投影矩阵 W_K, W_V, W_Q），最内层维护状态 $\boldsymbol{S}_t^{(1)} \in \mathbb{R}^{d\times d}$。

局部目标函数：$\ell^{(1)}(\boldsymbol{S}, k_t, v_t) = \frac{1}{2}\|\boldsymbol{S}k_t - v_t\|_2^2$。

选择 $\mathrm{Opt}^{(1)}$ 为“从 $\boldsymbol{S}_{t-1}$ 出发做一步 SGD，但忽略预测反馈项 $\boldsymbol{S}_{t-1}k_t$”：

$$\nabla_{\boldsymbol{S}}\ell^{(1)}|_{\boldsymbol{S}=0} = (0\cdot k_t - v_t)k_t^\top = -v_t k_t^\top \tag{11.7}$$

$$\boldsymbol{S}_t = \boldsymbol{S}_{t-1} + \eta_1 \cdot v_t k_t^\top \tag{11.8}$$

令 $\eta_1 = 1$，得到 $\boldsymbol{S}_t = \boldsymbol{S}_{t-1} + v_t k_t^\top$——这正是线性注意力 / 快权重的递推公式（第 五 章式 (5.13)）。

输出 $y_t = q_t^\top \boldsymbol{S}_t$ 是标准的线性注意力读取。

结论：线性注意力是 Nested Learning 在 $L=2$、$\mathrm{Opt}^{(1)}$ = one-step SGD without prediction feedback 条件下的特例。

11.6 共识与解释：必须澄清的学术边界

这一节是全书最重要的学术界分节。Nested Learning 框架涉及两类性质截然不同的主张，必须严格区分。

属于 “机制层共识” 的主张

以下内容已被数学证明和/或广泛的实验验证：

1. **线性注意力 ≡ 快权重**（定理 5.2）：二者在数学上严格等价（不是近似）。
2. **线性注意力是在线回归的一步截断**（第 十 章）：公式对应关系已被明确推导。
3. **TTT-Layer 的隐状态更新是在线 SGD**（第 八 章）：构造性证明。
4. **Transformer 的 ICL 可以实现梯度下降**（第 六 章）：在线性回归任务上，构造性证明了一层线性注意力等价于一步梯度步。

属于 “研究议程” 的主张

以下内容是极具启发性的理论视角，但**尚未被充分证明为一般性结论**：

视角（非定论）

“架构是算法的幻像（Illusion）”——这是 Nested Learning 最核心的口号。它主张：深度网络的固定层级结构只是多层嵌套优化在硬件约束下的截断残影，不是产生智能行为的真正原因。
这个主张的科学地位：

- 在特定的线性/二次情形下，可以严格证明特定的网络层等价于在线优化的一步截断。
- 但在一般的非线性、有离散操作、有 Softmax 的架构中，这种等价是**近似的**，而非严格的。
- 它更适合被理解为一种**统一的分析语言**——一种组织思考的方式——而非一个已被证实的普遍定理。

请把 Nested Learning 当做一个 **“大统一场论” 级别的研究计划（Research Agenda）**，并在后续研究中尝试证实或证伪其各部分主张。不要当做 “已被应用证实一切” 的终极定论。

主张状态表

以下表格总结了 Nested Learning 框架中各主张的科学地位。读者应当把这张表作为阅读本章后的核心参考。

主张	证据类型	状态
线性注意力 ≡ 快权重	数学证明	**机制层共识**
TTT-Layer 更新 = 在线 SGD	构造性证明	**机制层共识**
ICL 可以实现 GD	在线性任务上构造性证明	**机制层共识**（限定条件下）
L 层前馈 = L 步截断优化	线性情形下的公式对应	**强 viewpoint**
“架构是优化的幻像”	部分形式化 + 概念论证	**研究议程**
L 层嵌套统一所有序列模型	部分特例验证	**研究议程**

11.7 Nested Learning 尚未证明什么

以下是 Nested Learning 框架**当前无法回答**的问题。列出这些限制不是为了否定框架的价值，而是为了帮助读者免于过度推广。

1. **训练收敛性**：Nested Learning 没有证明训练过的深度网络会收敛到 “嵌套优化的均衡点”。实际的预训练是否真的在寻找某种嵌套优化结构，还是在优化一个完全不同的目标，是一个开放的实验问题。

2. **涌现行为的解释**：深度网络展现出的推理能力、规划能力、多模态对齐等高层行为，目前没有被 Nested Learning 框架（或任何其他统一框架）所解释。“统一了底层运算的数学形式” 不等于 “解释了复杂行为的涌现”。

3. **非线性扩展**：当前的严格等价结果（线性注意力 = 快权重 = 在线回归一步截断）全部限于线性或二次情形。在包含 Softmax、LayerNorm、ReLU 的实际架构中，嵌套优化解读是**近似的**，精确的近似质量尚未被理论量化。

4. **工程可行性**：“放松架构截断”（允许更多层级的动态优化）在计算和数值稳定性上面临严重挑战。当前没有证据表明取消固定架构会带来性能提升。固定层数和标准前向传播是**工程上的**，不是 “审美上的” 选择。

11.8 为什么本章是一个综合，而不是一张许可证

Nested Learning 的最大价值不在于它 “解决了” 深度学习的所有问题，而在于它提供了一种**统一的分析语言**。

- **作为分析工具**：当遇到一个新的序列模型论文时，你可以问：“它对应 Nested Learning 中的哪个参数选择？它是在改进优化算法、还是在改变截断策略、还是在引入新的时间尺度？” 这种分析框架是有价值的，即使框架本身不构成对所有现象的解释。

- **作为设计原理**：第十章已经展示了如何从一个统一的目标函数出发，通过修改惩罚项和求解策略推导出新模型。Nested Learning 把这种方法论推广到了更多层级。

- **不应当用作**："既然架构是幻像，那我们可以无限嵌套"——这忽略了工程约束。"既然所有模型都是 NL 的特例，那 NL 框架解释了一切"——这混淆了"可以被统一描述"和"已被解释"。

11.9 代表论文

Behrouz et al. (2025b) — Nested Learning: The Illusion of Deep Learning Architectures 这篇论文是对 TTT 一线研究的理论收尾。它不提出一个新的跑分架构，而是用形式化的公式对位展示了：既然单个层可以被理解为在线学习的一步截断，那么所有的层级结构都可以被重写为带有不同步长和截断参数的嵌套优化方程。文章的贡献在于**统一语言**——为未来设计自适应架构提供了一套统一的数学工具。

11.10 神经科学直觉与类比边界

神经科学直觉：多时间尺度的神经可塑性层级

类比对象：我们把**大脑中多时间尺度的可塑性层级**视为帮助理解 Nested Learning 多层嵌套优化结构的一个类比。
在神经科学中，信息的存储和适应发生在多个时间尺度:

- **神经放电（Neural Firing）**：毫秒级，代表对当前刺激的即时响应。
- **短期突触可塑性（STP）**：秒到分钟级，突触传递效率的暂时性变化，支持工作记忆和上下文切换。
- **长期突触可塑性（LTP/LTD）**：小时到天级，突触权重的持久性改变，支持长期学习。
- **系统巩固（Systems Consolidation）**：天到年级，信息从海马体向新皮层的缓慢迁移，实现泛化的语义知识。

共同之处（算法组织层面）：
Nested Learning 的多层嵌套优化与上述生物机制在**算法组织**上相似：

神经可塑性层级	Nested Learning
神经放电（最快）	第 1 层：逐 token 的梯度更新
STP（快）	第 2 层：中等时间窗口的调参
LTP/LTD（中等）	中层：块级别的参数适应
系统巩固（最慢）	第 L 层：整个训练集的预训练

这个类比帮助理解什么：为什么需要多个嵌套的时间尺度？因为不同“保质期”的信息需要不同速度的处理通道——太快的通道记不住长期知识，太慢的通道来不及适应当前输入。大脑通过多层可塑性机制解决了这个矛盾，Nested Learning 通过多层优化的时间尺度分离做了类似的事情。

这个类比在哪里失效

一、更新规则根本不同（实现机制层）

Nested Learning	每一层的优化使用显式的损失函数和梯度计算，通过端到端的反向传播训练所有层级的参数。优化目标是全局的、可微的。
生物可塑性层级	各层级使用**局部的**学习规则：STP 依赖于突触前/后的活动相关性，LTP 依赖于 NMDA 受体和钙离子浓度，系统巩固依赖于离线重放（如睡眠）。没有全局目标函数，没有反向传播。

二、层级之间的耦合方式不同（信息流层）

在 Nested Learning 中，更高层级显式地通过可微函数向下传递参数（式 (11.4)），这是一种**自顶向下的参数传导**。在生物系统中，不同时间尺度的可塑性之间的耦合是通过**神经调质**（如多巴胺、去甲肾上腺素）和**离线重放**间接实现的，不是通过参数共享或可微传导。

三、“层”在两个系统中的含义不同（表征层）

Nested Learning 的“层”对应于模型中的计算单元，每一层有明确定义的输入、输出、目标函数。生物的“时间尺度分离”是一种连续的谱，不存在离散的“层 1、层 2、层 3”的划分。

读者不应从这个类比推出：

- **[X]** “Nested Learning 证明了大脑是嵌套优化”

- **[X]** "深度网络的层数对应大脑可塑性的层级数"
- **[X]** "模型的全局反向传播等价于神经调质的调控"
- **[X]** "取消固定架构就能构建类脑系统"

类比成立的层次：*算法组织上的相似*——两者都通过分离不同时间尺度的适应过程来同时处理 "立即响应" 和 "长期记忆" 的需求。**类比失效的层次**：实现机制、层间耦合方式、"层" 的离散性均不同。

11.11 和前后章节的关系

- **来自第 十 章**：Test-Time Regression 把第 四–九 章的所有模型统一到一个在线回归目标下。Nested Learning 在此基础上把 "统一" 推到了更极端的位置：不仅内层状态更新是优化，连 "网络架构本身" 也可以被解读为多层优化的截断。
- **来自第 三 章**：MAML 建立了二层嵌套优化的最基本形式。Nested Learning 是 MAML 向任意 L 层的推广。
- **全书总结**：这本书的旅程从 "测试时做一步梯度更新"（第 二 章的 TTT 2020），走到 "隐状态就是一个在线学习器"（第 八 章的 TTT-Layer），最终到 "" 所有的架构层级都可以被理解为嵌套的在线优化"。这条主线的一致性说明了：**训练和推断之间的界限不是黑白分明的，而是一个多时间尺度的连续谱**。

11.12 Lab 11：手写三层嵌套优化并测量截断效应

Lab 11：三层嵌套优化 vs. 固定网络的对比

目标：构建一个三层嵌套的在线回归系统，对比 "完整嵌套更新" 和 "截断为固定网络" 时的回归误差。

数据：

1. 生成 $T = 500$ 步的在线回归序列，$d = 16$。
2. 每 100 步切换一次目标函数的参数（模拟分布偏移），共 5 个 "任务段"。

三层嵌套系统：

1. **第 1 层（逐步更新）**：维护 $\boldsymbol{S}^{(1)} \in \mathbb{R}^{16\times16}$，每步做一步 SGD，学习率 η_1 由第 2 层提供。

2. **第 2 层（每 50 步更新）**：根据第 1 层最近 50 步的累积损失，调整 η_1（简单线搜索或梯度步）。
3. **第 3 层（固定）**：投影矩阵 W_K, W_V 预设为随机正交矩阵。

对比基线：

- 基线 A：固定 $\eta_1 = 0.01$，无第 2 层
- 基线 B：固定 $\boldsymbol{S}^{(1)} = W_{\text{pretrained}}$，无任何在线更新

测量：

- 在每个任务段的前 10 步和后 90 步分别记录 MSE
- 绘制 "时间步 vs. MSE" 曲线，标注任务切换点

预期观察：三层嵌套系统在任务切换后应该比两个基线更快恢复低误差，因为第 2 层能自适应地调整第 1 层的学习速度。

11.13 常见误解

常见误解

误解 1：既然层和架构只是优化的幻像，那么我们可以抛弃固定架构，用无限深的递归嵌套来获得无限能力。
纠正：这把 "数学视角的等价性" 误等同于 "计算资源的可替代性"。深度架构之所以采用固定层数和前向传播，是因为这在 GPU 上可以高效并行执行。无限深的嵌套优化在每一步都需要内层完全收敛才能向外层传播梯度——这在计算和数值稳定性上完全不可行。架构设计是**工程上必需的截断**，不是 "好看的装饰"。

常见误解

误解 2：Nested Learning 框架证明了我们已经完全理解了深度学习的所有涌现行为。
纠正：Nested Learning 在**机制层面**提供了一种统一的 "优化子过程的嵌套" 视角。但 "统一了底层运算的数学形式" 不等于 "解释了复杂行为的涌现"。这就像理解了蛋白质折叠的物理规则不等于理解了意识是如何产生的。对抗学习中的崩溃模式、多模态对齐的失效、推理能力的涌现等高层现象，至今没有被任何统一框架充分解释。

常见误解

误解 3：“这只是一种视角”意味着 Nested Learning 没有实际价值。
纠正：恰恰相反。Nested Learning 的最大价值在于它将“序列模型设计”从“搭积木试错”转化为“在一个统一目标函数下推导新变体”。第十章已经展示了这种方式的威力：通过修改目标函数的惩罚项和求解策略，可以系统地推导出新的模型层。这是方法论上的真正进步。

11.14 练习题

练习 11.1. 特例归约：从 Nested Learning 的通用更新公式 (11.3) 出发，通过指定 L、$\mathrm{Opt}^{(l)}$、$\ell^{(l)}$ 和截断策略，推导出以下模型的状态更新规则：(a) 快权重（$L=2$，外积写入）、(b) TTT-Linear（$L=2$，SGD with residual）、(c) Titans 的长时记忆（$L=2$，Surprise 门控 SGD）。

练习 11.2. 截断误差分析：设 $\ell(\boldsymbol{S},k,v)=\frac{1}{2}\|\boldsymbol{S}k-v\|_2^2$，$\boldsymbol{S}_0=0$。(a) 精确求解 $\boldsymbol{S}^*=\arg\min_{\boldsymbol{S}}\ell(\boldsymbol{S},k,v)$。(b) 计算一步 SGD 后的 $\boldsymbol{S}_1=-\eta\nabla\ell(0)$。(c) 计算截断误差 $\|\boldsymbol{S}^*-\boldsymbol{S}_1\|_F$，说明它何时较小（$\eta$ 接近什么值？k 须满足什么条件？）。

练习 11.3. 时间尺度的定量分析：在 Titans（第九章）中，STM（窗口注意力）的时间尺度约为 W（窗口大小），长时记忆的时间尺度约为 $1/(1-\lambda)$（有效记忆长度），持久记忆的时间尺度为 ∞（推断时不更新）。用 Nested Learning 的语言，把这三者对应到三层嵌套的第 1、2、3 层。它们各自的 $T^{(l)}$ 大约是多少？

练习 11.4. 诚实政策考核：写出一段 200 字以内的研究简报，向非专业同事介绍 Nested Learning。要求：必须包含至少一条“已被严格证明的机制等价”和至少一条“属于研究议程的视角主张”，并明确标注二者的区别。

11.15 本章小结

- **Nested Learning 的核心主张**：深度网络的层级结构可以被理解为多时间尺度嵌套优化的截断。“架构设计”等价于“选择每一层的优化算法和截断策略”。
- **形式化**：L 层嵌套优化由递推公式 (11.3) 和层间传导 (11.4) 定义。不同层操作在分离的时间尺度上。
- **统一特例**：快权重、线性注意力、TTT-Layer、Titans 都可以被归约为 $L=2$ 的 Nested Learning 在不同优化算法和截断策略下的特例。
- **学术边界**：机制层的公式等价（线性注意力 = 快权重 = 在线回归一步截断）已被严格证明。“架构是优化的幻像”是一个极具启发性的研究议程，但尚未在一般非线性情形下被完全证明。

- **神经科学类比**：大脑的多时间尺度可塑性层级在**算法组织层面**与 Nested Learning 的多层时间尺度相似；但实现机制（局部规则 vs. 全局梯度）、层间耦合方式、“层” 的离散性均有本质差异。
- **全书回顾**：从 TTT 2020（测试时做一步更新）到 Nested Learning（所有层级都是嵌套的优化），这条主线展示了：训练和推断之间的界限是一个连续谱，不同时间尺度的学习可以在同一个模型内共存。

读完本章并结合全书，你现在应该能够：

- 用 L 层嵌套优化的语言重新描述任意序列模型架构
- 把快权重、线性注意力、TTT-Layer 分别归约为 Nested Learning 的特例
- 区分 “已被严格证明的机制等价” 和 “尚在研究中的统一视角”
- 用多时间尺度的神经可塑性类比解释 Nested Learning 的设计动机，并**显式指出类比在哪个层面成立，在哪个层面失效**
- 独立评判一篇新的序列建模论文：它对应 Nested Learning 框架中的哪个参数选择？它是在改进优化算法还是在改变截断策略？

最低必要数学背景

本附录汇集了理解本书所需的核心数学工具。目标不是完整的线性代数或优化教材，而是让读者在遇到具体推导时可以快速查阅。最重要的提示：**如果你理解了外积和矩阵梯度，本书 80% 的公式就会变得自然**。

.1 线性代数核心

.1.1 外积（Outer Product）

向量 $a \in \mathbb{R}^m, b \in \mathbb{R}^n$ 的外积为矩阵：

$$ab^\top \in \mathbb{R}^{m\times n}, \quad (ab^\top)_{ij} = a_i b_j$$

核心性质：

- $ab^\top$ 是一个**秩 1 矩阵**——它的所有列都是 a 的标量倍。
- 对任意向量 c：$(ab^\top)c = a(b^\top c)$——右乘一个向量，结果是 a 方向上的投影，幅度由 $b^\top c$ 决定。

为什么外积在本书中反复出现：

1. **联想记忆**（第 四 章）：存储一对 (k, v) 到矩阵中：$\boldsymbol{S} \leftarrow \boldsymbol{S} + vk^\top$。读取时 $\boldsymbol{S}k' = v(k^\top k')$——如果 $k' \approx k$，输出 $\approx v$。
2. **梯度结构**：损失 $\ell = \|Wx - y\|^2$ 的梯度 $\nabla_W \ell$ 也是外积形式（见下一节）。
3. 这就是为什么 “快权重更新” 和 “梯度步” 在数学上长得一样。

.1.2 矩阵的 Frobenius 范数

$\|A\|_F = \sqrt{\sum_{i,j} A_{ij}^2} = \sqrt{\mathrm{tr}(A^\top A)}$。
在本书中，正则化项 $\frac{\gamma}{2}\|\boldsymbol{S}\|_F^2$ 惩罚状态矩阵的 “总能量”，防止无限增长。

.2 最小二乘与岭回归

.2.1 普通最小二乘（OLS）

给定数据矩阵 $X \in \mathbb{R}^{n\times d}$（$n$ 个样本，d 维特征）和目标 $Y \in \mathbb{R}^{n\times m}$，OLS 求解：

$$W^* = \arg\min_W \|XW - Y\|_F^2$$

正规方程（Normal Equations）：

$$W^* = (X^\top X)^{-1} X^\top Y \tag{9}$$

几何解释：XW^* 是 Y 在 X 列空间上的正交投影。$(X^\top X)^{-1}X^\top$ 是伪逆矩阵 X^+。

当 $X^\top X$ 不可逆（$n < d$ 或特征共线性）时，OLS 不唯一。

.2.2 岭回归（Ridge Regression）

加入 L_2 正则化解决 $X^\top X$ 的不可逆问题：

$$W^* = \arg\min_W \|XW - Y\|_F^2 + \gamma\|W\|_F^2 = (X^\top X + \gamma I)^{-1} X^\top Y \tag{10}$$

$\gamma > 0$ 保证 $X^\top X + \gamma I$ 总是可逆的。在本书中，第十章的 TTR 目标函数正是在线版本的岭回归。

.3 递归最小二乘（RLS）

在在线设定中，数据点逐个到来。目标：每次新增一个样本 (x_t, y_t) 时，不重新计算整个 $(X^\top X)^{-1}$，而是用 $O(d^2)$ 的代价增量更新。

.3.1 推导思路

设到时刻 $t-1$ 时，我们维护：

- 状态矩阵 $W_{t-1} = P_{t-1} \cdot (\text{累积} X^\top Y)$
- 精度矩阵 $P_{t-1} = (X_{1:t-1}^\top X_{1:t-1} + \gamma I)^{-1}$

新增样本 (x_t, y_t) 后：

$$P_t^{-1} = P_{t-1}^{-1} + x_t x_t^\top$$

这是一个**秩 1 更新**。用 Sherman-Morrison 公式求 P_t（见下一节），避免了 $O(d^3)$ 的矩阵求逆。

.3.2 RLS 更新公式

$$\kappa_t = \frac{P_{t-1}x_t}{1 + x_t^\top P_{t-1}x_t} \tag{11}$$

$$W_t = W_{t-1} + (y_t - W_{t-1}x_t)\kappa_t^\top \tag{12}$$

$$P_t = P_{t-1} - \kappa_t x_t^\top P_{t-1} \tag{13}$$

这里 κ_t 称为 **Kalman 增益**。$y_t - W_{t-1}x_t$ 是预测残差（prediction error）。

观察：式 (12) 的形式是 $W_t = W_{t-1} + (\text{残差}) \cdot (\text{方向})^\top$——又是一个外积更新！这与 TTT-Layer 的梯度步在结构上完全一致。

.4 Sherman-Morrison 公式

定理 .3 (Sherman-Morrison)**.** 设可逆矩阵 $A \in \mathbb{R}^{d\times d}$，向量 $u, v \in \mathbb{R}^d$，且 $1+v^\top A^{-1}u \neq 0$，则：

$$(A + uv^\top)^{-1} = A^{-1} - \frac{A^{-1}uv^\top A^{-1}}{1 + v^\top A^{-1}u} \tag{14}$$

推导要点

设 $B = A + uv^\top$，令 $B^{-1} = A^{-1} + C$（待定修正项）。

由 $BB^{-1} = I$：

$$(A + uv^\top)(A^{-1} + C) = I \tag{15}$$

$$I + AC + uv^\top A^{-1} + uv^\top C = I \tag{16}$$

$$AC = -uv^\top A^{-1} - uv^\top C \tag{17}$$

$$AC = -u(v^\top A^{-1} + v^\top C) \tag{18}$$

猜测 C 的形式为 $C = \alpha \cdot A^{-1}uv^\top A^{-1}$（秩 1），代入求解 $\alpha = -\frac{1}{1+v^\top A^{-1}u}$，即得证。

核心用途：在 RLS 中，$A = P_{t-1}^{-1}$，$u = v = x_t$。每次新增一个样本，只需 $O(d^2)$ 的矩阵向量乘法就能更新逆矩阵，无需 $O(d^3)$ 求逆。

.5 矩阵微积分基础

.5.1 标量对矩阵的梯度

设 $L(W) = \frac{1}{2}\|Wx - y\|_2^2$，其中 $W \in \mathbb{R}^{m\times d}$。

展开：

$$L(W) = \frac{1}{2}(Wx - y)^\top (Wx - y) \tag{19}$$

$$\nabla_W L = (Wx - y)x^\top \in \mathbb{R}^{m \times d} \tag{20}$$

这是一个**外积**：误差向量 $(Wx - y)$ 乘以输入 $x^\top$。这就是为什么本书中梯度步和快权重更新在代数上完全一样。

.5.2 链式法则（矩阵版本）

设 $L = g(f(W))$，其中 $f : \mathbb{R}^{m \times d} \to \mathbb{R}^p$，$g : \mathbb{R}^p \to \mathbb{R}$。则：

$$\frac{\partial L}{\partial W_{ij}} = \sum_k \frac{\partial g}{\partial f_k} \cdot \frac{\partial f_k}{\partial W_{ij}}$$

在 MAML（第三章）中，外循环梯度需要对“内循环结果相对于初始化参数”求导，这涉及的链式法则包含 Hessian 矩阵（见下一小节）。

.5.3 Jacobian 与 Hessian

Jacobian：$J = \frac{\partial f}{\partial x} \in \mathbb{R}^{m \times d}$，函数 $f : \mathbb{R}^d \to \mathbb{R}^m$ 的一阶导数矩阵。第 i 行 j 列是 $\frac{\partial f_i}{\partial x_j}$。

Hessian：$H = \frac{\partial^2 L}{\partial x^2} \in \mathbb{R}^{d \times d}$，标量函数 $L : \mathbb{R}^d \to \mathbb{R}$ 的二阶导数矩阵。

.5.4 预处理（Preconditioning）

标准梯度下降 $x \leftarrow x - \eta \nabla L$ 在高曲率方向上振荡。预处理梯度下降用 H^{-1}（或其近似）来校正：

$$x \leftarrow x - \eta H^{-1} \nabla L$$

这使得高曲率方向上的步长自动缩小，低曲率方向上自动放大。在第六章中，Transformer 的 FFN 被解释为在近似计算 H^{-1}，从而实现了隐式的预处理梯度下降。

.6 在线学习与遗憾（Regret）

.6.1 设定

在在线学习中，学习器与数据流交互：

1. 时步 t：学习器做预测 $\hat{y}_t$
2. 环境揭示真实值 y_t

3. 学习器承受损失 $\ell_t(\hat{y}_t)$ 并更新策略

.6.2 累积遗憾

$$\text{Regret}(T) = \sum_{t=1}^{T} \ell_t(\hat{y}_t) - \min_{y^*} \sum_{t=1}^{T} \ell_t(y^*) \tag{21}$$

第一项是学习器的累积损失，第二项是“事后诸葛亮最优”的累积损失。好的在线算法的遗憾增长率为 $O(\sqrt{T})$（次线性），意味着平均每步遗憾趋向零。

.6.3 Worked Example：Follow-the-Leader

设 $\ell_t(w) = (w - y_t)^2$，标量情形。Follow-the-Leader (FTL) 策略：$\hat{w}_t = \frac{1}{t-1}\sum_{i=1}^{t-1} y_i$（历史均值）。

对于 T 步，FTL 的遗憾为 $O(\log T)$——非常好，因为 $\log T \ll T$。

与本书的联系：线性注意力的外积累积 $\boldsymbol{S}_t = \sum_{i \le t} v_i k_i^\top$ 可以被视为一种加权的 Follow-the-Leader 策略。TTT-Layer 的在线 SGD 更新对应于 Online Gradient Descent，其遗憾为 $O(\sqrt{T})$。

.7 常用符号对照表

符号	含义	首次出现
θ	慢权重 / 全局参数	第 一 章
$\boldsymbol{S}_t$	快权重 / 隐状态矩阵	第 四 章
W_t^{inner}	内层学习器参数	第 七 章
k_t, v_t, q_t	键、值、查询向量	第 五 章
η	学习率	第 三 章
$\ell^{(l)}$	第 l 层局部目标函数	第 十一 章
$T^{(l)}$	第 l 层的时间尺度	第 十一 章
λ	遗忘因子 / 衰减率	第 五 章

Lab 汇总与动手实验指南

.8 环境准备

所有 Lab 均基于 Python 3.9+，以下依赖已经足够：

```
pip install torch numpy matplotlib  # 核心
pip install higher                  # Lab 3 (MAML) 可选
```

.9 Lab 索引

Lab	对应章节	核心概念
Lab 1	第一章	静态模型 OOD 脆弱性
Lab 2	第二章	分布偏移 + TTT 自监督微调
Lab 3	第三章	MAML 正弦波
Lab 4	第四章	外积联想记忆
Lab 5	第五章	线性注意力递推 vs 朴素版
Lab 6	第六章	ICL 与最小二乘对比
Lab 7	第七章	TTT-RNN 单元（NumPy）
Lab 8	第八章	TTT-Linear vs GRU
Lab 9	第九章	Needle-in-Haystack 混合记忆
Lab 10	第十章	在线 OLS 与 TTT-Linear 对比
Lab 11	第十一章	NL 对应表（开放性思考）

.10 Lab 设计原则

每个 Lab 的设计遵循以下约束：

1. 代码不超过 200 行（含注释）
2. 运行时间不超过 5 分钟（CPU）
3. 有明确的” 预期观察”——学生应该知道自己在验证什么

4. 失败或非预期结果同样值得分析——Lab 不是黑盒验证工具

.11 各 Lab 详细指南

.11.1 Lab 1：静态参数模型的 OOD 脆弱性

章节：第 一 章 **核心验证**：固定参数模型在分布偏移下的失败

- **数据**：二维分类，训练在 $[-1,1]^2$，测试平移至 $[49,51]^2$。
- **预期结果**：模型在训练域 $> 99\%$ 准确率，平移后降至 $\sim 50\%$（接近随机猜测）。1 步 SGD 自适应后恢复至 $> 95\%$。
- **预期图表**：柱状图对比三种条件的准确率。
- **常见问题**：如果平移后准确率仍然高，检查是否使用了 BatchNorm（它会自适应均值）。
- **失败意味着什么**：如果 1 步 SGD 没有改善，可能是学习率太大（过冲）或辅助损失选择不当。

.11.2 Lab 2：TTT 2020 自监督旋转预测

章节：第 二 章 **核心验证**：旋转预测损失能间接改善分类

- **数据**：CIFAR-10 的子集（100 张），测试图加高斯噪声模拟分布偏移。
- **预期结果**：基线准确率在噪声下降至 $\sim 40\%$，TTT 自适应后恢复至 ~ 55–65%。
- **注意**：只更新 θ_e（特征提取器），冻结 θ_m（分类头）和 θ_s（旋转头）。

.11.3 Lab 3：MAML 正弦波回归

章节：第 三 章 **核心验证**：MAML 学到的初始化点能在 1–5 步内适应新任务

- **任务族**：$y = A\sin(x+\phi)$，$A \sim \text{Uniform}(0.1, 5.0)$，$\phi \sim \text{Uniform}(0, \pi)$。
- **预期结果**：随机初始化模型需要 > 50 步适应；MAML 初始化后 1–5 步即可拟合新的 (A, ϕ)。
- **预期图表**：对比 “pre-adaptation” 和 “post-adaptation (1 step)” 的拟合曲线。
- **常见问题**：如果 MAML 性能不如随机，检查 α（内循环学习率）是否太大。推荐 $\alpha = 0.01$，$\beta = 0.001$。

.11.4 Lab 4：外积联想记忆

章节：第 四 章　**核心验证**：外积存储的容量限制和干扰效应

- **实验**：存储 M 对随机正交键值对到 $\boldsymbol{S} = \sum v_i k_i^\top$，变化 M 从 1 到 $2d$。
- **预期结果**：$M \leq d$ 时精确回忆（误差 $< 10^{-6}$）；$M > d$ 时干扰急剧增加。
- **拓展**：用非正交键重复实验，观察 $M \ll d$ 时就出现干扰。

.11.5 Lab 5：线性注意力与 Softmax 注意力对比

章节：第 五 章　**核心验证**：线性注意力的递推公式与朴素实现一致

- **实验**：生成随机序列，分别用朴素 $O(N^2)$ 实现和递推 $O(d^2)$ 实现计算线性注意力，验证结果一致（浮点误差 $< 10^{-5}$）。
- **预期图表**：运行时间 vs 序列长度 N，朴素版本呈二次增长，递推版本线性增长。

.11.6 Lab 6：ICL 的隐式梯度下降验证

章节：第 六 章　**核心验证**：一层线性注意力的 ICL 输出与 OLS 结果对比

- **实验**：构造线性回归任务 $y = W^* x + \epsilon$，提供 n 个上下文示例，比较注意力输出与 OLS 解 $\hat{W} = (X^\top X)^{-1} X^\top Y$。
- **预期结果**：当 W_K, W_V 按定理 6.1 的构造设定时，注意力输出与一步 GD 结果完全一致。
- **拓展**：用训练过的（而非手工构造的）权重重复，观察行为相似性但非严格等价。

.11.7 Lab 7：TTT-RNN 单元

章节：第 七 章　**核心验证**：用 NumPy 手写一个最简 TTT 更新单元

- **实验**：实现单步 $\boldsymbol{S}_t = \boldsymbol{S}_{t-1} - \eta((\boldsymbol{S}_{t-1} k_t - v_t) k_t^\top)$，在合成序列上验证回忆能力。
- **预期结果**：前 d 个正交键值对可被近似回忆。

.11.8 Lab 8：TTT-Linear vs GRU 遗忘对比

章节：第 八 章　**核心验证**：TTT-Linear 的 $O(d^2)$ 容量优于 GRU 的 $O(d)$

- **实验**：序列长 $N = 50$，$d = 32$。在 $t = 10$ 存入特殊键值对，在 $t = 50$ 查询。对比 TTT-Linear 和 GRU 的回忆质量。
- **预期结果**：TTT-Linear 在 $t = 50$ 仍有可测的回忆，GRU 的回忆因向量压缩而更快衰退。
- **拓展**：加入遗忘因子 $\lambda = 0.98$，观察遗忘曲线变化。

.11.9 Lab 9：Needle-in-Haystack 混合记忆

章节：第 九 章　**核心验证**：Surprise 门控保护关键信息

- **实验**：在 $N = 1000$ 的序列中，位置 $t = 50$ 放置一条高价值信息，其余为噪声。对比有无 Surprise 门控的回忆效果。
- **预期结果**：无门控时，$t = 1000$ 处查询无法回忆；有门控时，查询仍能检索到关键信息。
- **预期图表**："时间步 vs 回忆 MSE" 曲线，标注门控开/关。

.11.10 Lab 10：在线 OLS 与 TTT-Linear 对比

章节：第 十 章　**核心验证**：TTR 框架下两种求解策略的行为差异

- **实验**：在相同的在线回归序列上，分别运行 RLS（精确解）和 TTT-Linear（一步 SGD 近似），对比每步的 MSE。
- **预期结果**：RLS 永远不劣于 TTT-Linear。差距在键非正交时更明显。
- **拓展**：实现线性注意力（忽略预测反馈），对比三条 MSE 曲线。

.11.11 Lab 11：Nested Learning 对应表

章节：第 十一 章　**性质**：开放性思考实验

- **任务**：构建三层嵌套优化系统（第 1 层逐步更新，第 2 层每 50 步调整学习率，第 3 层预设投影矩阵）。
- **预期观察**：三层系统在任务切换后恢复快于固定学习率基线。
- **注意**：这是一个概念验证实验，结果的绝对值不重要。重要的是学生能够用代码构建 "嵌套优化" 并理解每一层的角色。

术语、符号与神经科学对照表

为防止跨章阅读时的混淆，并在大量使用神经科学辅助直觉时保持学术严谨边界感，本附录提供三部分内容：核心数学符号表、机器学习 ↔ 神经科学术语对照表，以及一份”容易混同但绝不能混同”的概念列表。

.12 核心数学符号

符号	名称	含义与注意事项
θ	慢权重 (Slow Weights)	通过大规模数据集上的离线训练优化。推断时固定不变。跨任务、跨序列的通用知识载体。
$\boldsymbol{W}_t, \boldsymbol{S}_t, \Phi_t$	快权重 / 内层状态 (Fast Weights)	在推断时由当前序列输入动态更新。生命周期随当前序列。不是通过离线梯度下降，而是通过显式写入规则（外积、梯度步）更新。
$\mathcal{L}_{\text{outer}}$	外层损失	控制慢权重更新方向。
$\mathcal{L}_{\text{inner}}, \ell_t$	内层损失	在推断期间自构造的局部损失，驱动快权重更新。
s_t	惊讶度 (Surprise)	当前输入与模型预测的误差幅度，用于门控快权重更新强度。数学定义：$s_t = \sigma(\|\hat{v}_t - v_t\|_2 - b)$。不等于人类的”惊讶”情绪体验。
λ	遗忘因子 (Forget Factor)	指数衰减系数，$\lambda \in (0, 1]$。$\lambda = 1$ 为均等历史。
$\boldsymbol{k}_t, \boldsymbol{v}_t, \boldsymbol{q}_t$	键、值、查询 (Key, Value, Query)	联想记忆的标准三元组。逐章含义一致。

.13 机器学习术语 vs. 神经科学类比术语对照

以下每对术语均标明：**共同功能**（类比为何有帮助）和**不能等同原因**（类比在哪里失效）。

对 1：上下文窗口隐状态 vs. 工作记忆

AI 概念	神经科学类比	边界
Context Window State（上下文窗口隐状态）	Working Memory（工作记忆）	**共同功能**：两者都短暂持有当前"在用"的信息，容量有限，用完即释放。**不能等同**：生物工作记忆依赖前额叶皮层的持续神经振荡和多脑区同步；模型是被动持有过去 token 的键值张量，无主动维护机制，无容量的生理约束。

对 2：内层参数梯度更新 vs. 突触可塑性（LTP）

AI 概念	神经科学类比	边界
Inner-loop parameter update（内层参数梯度更新）	Synaptic Plasticity / LTP（突触长时增强）	**共同功能**：两者都使系统根据输入调整内部连接强度。**不能等同**：梯度更新需要精确计算并沿整个计算图反向传播；突触 LTP 由局部电位、钙离子内流和时序相关性驱动（STDP），是高度局部的非精确过程。

对 3：长期神经记忆模块 vs. 系统级皮层巩固

AI 概念	神经科学类比	边界
Long-Term Neural Memory Module（如 Titans 的慢层）	Systems Consolidation（系统级皮层巩固）	**共同功能**：两者都用于长期存储，更新频率低，相对稳定，不受单次输入干扰。**不能等同**：生物皮层巩固历经睡眠中的多次海马体重放，跨脑区物理迁移，持续数天；模型只是降低了一个参数矩阵的学习率，无时间维度和物理迁移过程。

对 4：Transformer Attention vs. 选择性注意

AI 概念	神经科学类比	边界
Transformer Attention	Selective Attention（选择性注意）	**共同功能**：两者都对不同信息来源分配不同权重，实现”相关信息优先”。**不能等同**：模型 Attention 是对所有 token 的穷举点积，是精确可微的数学运算；生物注意力伴随眼跳、感知-行动耦合、执行功能调控，具有显著的具身性和生存动机驱动，不可简化为加权求和。

对 5：快权重外积写入 vs. Hebbian 学习

AI 概念	神经科学类比	边界
Fast Weight outer product update $\boldsymbol{W} \leftarrow \boldsymbol{W} + vk^\top$	Hebb’s Rule（赫布规则）”同时激发的神经元，连接增强”	**共同功能**：两者都通过”共同出现”来增强连接，在矩阵语言中有形式对应。**不能等同**：Hebb 规则无遗忘，无全局目标，不保证收敛；外积更新是精确矩阵代数，配合遗忘因子 λ 和投影矩阵完成工程精确控制。

对 6：Surprise 门控 vs. 新异刺激驱动的突触强化

AI 概念	神经科学类比	边界
Surprise-gated memory write（惊讶度门控写入）	新异刺激（Novelty）驱动的突触强化；多巴胺奖励预测误差（RPE）	**共同功能**：两者都用”预测误差大”作为触发强记忆写入的信号，对高可预测输入保持保守。**不能等同**：Surprise $s_t = \sigma(\|\hat{v}_t - v_t\|_2)$ 是单一标量，由模型当前预测误差决定；生物的多巴胺信号来自复杂的皮层-纹状体回路，受期望、奖励历史、上下文共同调制，不可简化为一个误差范数。

对 7：外层/内层双循环 vs. 快/慢双速学习

AI 概念	神经科学类比	边界
Outer-loop / Inner-loop（双层优化）	互补学习系统（CLS）理论：海马体（快）+ 新皮质（慢）	**共同功能**（算法组织层）：两者都有快速适应系统和缓慢巩固系统的分工。**不能等同**：CLS 中海马体和新皮质是解剖实体，通过睡眠回放进行信息迁移；outer/inner loop 是训练算法的两阶段，没有对应的物理分离或时序信息迁移过程。

对 8：批次梯度下降 vs. 经验回放/系统巩固

AI 概念	神经科学类比	边界
Mini-batch training（批量训练）	Hippocampal Replay during sleep（睡眠期海马体重放）	**共同功能**：两者都通过对过去经验的”再次处理”来强化已学内容。**不能等同**：批量训练是人为构造的随机采样；海马体重放在睡眠期自发产生，具有时序压缩和跨脑区协调特性，且重放的是压缩后的模式而非原始感觉数据。

对 9：TTT-Layer 写入 vs. 任务内可塑性

AI 概念	神经科学类比	边界
TTT-Layer inner update（TTT 层内循环更新）	Within-episode fast plasticity（任务内快速可塑性）	**共同功能**（计算角色）：两者都在一次任务执行过程中实时调整内部状态，不依赖跨任务的长期训练。**不能等同**：TTT 更新是有明确损失目标的梯度步，通过标准反向传播实现；生物任务内可塑性依赖局部突触修饰，受乙酰胆碱等神经调质调控，无全局损失目标，无反向传播。

对 10：Softmax Attention 的 softmax 归一化 vs. 侧抑制

AI 概念	神经科学类比	边界
Softmax normalization in Attention	Lateral Inhibition（侧向抑制）	**共同功能**：两者都使”赢家”的激活被相对放大，”输家”被相对抑制，形成竞争性选择。**不能等同**：Softmax 是全局精确的指数归一化，对所有位置同时计算；侧向抑制是局部的皮层电路现象，受解剖距离、抑制性神经元类型的制约，不是精确的全局归一化。

对 11：在线学习（Online Learning）vs. 突触元可塑性（Metaplasticity）

AI 概念	神经科学类比	边界
Online Learning Rate Scheduling	Metaplasticity（突触元可塑性）：BCM 规则中 θ_M 的动态调整	**共同功能**（算法组织）：两者都根据近期活动历史动态调整”可学性”——近期学了很多则降低可塑性，近期接受较少则提高可塑性。**不能等同**：学习率调度是工程超参数；BCM 规则的 θ_M 是由每个突触后神经元的历史活动均值确定的生物变量，逐神经元独立维护。Adam 的二阶矩类似 BCM 的精神，但粒度（per-scalar）和机制（移动平均 vs 生化级联）完全不同。

对 12：Nested Learning 多层优化 vs. 多时间尺度神经活动

AI 概念	神经科学类比	边界
Nested / Multi-level Optimization（嵌套多层优化）	Multi-timescale neural dynamics：neural firing → synaptic change → consolidation → development	**共同功能**（算法组织）：两者都有多个运行在不同时间尺度上的更新层，慢层约束快层，快层适应当前输入。**不能等同**：嵌套优化是形式上的统一视角，是一种数学建模语言；神经系统的多时间尺度协调是真实存在于不同生理过程中的，不应被直接”实例化”为嵌套优化的证据。

.14 容易混同但绝不能混同的概念

- **Working Memory vs. Context Window**：工作记忆是主动维护的（需要持续神经活动），Context Window 是被动的（KV Cache 无需持续维护）。
- **Synaptic Plasticity vs. Parameter Update**：突触可塑性是局部化学过程；参数更新是全局梯度反向传播。两者的学习信号来源、实现机制、时间尺度均不同。

- **Systems Consolidation vs. Long-term Neural Memory Module**：系统巩固是跨脑区、跨天级别的物理重组过程；长期神经记忆模块只是一个更新频率较低的参数矩阵。
- **Selective Attention vs. Transformer Attention**：生物选择性注意涉及执行控制、眼动、具身性；Transformer Attention 是对所有位置的穷举点积，无”主动选择”过程。
- **Hippocampus vs. Fast-updating Component**：海马体负责情景记忆编码和索引化，不是简单的”短期记忆”；快速更新组件（TTT-Layer、LoRA）只是更新频率快，不具有海马体的稀疏编码、索引化等特定功能。

.15 学术诚实申明

如果你在其他文献中看到”此模型通过模仿人脑的 XX 机制实现了突破”，请对照本书的类比分级框架重新审视：那个类比属于哪一层相似性？作者是否明确了不同之处？

工程与生物是从不同山脚攀登”智能”高峰的两条路径。本书借用神经科学的望远镜来获取直觉和方向感，但脚下走的永远是由梯度、矩阵运算和损失函数构成的工程之路。

参考文献

Ahn, Kwangjun, Xiang Cheng, Hadi Daneshmand, and Suvrit Sra (2023). "Transformers learn to implement preconditioned gradient descent for in-context learning". In: URL: https://arxiv.org/abs/2306.00297.

Andrychowicz, Marcin, Misha Denil, Sergio Gomez, Matthew W. Hoffman, David Pfau, Tom Schaul, Brendan Shillingford, and Nando de Freitas (2016). "Learning to learn by gradient descent by gradient descent". In: *Advances in Neural Information Processing Systems (NeurIPS)*. URL: https://arxiv.org/abs/1606.04474.

Ba, Jimmy, Geoffrey E. Hinton, Volodymyr Mnih, Joel Z. Leibo, and Catalin Ionescu (2016). "Using Fast Weights to Attend to the Recent Past". In: *Advances in Neural Information Processing Systems (NeurIPS)*. URL: https://arxiv.org/abs/1610.06258.

Behrouz, Ali et al. (2025a). "It's All Connected: A Journey Through Test-Time Memorization, Attentional Bias, Retention, and Online Optimization". In: *arXiv preprint arXiv:2504.13173*. URL: https://arxiv.org/abs/2504.13173.

Behrouz, Ali, Peilin Zhong, and Vahab Mirrokni (2025b). "Nested Learning: The Illusion of Deep Learning Architectures". In: *arXiv preprint arXiv:2512.24695*. URL: https://arxiv.org/abs/2512.24695.

— (2025c). "Titans: Learning to Memorize at Test Time". In: *arXiv preprint arXiv:2501.00663*. URL: https://arxiv.org/abs/2501.00663.

Garg, Shivam, Dimitris Tsipras, Percy Liang, and Gregory Valiant (2022). "What Can Transformers Learn In-Context? A Case Study of Simple Function Classes". In: URL: https://arxiv.org/abs/2208.01066.

Ha, David, Andrew Dai, and Quoc V. Le (2016). "HyperNetworks". In: *arXiv preprint arXiv:1609.09106*. URL: https://arxiv.org/abs/1609.09106.

Katharopoulos, Angelos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret (2020). "Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention". In: *Proceedings of the 37th International Conference on Machine Learning (ICML)*. URL: https://arxiv.org/abs/2006.16236.

Krause, Ben, Emmanuel Kahembwe, Iain Murray, and Steve Renals (2018). "Dynamic Evaluation of Neural Sequence Models". In: *Proceedings of the 35th International*

Conference on Machine Learning (ICML). URL: https://arxiv.org/abs/1709.07432.

Nichol, Alex, Joshua Achiam, and John Schulman (2018). "On First-Order Meta-Learning Algorithms". In: *arXiv preprint arXiv:1803.02999*. URL: https://arxiv.org/abs/1803.02999.

Olsson, Catherine, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah (2022). "In-context Learning and Induction Heads". In: *Transformer Circuits Thread*. URL: https://arxiv.org/abs/2209.11895.

Oswald, Johannes von, Eyvind Niklasson, Ettore Randazzo, João Sacramento, Alexander Mordvintsev, Andrey Zhmoginov, and Max Vladymyrov (2023a). "Transformers learn in-context by gradient descent". In: *Proceedings of the 40th International Conference on Machine Learning (ICML)*. URL: https://arxiv.org/abs/2212.07677.

— (2023b). "Transformers learn in-context by gradient descent". In: URL: https://arxiv.org/abs/2212.07677.

Santoro, Adam, Sergey Bartunov, Matthew Botvinick, Daan Wierstra, and Timothy Lillicrap (2016). "Meta-Learning with Memory-Augmented Neural Networks". In: *Proceedings of the 33rd International Conference on Machine Learning (ICML)*. URL: https://proceedings.mlr.press/v48/santoro16.html.

Schlag, Imanol, Kazuki Irie, and J"urgen Schmidhuber (2021). "Linear Transformers Are Secretly Fast Weight Programmers". In: *Proceedings of the 38th International Conference on Machine Learning (ICML)*. URL: https://arxiv.org/abs/2102.11174.

Sun, Yu et al. (2025). "Test-Time Training Done Right". In: *arXiv preprint arXiv:2505.23884*. URL: https://arxiv.org/abs/2505.23884.

Sun, Yu, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Garg, Genghan Zhang, Yann LeCun, and Xiaolong Wang (2023). "Learning to (Learn at Test Time): RNNs with Expressive Hidden States". In: *arXiv preprint arXiv:2310.13807*. URL: https://arxiv.org/abs/2310.13807.

— (2024a). "Learning to (Learn at Test Time): RNNs with Expressive Hidden States". In: *arXiv preprint arXiv:2407.04620*. URL: https://arxiv.org/abs/2407.04620.

— (2024b). "Learning to (Learn at Test Time): RNNs with Expressive Hidden States". In: *arXiv preprint arXiv:2407.04620*. URL: https://arxiv.org/abs/2407.04620.

Sun, Yu, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt (2020). "Test-Time Training with Self-Supervision for Generalization under Distribution Shifts". In: *Proceedings of the 37th International Conference on Machine Learning (ICML)*. URL: https://arxiv.org/abs/1909.13231.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin (2017). "Attention Is All You Need". In: *Advances in Neural Information Processing Systems (NeurIPS)*. URL: https://arxiv.org/abs/1706.03762.

Xiong, Zhen et al. (2025). "Test-time regression: a unifying framework for designing sequence models with associative memory". In: *arXiv preprint arXiv:2501.12352*. URL: https://arxiv.org/abs/2501.12352.

Zhang, Ruiqi, Spencer Frei, and Peter L. Bartlett (2024). "Trained Transformers Learn Linear Models In-Context". In: *Journal of Machine Learning Research* 25, pp. 1–55. URL: https://arxiv.org/abs/2306.09927.